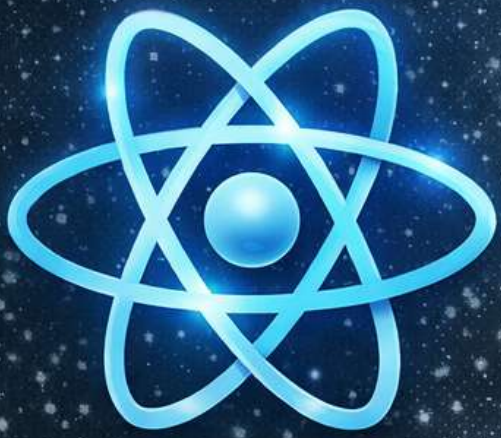

ReactJS INTERVIEW

MASTER BOOK

Ace Your React Interviews with 150+ Advanced Questions

Upamanyu Deka

1) What is React and why is it popular?

What it is

React is a JavaScript library for building user interfaces (UIs), especially single-page applications where the UI changes frequently without reloading the page. React focuses on the “View” layer: rendering UI based on state.

Why it exists

Before React, many apps manually manipulated the DOM (e.g., document.querySelector(...).innerHTML = ...). That approach becomes:

- hard to maintain as apps grow
- prone to bugs (UI state out of sync)
- inefficient (too many DOM updates)

React’s core promise: “Describe what the UI should look like for a given state, and React takes care of updates.”

Why it’s popular (practical reasons)

- Component model: Breaks UI into reusable, testable pieces.
- Declarative UI: You express UI as a function of state.
- Efficient updates: React minimizes DOM changes via reconciliation.
- Huge ecosystem: Router, state management, UI libs, Next.js, etc.
- Cross-platform: React Native extends the mental model to mobile.

How to explain in an interview (clean)

“React is a UI library that uses components and declarative rendering. You update state, React reconciles changes and updates the DOM efficiently. It scales well because components are reusable and the ecosystem is strong.”

2) What are components in React?

What they are

A component is a reusable UI unit. In modern React, a component is usually a function that returns JSX.

Think

Component = UI + logic + state (optional) + lifecycle effects (optional)

Types (interview-friendly)

- Function components (recommended, modern)
- Class components (older; still exists in legacy code)

Example

```
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
```

Upamanyu Deka

Why components matter

- Reuse (same button everywhere)
- Separation of concerns (compose UI like LEGO)
- Better testing (unit test component behavior)
- Scalability (split work across teams/features)

Common mistake

Treating components like templates only. In React, components can hold state, handle events, and manage side effects via hooks.

3) Difference between class and functional components?

Core difference

- Class components use ES6 classes and lifecycle methods (componentDidMount, etc.).
- Function components use functions + hooks (useEffect, useState) for state and lifecycle.

State handling

Class

```
class Counter extends React.Component {
  state = { count: 0 };
  render() { return <div>{this.state.count}</div>; }
}
```

Function

```
function Counter() {
  const [count, setCount] = React.useState(0);
  return <div>{count}</div>;
}
```

Lifecycle / side effects

- Class: lifecycle methods (componentDidMount, componentDidUpdate, componentWillUnmount)
- Function: useEffect (and useLayoutEffect when needed)

Why functional is preferred today

- Less boilerplate
- Easier composition via custom hooks
- Better alignment with React's direction (Concurrent features + modern patterns)
- Cleaner reuse of stateful logic (hooks > HOCs/render props in most cases)

Interview summary line

"Function components with hooks are the modern standard. Class components are legacy but still relevant to understand lifecycle equivalents."

4) What is JSX and how does it work?

What JSX is

JSX is a syntax extension that lets you write HTML-like code in JavaScript:

```
const el = <h1>Hello</h1>;
```

How it works internally

Upamanyu Deka

JSX is not HTML. It is compiled (by Babel/TypeScript) into JavaScript calls that create React elements.

Conceptually, this:

```
<h1 className="title">Hello</h1>
```

Becomes something like:

```
React.createElement("h1", { className: "title" }, "Hello");
```

React element vs DOM node

A React element is a lightweight object describing what you want on screen. React later uses this to update the real DOM.

JSX rules you must know

- Must return one parent (or fragment <>...)
- Use className not class
- Use {} to embed JS expressions
- Attributes are camelCased: onClick, tabIndex

Interview explanation

“JSX is syntactic sugar that compiles to React element creation. React uses those elements to build a virtual tree and reconcile updates.”

5) What are props in React?

What they are

Props (properties) are inputs passed from a parent component to a child component. They are read-only from the child’s perspective.

```
function Avatar({ url, size }) {
  return <img src={url} width={size} height={size} />;
}
```

Why props exist

They enable:

- configuration (same component, different data)
- reusability (component works for many scenarios)
- one-way data flow (predictability)

Important: props are immutable (from child)

A child should not modify props. If something must change, it should be represented as state in the owning component, and updated via callbacks.

Common pattern: “pass data down, pass actions up”

```
function Parent() {
  const [name, setName] = React.useState("Ava");
  return <Child name={name} onChangeName={setName} />;
}

function Child({ name, onChangeName }) {
  return (
    <button onClick={() => onChangeName("Mia")}>
      Current: {name}
    </button>
```

Upamanyu Deka

```
  );
}
```

Interview line

“Props are immutable inputs to a component. They support one-way data flow and help keep UI predictable.”

6) What is state and how is it used?

What state is

State is data that belongs to a component (or shared owner) that can change over time and should trigger a UI update.

How it’s used

When state changes via its setter (e.g., setCount), React schedules a re-render.

```
function Counter() {
  const [count, setCount] = React.useState(0);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}
```

Key concept: state updates cause re-render

React re-runs the component function to compute the next UI output.

Best practices

- Keep state minimal (avoid storing derived values)
- Treat state as immutable (don’t mutate arrays/objects in place)
- Use functional updates when next state depends on previous state

Interview framing

“State holds UI-changing data. Updating state triggers a re-render, and React reconciles DOM changes efficiently.”

7) Difference between props and state?

Quick comparison

- Props: inputs passed from parent → child, read-only for child
- State: internal data managed by component (or owner), changes via setters

Ownership

- Props are owned by parent
- State is owned by the component that defines it

Example mental model

If a component is a “function,” then:

- Props are function parameters
- State is internal memory

Upamanyu Deka

Example

```
function ProductCard({ product }) {  // product is props
  const [expanded, setExpanded] = React.useState(false); // expanded is state
  // ...
}
```

Common interview pitfall

People say “state is mutable.” Correct: You don’t mutate state directly; you replace it with a new value via setters.

8) What are React hooks?

What hooks are

Hooks are functions that let you use React features (state, lifecycle, context, refs) in function components.

Examples

- useState → component state
- useEffect → side effects
- useContext → consume context
- useRef → mutable refs without re-render
- useMemo, useCallback → memoization
- useReducer → structured state transitions

Why hooks exist

They solve classic problems:

- Reusing stateful logic without HOCs/render props complexity
- Making components smaller and easier to reason about
- Encouraging composition

Rules of hooks (must know)

- Call hooks only at the top level (not in loops/conditions)
- Call hooks only from React functions (components or custom hooks)

Reason: React relies on call order to map hook calls to internal state slots.

Interview line

“Hooks bring state and lifecycle features to function components and enable reusable logic via custom hooks, with strict call-order rules.”

9) Explain useState hook with example.

What it does

useState lets a function component store state and update it.

```
const [value, setValue] = useState(initialValue);

// value: current state
// setValue: updater function
// initialValue: initial state (or initializer function)
```

Example: controlled input

Upamanyu Deka

```
function NameForm() {
  const [name, setName] = React.useState("");

  return (
    <div>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Type your name"
      />
      <p>Hello, {name || "stranger"}!</p>
    </div>
  );
}
```

Important details (mid → senior)

- State update is async-ish: React batches updates.
- Functional update avoids stale state: setCount(prev => prev + 1);
- Lazy initialization (performance): const [items] = React.useState(() => expensiveInit());

Common mistakes

Mutating objects/arrays:

```
state.list.push(x) // ■
setState(state)    // ■
```

Correct:

```
setList(prev => [...prev, x]) // ■
```

10) Explain useEffect hook and its dependencies.

What useEffect is for

useEffect runs side effects after rendering:

- API calls
- subscriptions
- timers
- DOM integrations (usually; sometimes useLayoutEffect)

Signature

```
useEffect(effectFn, dependencies?)
```

Core mental model

- Render computes UI
- Effect runs after paint (generally)
- Cleanup runs before next effect (or on unmount)

Three dependency patterns

A) No dependency array → runs after every render

```
React.useEffect(() => {
  console.log("Runs after every render");
});
```

Use rarely; can be expensive.

Upamanyu Deka

B) Empty array [] → runs once after mount (and cleanup on unmount)

```
React.useEffect(() => {
  const id = setInterval(() => console.log("tick"), 1000);
  return () => clearInterval(id);
}, []);
```

C) With dependencies [x, y] → runs when any dependency changes

```
function UserProfile({ userId }) {
  const [user, setUser] = React.useState(null);

  React.useEffect(() => {
    let cancelled = false;

    async function load() {
      const res = await fetch(`/api/users/${userId}`);
      const data = await res.json();
      if (!cancelled) setUser(data);
    }

    load();
    return () => { cancelled = true; };
  }, [userId]);

  return <pre>{JSON.stringify(user, null, 2)}</pre>;
}
```

How dependencies work (important)

React compares dependencies by reference (shallow equality using Object.is).

- primitives: usually fine
- objects/functions created inline: change every render unless memoized

Common interview traps

- Missing dependencies causes stale closures (buggy logic).
- Adding everything blindly can cause loops.

Fixes

- Move functions inside effect
- Use useCallback / useMemo when necessary
- Use functional updates for state

StrictMode note (React 18 dev)

In dev with StrictMode, effects may run twice to help surface unsafe patterns. This is expected during development.

Upamanyu Deka

11) What are controlled components?

What they are

Controlled components are form elements whose displayed value is driven entirely by React state. React becomes the “single source of truth” for the form value, and the DOM is essentially just the rendering target. This means what the user sees in the input is always equal to your component state.

How it works (mechanics)

You bind the input’s value to a state variable and update that state in an onChange handler. Every keystroke triggers onChange → setState → re-render → the value prop is re-applied to the input.

```
function ControlledInput() {
  const [email, setEmail] = React.useState("");

  return (
    <label>
      Email:
      <input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="you@example.com"
      />
    </label>
  );
}
```

Why they matter (real-world reasons)

- Validation becomes straightforward (sync or async). You can show errors as the user types.
- You can enforce formatting (e.g., uppercase, trimming, masking, allowed characters).
- You can derive UI from state (disable submit if invalid, show character count, etc.).
- You can reliably prefill/reset forms (set state to initial values).

Common pitfalls

- Performance: very large forms can re-render frequently. Mitigate with component splitting, memoization, or libraries (React Hook Form).
- Stale state issues when computing next state from previous state (use functional updates).
- Cursor-jump bugs when you transform value on every keystroke (masking). You may need to preserve selection positions.

Interview framing

“Controlled components store form data in React state, which makes validation, conditional UI, and resets predictable because state is the single source of truth.”

12) What are uncontrolled components?

What they are

Upamanyu Deka

Uncontrolled components are form elements where the DOM maintains the current value internally, and React reads that value only when needed (usually on submit). Instead of binding `value` to state, you typically use `defaultValue` and access the live value using a ref.

How it works

Because React does not re-apply the input's value on every render, the input behaves like a normal HTML input. You read the current value by referencing the DOM node (ref).

```
function UncontrolledForm() {
  const inputRef = React.useRef(null);

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(inputRef.current?.value);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={inputRef} defaultValue="initial text" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

When to use (practical)

- Quick/simple forms where you only need the value at submit time.
- Integrating with non-React code or existing DOM-based form libraries.
- Performance-sensitive scenarios where you want to avoid re-render per keystroke (though modern patterns often handle this well).

Trade-offs / drawbacks

- Harder to do real-time validation and conditional UI based on current input.
- State and UI can drift if you also store related data in React state.
- Testing and debugging can be harder because the single source of truth is in the DOM.

Interview line

"Uncontrolled components let the DOM own the input value; React reads it via refs, typically on submit."

13) What is conditional rendering?

What it is

Conditional rendering means rendering different UI branches based on current state/props. Because React rendering is just JavaScript, you can use normal JS control flow to choose what to return.

Common patterns (and when to prefer them)

- Logical &&: best for showing something only when a condition is true.
- Ternary ?: best when there are two UI branches (if/else).
- Early return: best when a condition changes the entire component output (loading/error states).

```
function Profile({ user }) {
  if (!user) return <p>Please log in</p>; // early return
  return (
    <div>
      <h2>{user.name}</h2>
      {user.isAdmin && <span>Admin</span>} {/* && */}
```

Upamanyu Deka

```
      {user.plan === "pro" ? <ProBadge /> : <FreeBadge />} {/* ternary */}
    </div>
  );
}
```

Why it matters

Most product UIs are state-driven: loading states, error states, permissions, feature flags, experiments, and responsive layouts. A strong React engineer uses conditional rendering to keep UI consistent with business logic.

Common interview trap

Rendering 'nothing' incorrectly. In React you can return `null` to render nothing, or conditionally skip rendering a piece with `&&`.

Interview line

“Conditional rendering is rendering different UI based on state/props using JavaScript expressions like ternaries, &&, or early returns.”

14) What are fragments in React?

What they are

Fragments let you group multiple children without adding an extra DOM element. They’re useful when you need to return multiple siblings from a component but don’t want an extra wrapper like a ``.

How to use

```
function ListItem({ label, value }) {
  return (
    <>
      <dt>{label}</dt>
      <dd>{value}</dd>
    </>
  );
}
```

Why fragments matter (practical cases)

- Keeping DOM clean: fewer wrapper nodes helps styling and layout consistency.
- Valid HTML structures: tables (`` children) and definition lists (``) often require specific element nesting.
- Reduced CSS headaches: fewer wrappers means fewer unexpected flex/grid behaviors.

Named fragments

You can also use and give it a key when mapping lists of fragments:

```
items.map(item => (
  <React.Fragment key={item.id}>
    <Row item={item} />
    <Divider />
  </React.Fragment>
))
```

Interview line

“Fragments allow returning multiple sibling elements without adding an extra DOM node.”

15) What is the purpose of keys in React lists?

Upamanyu Deka

What keys are

Keys are stable identifiers that tell React which list item corresponds to which rendered element across renders. They are used during reconciliation to match old and new children efficiently.

Why keys matter (the real reason)

When you re-render a list after inserts/deletes/reorders, React needs to decide: Is this the same item as before, or a different one? A good key lets React preserve:

- Component state inside list items (e.g., input value inside a row).
- DOM elements when appropriate (less re-creation).
- Correct animations and transitions.

Correct usage

```
{todos.map(todo => (
  <TodoRow key={todo.id} todo={todo} />
))}
```

Common mistake: index as key

Using array index as key can break state preservation when items are reordered/inserted. Example: if you render inputs in a list and insert at the top, the input values can appear to ‘move’ to the wrong row.

Rule of thumb: Use a key that uniquely identifies the item, stable across reorders (id from data).

Interview line

“Keys help React match list items between renders so updates are minimal and item state doesn’t get mixed up.”

16) What is prop drilling?

What it is

Prop drilling happens when you pass props through multiple intermediate components that don’t actually need the props, just to reach a deeply nested component that does.

Why it becomes a problem

- Boilerplate: many components need to accept and forward props they don’t use.
- Refactor pain: changing a prop name/type requires editing multiple layers.
- Coupling: unrelated components become tied to data they don’t care about.

Example (typical)

```
function App() {
  const user = { name: "Ava" };
  return <Layout user={user} />;
}

function Layout({ user }) {
  return <Sidebar user={user} />; // Layout doesn't use user directly
}

function Sidebar({ user }) {
  return <UserBadge user={user} />; // actual consumer
}
```

Solutions (choose based on scope)

- Context API for truly shared/global data (auth, theme, locale).

Upamanyu Deka

- Component composition: pass children instead of threading props.
- State libraries (Redux/Zustand/Recoil) for complex global state.

Interview line

“Prop drilling is forwarding props through layers of components that don’t need them, usually solved with context or composition.”

17) What is React.memo and when to use it?

What it is

React.memo is a higher-order component that memoizes a function component. It prevents re-renders when props are the same (based on shallow comparison with Object.is).

How it works

By default, when a parent re-renders, children re-render too. React.memo tells React: ‘Only re-render this child if its props changed.’

```
const ProductRow = React.memo(function ProductRow({ product, onSelect }) {
  return (
    <div onClick={() => onSelect(product.id)}>
      {product.name}
    </div>
  );
});
```

When it helps (good candidates)

- Child renders are expensive (heavy JSX, complex calculations, large subtrees).
- Props are stable and don’t change often.
- Parent re-renders frequently for unrelated reasons (e.g., typing in a search box).

When it doesn’t help (or hurts)

- Props change every render (e.g., inline objects/functions) → memo will still re-render.
- Overuse adds complexity and can make debugging harder.
- Shallow compare may miss deeper equality needs (custom compare may be needed but should be used carefully).

Interview line

“React.memo memoizes a component so it only re-renders when its props change, useful for expensive children with stable props.”

18) What is the difference between useMemo and useCallback?

Core difference

Both are memoization hooks, but they memoize different things:

- useMemo memoizes a computed value (result of a function).
- useCallback memoizes a function reference (so it doesn’t get recreated each render).

Why it matters

In React, changing a function reference counts as a prop change. If you pass a callback to a memoized child, the child may re-render unless the callback reference is stable. Similarly, expensive computations can be avoided with useMemo.

Upamanyu Deka

Examples

```
function Example({ items }) {
  const expensiveValue = React.useMemo(() => {
    // pretend this is CPU-heavy
    return items.reduce((sum, x) => sum + x.price, 0);
  }, [items]);

  const onClick = React.useCallback(() => {
    console.log("Clicked, total:", expensiveValue);
  }, [expensiveValue]);

  return <button onClick={onClick}>Total: {expensiveValue}</button>;
}
```

Common mistakes / interview traps

- Using useMemo/useCallback everywhere: memoization has overhead and should be justified.
- Wrong dependency arrays (stale closures) — always include values used inside.
- Trying to use useMemo for side effects (don't). Side effects belong in useEffect.

Interview line

"useMemo caches values, useCallback caches function references; both help avoid unnecessary work and re-renders when used correctly."

19) What is Context API and when should it be used?

What it is

Context is React's built-in mechanism for passing data through the component tree without manually threading props at every level. It provides a Provider to supply a value and a Consumer (useContext) to read it.

How it works (mental model)

A Provider sets a value for a subtree. Any component inside that subtree can read the value. When the Provider's value changes, React schedules re-renders for consuming components.

```
const ThemeContext = React.createContext("light");

function App() {
  const [theme, setTheme] = React.useState("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const { theme, setTheme } = React.useContext(ThemeContext);
  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Theme: {theme}
    </button>
  );
}
```

When to use (good use cases)

- Theme (dark/light)

Upamanyu Deka

- Authenticated user/session
- Localization/locale and formatting settings
- Feature flags/config shared across many components

When NOT to use

- Highly frequent updates affecting many consumers (can cause broad re-renders).
- Complex global state with many actions/selectors (Redux/Zustand may fit better).

Interview line

“Context avoids prop drilling for shared data like theme/auth; it’s best for data that truly needs to be global and changes infrequently.”

20) What is lifting state up?

What it is

Lifting state up means moving state to the nearest common ancestor of components that need to share it. Instead of each component keeping its own copy (which can diverge), you keep one authoritative state and pass down values and callbacks.

Why it’s needed

When two sibling components need to stay in sync (e.g., filter input and filtered list), separate local states cause inconsistent UI. Lifting state up ensures both siblings read the same value.

```
function Parent() {
  const [query, setQuery] = React.useState("");

  return (
    <>
      <SearchBox query={query} onChange={setQuery} />
      <Results query={query} />
    </>
  );
}

function SearchBox({ query, onChange }) {
  return <input value={query} onChange={e => onChange(e.target.value)} />;
}

function Results({ query }) {
  return <div>Searching for: {query}</div>;
}
```

Common pitfall

Over-lifting: moving state too high can cause unnecessary re-renders of large subtrees. Aim for the closest common ancestor, not always the top-level App component.

Interview line

“Lifting state up means storing shared state in the closest common parent and passing it down so sibling components remain synchronized.”

21) How do you handle forms in React?

Mental model (what interviews actually test)

Handling forms in React is about choosing where the “source of truth” lives and how you keep UX responsive while enforcing correctness. The interviewer is checking if you understand: controlled vs uncontrolled inputs, validation strategies, performance trade-offs, and scaling patterns.

Approach 1: Controlled inputs (source of truth = React state)

You bind the input’s value to state, and update state on every change. This makes validation, conditional UI, and resets predictable.

```
function SignupForm() {
  const [values, setValues] = React.useState({ email: "", password: "" });
  const [errors, setErrors] = React.useState({});

  const onChange = (e) => {
    const { name, value } = e.target;
    setValues(v => ({ ...v, [name]: value }));
  };

  const validate = (v) => {
    const next = {};
    if (!v.email.includes("@")) next.email = "Invalid email";
    if (v.password.length < 8) next.password = "Min 8 characters";
    return next;
  };

  const onSubmit = (e) => {
    e.preventDefault();
    const nextErrors = validate(values);
    setErrors(nextErrors);
    if (Object.keys(nextErrors).length === 0) console.log("Submit", values);
  };

  return (
    <form onSubmit={onSubmit}>
      <input name="email" value={values.email} onChange={onChange} />
      {errors.email && <p>{errors.email}</p>}
      <input name="password" type="password" value={values.password} onChange={onChange} />
      {errors.password && <p>{errors.password}</p>}
      <button>Submit</button>
    </form>
  );
}
```

Approach 2: Uncontrolled inputs (source of truth = DOM)

You let the DOM manage the value and read it via refs on submit. This avoids re-render-per-keystroke but makes real-time validation/UI harder.

```
function QuickForm() {
  const emailRef = React.useRef(null);
  const onSubmit = (e) => {
    e.preventDefault();
```

Upamanyu Deka

```
    console.log("Email:", emailRef.current.value);
  };
  return (
    <form onSubmit={onSubmit}>
      <input ref={emailRef} defaultValue="" />
      <button>Submit</button>
    </form>
  );
}
```

Scaling forms (senior-level considerations)

- Performance: splitting large forms into smaller memoized components reduces re-render impact.
- Validation: sync validation onChange vs async validation onBlur; avoid validating on every keystroke for expensive checks.
- UX: preserve cursor position when formatting/masking input; avoid value transforms that fight the user.
- Libraries: React Hook Form uses uncontrolled inputs + refs for performance, while Formik often uses controlled inputs for explicitness.

Common misconceptions

- “Controlled forms are always best” → not always; large forms can benefit from uncontrolled + ref-based libraries.
- “Uncontrolled means you can’t validate” → you can validate on submit or on blur by reading values, but real-time UX is harder.

Interview-ready framing

“I default to controlled inputs for correctness and real-time validation. For very large forms, I consider React Hook Form/uncontrolled patterns to reduce re-renders, while still keeping validation predictable.”

22) Explain event handling in React.

What React is doing (step-by-step)

React attaches event listeners at a higher level (historically the document/root), then dispatches events through its SyntheticEvent system. This is event delegation: fewer listeners, consistent behavior, and easier cross-browser normalization.

How you write it

```
function Example() {
  const onClick = (e) => {
    // e is a SyntheticEvent (wrapper around native event)
    console.log("type:", e.type);
  };
  return <button onClick={onClick}>Click</button>;
}
```

Key differences vs raw DOM

- Handler names are camelCased (onClick, onChange).
- You receive a SyntheticEvent (React wrapper) with a similar API to native events.
- React manages propagation and delegation; you can still use stopPropagation/preventDefault.

Misconceptions & follow-ups

- “React events are always identical to native events” → mostly, but they’re wrappers and can differ in edge cases.

Upamanyu Deka

- “onChange fires only on blur like old DOM” → in React it fires on input changes for text inputs.

Interview-ready framing

“React uses event delegation and SyntheticEvents to normalize browser differences and reduce listeners. I treat it like native events but remember React controls dispatching.”

23) What is a custom hook?

Thinking-level explanation

A custom hook is not just a function that calls hooks. It’s a way to package a state machine or side-effectful behavior into a reusable unit. The goal is to reuse logic without coupling it to UI structure.

Example: reuse async state safely

```
function useAsync(fn, deps) {
  const [state, setState] = React.useState({ status: "idle", data: null, error: null });

  React.useEffect(() => {
    let cancelled = false;
    setState({ status: "loading", data: null, error: null });

    fn()
      .then(data => !cancelled && setState({ status: "success", data, error: null }))
      .catch(error => !cancelled && setState({ status: "error", data: null, error }));

    return () => { cancelled = true; };
  }, deps);

  return state;
}
```

Rules of Hooks (why they exist)

Hooks must be called unconditionally at the top level so React can map calls to internal slots by call order. A custom hook inherits these rules.

Common mistakes

- Calling a hook conditionally inside a custom hook.
- Putting side effects in useMemo/useCallback instead of useEffect.
- Returning unstable objects/functions unnecessarily (causes re-renders).

Interview-ready framing

“Custom hooks encapsulate reusable stateful logic—like subscriptions, async flows, and derived state—so multiple components can share behavior without sharing markup.”

24) What is reconciliation in React?

What it actually means (beyond definition)

Reconciliation is the algorithmic process React uses to decide how to update the UI when state/props change. It answers: ‘Given previous UI output and next UI output, what minimal operations produce the next UI?’

Step-by-step flow

- 1) State/props change triggers an update.
- 2) React re-runs render functions to produce a new element tree.

Upamanyu Deka

- 3) React compares old vs new trees using heuristics (diffing).
- 4) React produces an internal list of effects (insert/update/delete).
- 5) Commit phase applies those effects to the host environment (DOM).

Heuristics (why it’s fast)

- Different component/element type → replace subtree (don’t deep-diff).
- Same type → update props and reconcile children.
- Keys → match list items correctly across reorders/inserts/deletes.

Misconceptions

- “React deep-compares everything” → no; it uses heuristics to keep it near O(n).
- “Reconciliation = DOM updates” → reconciliation is planning; commit is applying.

Interview-ready framing

“Reconciliation is React’s planning phase: it re-renders to build a new tree, diffs with heuristics and keys, and produces minimal effects that get applied in the commit phase.”

25) What is Virtual DOM and how does it work?

Correct mental model (survives follow-ups)

The Virtual DOM is a concept: React represents UI as a tree of lightweight JavaScript objects (React elements) in memory. It is not a browser DOM clone; it’s a description of what the UI should be.

Why this exists (real reason)

The benefit is not that ‘JS objects are faster than the DOM’ (common misconception). The real benefit is that React can compute updates deterministically in memory, batch them, and then apply minimal DOM mutations in a controlled commit phase.

Step-by-step flow

- 1) You change state → React schedules an update.
- 2) React runs render functions → produces a new element tree (in-memory).
- 3) Reconciliation compares old vs new trees using heuristics and keys.
- 4) React builds a list of effects (what to insert/update/delete).
- 5) Commit phase applies effects to the real DOM (and runs layout effects).

What is actually compared?

React compares element types, props, and child structure—not raw DOM nodes. With Fiber, React also keeps alternate trees to compare current vs work-in-progress.

Misconceptions (interview gold)

- “Virtual DOM is faster than DOM” → Virtual DOM is an enabler; scheduling and batching are the bigger wins.
- “React diffs the whole tree always” → heuristics avoid deep diffs when types change.
- “State change directly mutates DOM” → render creates a plan; commit applies it.

Senior insight

With Fiber, React can pause/resume rendering work (time slicing) and prioritize updates. Virtual DOM representations + Fiber enable concurrency features; the commit still happens atomically to keep UI consistent.

Interview-ready framing

“Virtual DOM is React’s in-memory UI description. React re-renders to produce a new tree, reconciles old vs new with heuristics and keys, then commits minimal DOM updates. The performance win is controlled batching and scheduling, not ‘virtual DOM magic’.”

26) Explain how React updates the real DOM efficiently.

What the interviewer is looking for

They want to know that React doesn’t ‘rerender the DOM’ wholesale. It re-runs component render functions to produce a new description, then applies minimal DOM mutations.

Step-by-step

- 1) Update scheduled (state/props).
- 2) Render phase builds work-in-progress tree (no DOM mutations).
- 3) Reconciliation produces effects list (insert/update/delete).
- 4) Commit phase applies minimal DOM operations.
- 5) Effects run (useLayoutEffect before paint-sensitive changes, useEffect after paint).

Key optimizations

- Batching: multiple updates can be grouped into one render/commit.
- Keys: allow correct list reconciliation and state preservation.
- Bailouts: if nothing changed (same props/state), React can skip work (e.g., memo/pure patterns).

Misconception

“React avoids re-rendering components” → React often re-renders components (runs functions), but avoids unnecessary DOM writes.

Interview-ready framing

“React efficiently updates the DOM by separating render (compute changes) from commit (apply minimal DOM mutations), using reconciliation, keys, batching, and bailouts.”

27) What is React.StrictMode?

What it is (dev-only)

StrictMode is a development-only wrapper that intentionally adds extra checks to surface unsafe patterns. It does not affect production behavior.

Why it double-invokes in React 18

In development, StrictMode may intentionally re-run certain functions/effects to detect side effects that are not resilient to remounts (important for future concurrent features).

What you observe

- Some lifecycle methods/effects may run twice in dev.
- Warnings for legacy/unsafe APIs and patterns.

Common trap

Thinking it’s a bug in production. It’s a dev signal: your effect should be idempotent and properly cleaned up.

Interview-ready framing

Upamanyu Deka

“StrictMode is a dev tool: it highlights side-effect bugs by intentionally re-invoking certain code paths, helping you write resilient effects.”

28) What are default props?

What they are (modern best practice)

Default props provide fallback values when a prop is undefined. In modern React with function components, the simplest and clearest approach is default parameters or destructuring defaults.

```
function Button({ label = "Click me", variant = "primary" }) {
  return <button data-variant={variant}>{label}</button>;
}
```

Why it matters

- Prevents undefined usage and reduces defensive checks.
- Improves component ergonomics and documentation.
- Makes components safer to reuse across codebases.

Interview-ready framing

“Default props ensure components behave predictably when optional props aren’t provided; I usually use default values in destructuring for function components.”

29) How to share data between sibling components?

Thinking-level principle

Siblings cannot directly share state unless there is a shared owner. The solution is to create a single source of truth in a common parent and pass values/callbacks down.

Step-by-step approach

- 1) Identify the smallest common ancestor that can own the shared state.
- 2) Lift the state there.
- 3) Pass down current value to both siblings.
- 4) Pass down callbacks to update state from either sibling.

```
function Parent() {
  const [query, setQuery] = React.useState("");
  return (
    <>
      <Search value={query} onChange={setQuery} />
      <Results query={query} />
    </>
  );
}
```

Alternatives and trade-offs

- Context: good if many levels need the value; avoid for extremely frequent updates across many consumers.
- Global stores: good for app-wide state with selectors and actions; keep local UI state local.
- Composition: sometimes pass components as children instead of threading props.

Interview-ready framing

“I lift state to the closest common parent for siblings. If the value is truly shared broadly, I consider context or a store with selectors.”

Upamanyu Deka

30) What is the difference between mounting and updating phases?

Mounting (first time on screen)

Mounting is when React creates the component instance (or runs the function component for the first time), creates DOM nodes, sets initial state, and commits the initial UI to the screen.

Updating (subsequent renders)

Updating happens when props/state/context change. React re-runs render to compute the next UI, reconciles differences, and commits minimal changes.

Follow-up readiness (what interviewers ask next)

- When do effects run? useEffect runs after paint; useLayoutEffect runs earlier for layout-critical work.
- What triggers an update? state/props/context changes, and sometimes parent renders (depending on bailouts).
- Unmounting? cleanup functions in effects run; DOM nodes are removed.

Interview-ready framing

“Mounting is the first render + commit of a component. Updating is re-rendering due to state/props/context changes and committing minimal UI changes.”

31) What are pure components?

What interviews test here

Not the textbook definition—interviewers want to know whether you understand referential equality, shallow comparison, and when React can safely skip renders.

Thinking-level explanation

A 'pure' component is one whose output is determined only by its inputs (props/state), and it avoids side effects in render. In practice, React’s notion of purity is tied to whether props/state are equal to previous values—often checked via shallow comparison.

How React uses this

- If props/state didn’t change (by shallow comparison), React can skip re-rendering the component (bailout).
- Class PureComponent implements shouldComponentUpdate with a shallow compare.
- Functional equivalent is React.memo (with shallow compare by default).

Common pitfalls

- Mutating objects/arrays breaks purity assumptions; shallow compare won’t detect internal mutation.
- Passing new inline objects/functions each render makes props look 'changed' every time.

Interview-ready framing

“Pure components allow React to bail out of renders when inputs are referentially equal. This works best with immutable updates and stable props.”

32) What is shallow comparison in React?

What it means in practice

Shallow comparison compares top-level references/primitive values only (Object.is for each key). It does not deep-compare nested objects.

Why React uses it

Deep comparisons are expensive and unpredictable. Shallow compare is fast and encourages immutable state updates, which makes change detection cheap.

Example

```
// prevProps.user === nextProps.user ?
const user = { name: "Ava" };
<UserCard user={user} />

// If you create a new object each render:
<UserCard user={{ name: "Ava" }} /> // shallow compare sees 'changed' every render
```

Interview trap

Saying 'shallow compare checks values'—it checks references for objects/arrays. To benefit, you must update immutably.

Interview-ready framing

“Shallow compare checks only top-level references. It’s why immutability and stable references are critical for memoization.”

33) How do you optimize rendering performance in React?

First principle (senior approach)

You don’t guess—you measure and then optimize the biggest wins. React performance issues are usually caused by unnecessary renders or expensive renders.

Step-by-step approach

- 1) Identify the re-render source (React DevTools Profiler).
- 2) Reduce re-render frequency (lift state appropriately, split components, avoid prop drilling of frequently changing values).
- 3) Reduce re-render cost (memoize expensive subtrees, virtualize lists, code-split).

Tactics (with reasoning)

- Memoize expensive children with React.memo when props are stable.
- Use useCallback/useMemo to stabilize props passed to memoized children (but don’t overuse).
- Normalize state and avoid derived state; compute derived values with memoization if expensive.
- Virtualize long lists (react-window/react-virtual) to reduce DOM nodes.
- Defer non-urgent updates with startTransition/useTransition (React 18).

Common misconceptions

- “Virtual DOM makes performance automatic” → unnecessary renders still hurt.
- “useMemo everywhere” → memoization has overhead; apply where it changes render count or expensive computation.

Interview-ready framing

“I profile first, then target unnecessary renders (memoization, stable props) and expensive work (virtualization, code splitting, transitions).”

34) What are higher-order components (HOCs)?

Thinking-level explanation

An HOC is a function that takes a component and returns a new component with enhanced behavior. Historically used to share cross-cutting concerns before hooks existed.

Example

```
function withLogger(Wrapped) {
  return function WithLogger(props) {
    console.log("Render:", Wrapped.name, props);
    return <Wrapped {...props} />;
  };
}
```

Trade-offs (why hooks replaced many HOCs)

- Wrapper hell: debugging and DevTools trees become deeper.

Upamanyu Deka

- Props collisions: multiple HOCs can inject overlapping props.
- Harder to compose compared to custom hooks (logic reuse without UI wrappers).

Where HOCs still appear

- Legacy codebases
- Libraries that wrap components (routing, theming, analytics)

Interview-ready framing

“HOCs are component enhancers. Today, custom hooks cover many HOC use cases with cleaner composition, but HOCs still appear in legacy libraries/code.”

35) What are render props?

Thinking-level explanation

Render props is a pattern where a component’s child is a function. The component calls that function to render UI, passing state/behavior as arguments. It was an early solution for sharing logic without inheritance.

Example

```
function Mouse({ render }) {
  const [pos, setPos] = React.useState({ x: 0, y: 0 });
  return (
    <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>
      {render(pos)}
    </div>
  );
}

// Usage:
<Mouse render={({x,y}) => <p>{x},{y}</p>} />
```

Trade-offs

- Can lead to nested render functions (less readable).
- Creates new function props frequently unless stabilized.
- Hooks often provide a cleaner alternative for logic reuse.

Interview-ready framing

“Render props share reusable logic by passing state to a render function; hooks typically replace this pattern in modern React.”

36) Explain useRef hook.

Two distinct useRef use cases (this is the key)

- 1) DOM access (imperative): store a reference to a DOM node.
- 2) Mutable value that persists across renders without causing re-renders.

DOM access example

```
function FocusInput() {
  const inputRef = React.useRef(null);
  return (
    <div>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current?.focus()}>Focus</button>
    </div>
```

Upamanyu Deka

```
  );
}
```

Mutable value example (no re-render)

Great for storing timers, previous values, and in-flight flags without triggering renders.

```
function Timer() {
  const idRef = React.useRef(null);

  React.useEffect(() => {
    idRef.current = setInterval(() => console.log("tick"), 1000);
    return () => clearInterval(idRef.current);
  }, []);

  return null;
}
```

Misconceptions

- “Updating ref re-renders” → no; ref changes do not trigger re-renders.
- “Refs replace state” → refs are for mutable, non-UI data; UI data should be state.

Interview-ready framing

“useRef persists a mutable value across renders. It’s used for DOM refs and for storing non-UI mutable values without re-rendering.”

37) What is useLayoutEffect?

Thinking-level difference vs useEffect

Both run after render, but useLayoutEffect runs synchronously after DOM mutations and before the browser paints. useEffect runs after paint (asynchronously).

When to use it

- Measuring DOM layout (getBoundingClientRect) and then synchronously applying changes to avoid flicker.
- Scroll position adjustments, tooltip positioning, layout-dependent animations.

Why you should avoid overuse

useLayoutEffect blocks painting, so heavy work inside it can harm responsiveness.

Interview-ready framing

“useLayoutEffect runs before paint for layout reads/writes to avoid flicker; useEffect is preferred for most side effects.”

38) How does React handle events internally?

Step-by-step internal model

- 1) React registers a small set of listeners at the root (delegation).
- 2) When an event occurs, React builds a SyntheticEvent wrapper.
- 3) React determines the target component and walks the fiber tree to simulate bubbling/capture.
- 4) React invokes handlers in the correct order and manages propagation flags.

Why React does this

- Cross-browser normalization (consistent event API).

Upamanyu Deka

- Fewer listeners → lower memory/CPU overhead.
- Integration with React scheduling (batching state updates triggered by events).

Misconceptions

- “React attaches listeners to every node” → no, delegation minimizes listeners.
- “Synthetic events are slower” → overhead exists but is usually outweighed by the benefits; bottlenecks are typically elsewhere.

Interview-ready framing

“React uses event delegation and a SyntheticEvent system; events are dispatched through the component tree, and updates triggered inside events are batched.”

39) What are React portals?

Thinking-level explanation

Portals let you render a component’s DOM output into a different DOM subtree than its parent, while keeping it in the same React component tree. This solves UI layering problems without breaking React composition.

Common use cases

- Modals and dialogs (escape parent overflow/stacking contexts).
- Tooltips/popovers (render near body to avoid clipping).
- Global toasts/notifications.

Key follow-up detail

Even though the DOM is elsewhere, event bubbling still follows the React tree (not the DOM tree).

Example

```
function Modal({ children }) {
  return ReactDOM.createPortal(
    <div className="backdrop">{children}</div>,
    document.body
  );
}
```

Interview-ready framing

“Portals render into a different DOM container while preserving React tree semantics like context and event bubbling.”

40) How do you perform conditional rendering efficiently?

First principle

Efficiency here means: avoid unnecessary mounts/unmounts, avoid expensive subtree recomputation, and keep state stable where needed.

Strategies (with reasoning)

- Use early returns for big branches (clear and avoids building unused UI).
- Prefer CSS visibility toggles when you must preserve component state (e.g., keep input values) instead of unmounting.
- Memoize expensive conditional subtrees (React.memo + stable props) if parent re-renders frequently.
- Code-split and lazy load rarely-used branches to reduce initial bundle size.

Upamanyu Deka

Example: preserve state vs unmount

```
function Panel({ open }) {
  // Unmounts when closed (state resets)
  // return open ? <ExpensiveComponent /> : null;

  // Preserve state (component stays mounted)
  return <div style={{ display: open ? "block" : "none" }}><ExpensiveComponent /></div>;
}
```

Misconception

“Unmounting is always better” → unmounting frees resources but resets internal state and can cost more if you frequently remount.

Interview-ready framing

“Efficient conditional rendering depends on whether you need to preserve state and how expensive the subtree is. I choose between unmounting, hiding, memoization, and code splitting accordingly.”

41) Explain React Fiber architecture.

What the interviewer is really asking

They want to know why React changed from the old stack reconciler, what Fiber is (data structure + algorithm), and how it enables concurrency features like time slicing, priorities, Suspense, and transitions.

Why Fiber exists (problem it solved)

Old React reconciliation (stack-based) was mostly synchronous: once rendering started, it tended to run to completion. On large trees, this could block the main thread, making the UI feel unresponsive (scroll jank, input lag). Fiber was introduced so React could split work into units, pause/resume, and prioritize user-visible updates.

Mental model: Fiber = 'work unit' + 'tree node'

- Fiber node: a JavaScript object representing a component instance in the tree (type, props, state, child/sibling/return pointers).
- Unit of work: React processes the tree incrementally—one fiber at a time—so it can yield to the browser.
- Double buffering: React often keeps a current tree and a work-in-progress tree ('alternate') to prepare the next UI without corrupting the current UI.

Step-by-step flow (render vs commit)

- 1) Update scheduled: state/props change creates an update in a queue.
- 2) Render phase (interruptible): React walks fibers, computes next props/state, and builds a work-in-progress tree + effect list.
- 3) Yielding: if time runs out or higher-priority work arrives, React can pause and resume later.
- 4) Commit phase (non-interruptible): React applies the computed effects to the DOM and runs layout effects.

Key senior insight: Fiber enables scheduling and priorities

Fiber alone isn't concurrency; it's the architecture that makes scheduling possible. React can assign priorities (lanes) to updates (e.g., input typing > background refresh) and decide what to render first.

Common misconceptions

- "Fiber is the Virtual DOM" → no. Virtual DOM is a concept; Fiber is the implementation that stores work and enables scheduling.
- "Concurrent rendering means React uses multiple threads" → in the browser, React mostly runs on the main thread; concurrency is cooperative scheduling/time slicing.
- "Commit can be paused" → commit is kept atomic to avoid tearing the UI.

Interview-ready framing

"Fiber is React's incremental reconciliation architecture. It represents UI as a fiber tree where each node is a unit of work. React can pause/resume rendering, prioritize updates, and then commit DOM mutations atomically—enabling features like time slicing, transitions, and Suspense."

Upamanyu Deka

42) How does React handle re-rendering efficiently?

First principle: React re-runs render functions; it avoids unnecessary DOM writes

React 're-rendering' usually means re-executing component functions to compute a new UI description. The efficiency comes from minimizing DOM mutations and skipping work when possible.

Step-by-step

- 1) Something triggers an update: setState, parent re-render, context change, store update.
- 2) React computes next element output (render phase).
- 3) Reconciliation compares old and new element trees using heuristics and keys.
- 4) React builds a list of DOM mutations (effects).
- 5) Commit applies minimal DOM changes.

Where the wins come from (practical)

- Batching: multiple state updates in the same tick can be combined into one render+commit.
- Bailouts: React can skip reconciling subtrees if inputs are referentially equal (React.memo, PureComponent).
- Stable references: immutability + memoization lets shallow comparisons work.
- List keys: preserve identity/state and avoid unnecessary re-mounts.

Example: avoid unnecessary child re-renders

```
const Row = React.memo(function Row({ item, onSelect }) {
  return <div onClick={() => onSelect(item.id)}>{item.name}</div>;
});

function List({ items }) {
  const onSelect = React.useCallback((id) => {
    console.log("selected", id);
  }, []);

  return items.map(i => <Row key={i.id} item={i} onSelect={onSelect} />);
}
```

Misconceptions & follow-ups

- “React only re-renders changed components” → parent renders can re-run children; bailouts decide whether to continue.
- “useMemo/useCallback always improves perf” → only if it reduces real work; otherwise it adds overhead.

Interview-ready framing

“React efficiently handles re-renders by separating compute (render phase) from DOM writes (commit), using reconciliation, batching, bailouts (memo/pure patterns), and stable keys to minimize updates.”

43) What is batching in React?

Thinking-level explanation

Batching means grouping multiple state updates into a single render+commit, so React doesn’t do redundant work. It’s about efficiency and consistency: the UI reflects the final state after a burst of updates.

Step-by-step

Upamanyu Deka

- 1) Multiple setState calls occur (often inside the same event handler).
- 2) React queues the updates rather than rendering immediately for each one.
- 3) React processes the queue and renders once with the final state.

Why React 18 matters

In React 18, automatic batching was expanded beyond React event handlers to include more async boundaries (like timeouts/promises in many cases), reducing extra renders.

Example: why functional updates matter

```
function Counter() {
  const [count, setCount] = React.useState(0);

  const incTwice = () => {
    setCount(c => c + 1);
    setCount(c => c + 1);
  };

  return <button onClick={incTwice}>{count}</button>;
}
```

Common misconceptions

- “Batching makes updates asynchronous” → batching groups updates; scheduling determines when they flush.
- “Two setState calls always mean two renders” → not necessarily; often one render due to batching.

Interview-ready framing

“Batching groups multiple state updates into one render/commit cycle, improving performance and preventing intermediate inconsistent UI. In React 18, batching applies more broadly across async boundaries.”

44) How does React reconcile lists with keys?

What this is really about

React needs stable identity for each list item across renders. Keys let React match old children to new children so it preserves component state and applies minimal changes.

Step-by-step (how matching works)

- 1) React renders a list and stores children fibers with their keys.
- 2) Next render, React creates the new children list (with keys).
- 3) React tries to match old and new children by key (and type).
- 4) Matched items are updated in place; unmatched ones are inserted or deleted.
- 5) Reorders become 'move' operations instead of destroy+recreate (when keys are stable).

Why index keys break things

If you use index as key and insert/remove/reorder, the key-to-item mapping changes. React may reuse DOM/state for the wrong item, leading to 'input values jumping rows' or incorrect component state.

Example: correct vs incorrect

```
// ■ stable id key
items.map(item => <Row key={item.id} item={item} />)

// ■ index key (breaks on reorder/insert/delete)
items.map((item, idx) => <Row key={idx} item={item} />)
```

Upamanyu Deka

Senior insight: keys are identity, not uniqueness alone

Keys must be unique among siblings, but more importantly they must be stable for the same conceptual item across renders. If your data lacks stable IDs, generate them when the data is created—not during render.

Interview-ready framing

“React reconciles lists by matching children using keys (and element type). Stable keys preserve state and minimize DOM work; index keys can mis-associate state on reorder/insert/delete.”

45) What are uncontrolled vs controlled inputs?

What the interviewer wants

Not just definitions—they want trade-offs: source of truth, validation strategy, performance, and integration concerns.

Controlled inputs (source of truth = React state)

- Value comes from state:
- Best for live validation, conditional UI, and predictable resets.
- Costs: more re-renders (usually fine; can be heavy in huge forms).

```
function Controlled() {
  const [v, setV] = React.useState("");
  return <input value={v} onChange={e => setV(e.target.value)} />;
}
```

Uncontrolled inputs (source of truth = DOM)

- Value lives in the DOM:
- Best for simple forms and performance-heavy scenarios (often with libraries).
- Costs: harder real-time validation/conditional UI; less explicit.

```
function Uncontrolled() {
  const ref = React.useRef(null);
  return <input ref={ref} defaultValue="" />;
}
```

Senior guidance (how to choose)

- If UI depends on the value (validation messages, enabling submit, live formatting) → controlled.
- If you only need values at submit and want fewer renders → uncontrolled or React Hook Form style.
- Avoid mixing both for the same field unless you know exactly why (can cause bugs).

Interview-ready framing

“Controlled inputs store form state in React, enabling predictable validation and UI. Uncontrolled inputs let the DOM manage values and are accessed via refs, often used for simpler forms or performance-focused libraries.”

46) What is lazy loading in React?

Thinking-level explanation

Lazy loading means deferring the loading/execution of code until it’s actually needed, reducing initial bundle size and speeding up first paint. In React, this is commonly done with React.lazy + dynamic import, usually combined with Suspense.

Step-by-step flow

- 1) App renders and hits a lazy component.
- 2) The lazy component triggers a dynamic import() (code split chunk).
- 3) While the chunk loads, React shows the nearest Suspense fallback.
- 4) Once loaded, React renders the real component.

Example

```
const Settings = React.lazy(() => import("./Settings"));

function App() {
  return (
    <React.Suspense fallback={<div>Loading…</div>}>
      <Settings />
    </React.Suspense>
  );
}
```

Misconceptions

- “Lazy loading speeds everything up” → it improves initial load but may delay navigation to lazy screens unless you prefetch.
- “React.lazy is the only code splitting” → route-level splitting (e.g., in Next.js) is common; bundlers can split in many ways.

Senior insight: prefetch on intent

For better UX, you can prefetch code on hover/visibility/idle so the chunk is likely ready when the user navigates.

Interview-ready framing

“Lazy loading defers loading code until needed (dynamic import). React.lazy + Suspense provides a structured fallback UI while the chunk loads, improving initial performance.”

47) What is React.Suspense used for?

What Suspense really is

Suspense is a boundary that can display a fallback while some part of the UI is not ready. Originally it was used for code splitting (React.lazy). In modern React ecosystems, it also coordinates data fetching and streaming SSR (framework/libraries dependent).

Step-by-step mental model

- 1) A child component 'suspends' (e.g., code chunk not loaded, or a data layer throws a Promise).
- 2) React stops rendering that subtree and shows the nearest Suspense fallback.
- 3) When the promise resolves, React retries rendering that subtree.

Example (code splitting)

```
const Profile = React.lazy(() => import("./Profile"));

<Suspense fallback={<Spinner />}>
  <Profile />
</Suspense>
```

Senior insights

- Place Suspense boundaries strategically (route-level vs component-level) to avoid blanking too much UI.
- Use multiple boundaries to allow progressive rendering (parts load independently).
- In SSR/streaming frameworks, Suspense enables sending HTML progressively instead of waiting for everything.

Misconceptions

- “Suspense replaces loading state everywhere” → not automatically; it depends on your data fetching approach/library/framework support.
- “Fallback means nothing renders” → only the suspended subtree swaps to fallback; parents and other subtrees can still render.

Interview-ready framing

“Suspense is a boundary for 'not ready' UI. It shows fallback when a subtree suspends (lazy chunk or promise-based data), then retries rendering when ready—enabling smoother loading UX and streaming patterns.”

48) How do you handle side effects in React?

Thinking-level explanation

Side effects are operations that interact with the outside world (network, subscriptions, timers, logging, DOM APIs). In React, the core rule is: keep render pure and do side effects in effects (useEffect/useLayoutEffect) or event handlers.

Where side effects belong

- Event handlers: user-driven side effects (click → POST request).
- useEffect: after paint side effects (fetching, subscriptions, timers).
- useLayoutEffect: layout-sensitive reads/writes before paint (measure & sync).

Step-by-step pattern for data fetching

- 1) Trigger fetch when dependencies change (e.g., userId).
- 2) Track loading/error states (or use a data library).
- 3) Handle cancellation/stale responses to avoid race conditions.
- 4) Cleanup subscriptions/timers on unmount.

Example: abortable fetch (race-safe)

```
function User({ userId }) {
  const [user, setUser] = React.useState(null);
  const [error, setError] = React.useState(null);

  React.useEffect(() => {
    const controller = new AbortController();
    setUser(null); setError(null);

    (async () => {
      try {
        const res = await fetch(`/api/users/${userId}`, { signal: controller.signal });
        const data = await res.json();
        setUser(data);
      } catch (e) {
        if (e.name !== "AbortError") setError(e);
      }
    })();
```

Upamanyu Deka

```
      return () => controller.abort();
    }, [userId]);

    if (error) return <p>Error</p>;
    if (!user) return <p>Loading</p>;
    return <pre>{JSON.stringify(user, null, 2)}</pre>;
  }
```

Common misconceptions

- “useEffect is the lifecycle” → it models side effects, not just lifecycle. It’s about syncing external systems with React state.
- “Empty deps means run once” → in StrictMode dev, effects may run twice to surface unsafe patterns; cleanup must be correct.

Interview-ready framing

“I keep render pure and handle side effects in useEffect/useLayoutEffect with correct dependencies and cleanup. For async work, I guard against races and stale updates (abort/cancel pattern or data libraries).”

49) Explain error boundaries in React.

What they are (practical mental model)

Error boundaries catch rendering errors in a subtree and let you render a fallback UI instead of crashing the whole app. They catch errors during render, in lifecycle methods, and in constructors of class components.

Important limitation (common follow-up)

They do NOT catch:

- Errors in event handlers (handle those with try/catch inside the handler).
- Errors in async code like setTimeout/promises (handle at the async boundary).
- Errors thrown outside React render tree.

Step-by-step behavior

- 1) A child throws during render/commit lifecycle.
- 2) Nearest error boundary captures the error.
- 3) React unmounts the broken subtree and renders fallback UI.
- 4) You can log/report the error (e.g., to Sentry).

Example (classic class boundary)

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError(err) {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.log("Report error", error, info);
  }

  render() {
    if (this.state.hasError) return <h3>Something went wrong</h3>;
    return this.props.children;
  }
}
```

Upamanyu Deka

Where to place boundaries (senior guidance)

- Route-level boundaries: prevent a whole page from crashing the app shell.
- Component-level boundaries around risky widgets (charts, editors).
- Avoid wrapping everything in one boundary (hard to isolate and recover).

Interview-ready framing

“Error boundaries are React’s mechanism to contain render-time failures: they catch errors in a subtree and show fallback UI. They don’t catch event/async errors, so you handle those separately.”

50) What is code splitting in React?

Thinking-level explanation

Code splitting is a bundling strategy: you break your JavaScript into multiple chunks so the browser downloads only what’s needed for the current route/feature. React participates via React.lazy/Suspense, but the actual splitting is done by the bundler/framework.

Step-by-step

- 1) Configure bundler (or framework) to output chunks.
- 2) Use dynamic import() to create a split point.
- 3) Load chunks on demand (route/feature).
- 4) Show fallback UI while chunks load (Suspense).

Example (route-level idea)

```
const Admin = React.lazy(() => import("./Admin"));

function Routes({ isAdmin }) {
  return (
    <Suspense fallback={<div>Loading route…</div>}>
      {isAdmin ? <Admin /> : <Home />}
    </Suspense>
  );
}
```

Senior insight: trade-offs & strategy

- Too many tiny chunks can add network overhead (waterfall).
- Prefetch chunks for likely next navigations (hover/idle) to hide latency.
- Split by routes first (biggest win), then by heavy components (editors, charts).

Misconceptions

- “React automatically code splits” → no; you must create split points via dynamic imports or framework routing.
- “Code splitting always improves performance” → depends on caching, network, and chunk strategy.

Interview-ready framing

“Code splitting loads only what’s needed by splitting bundles into chunks, typically via dynamic import + React.lazy/Suspense. The goal is faster initial load without hurting navigation via smart chunking/prefetch.”

Upamanyu Deka

51) Explain React’s diffing algorithm.

What the interviewer is testing

They want to see whether you understand that React does not do an expensive deep tree diff. It uses heuristics + keys to keep reconciliation near O(n) and preserve component identity/state.

The real problem being solved

Given two UI descriptions (old tree vs new tree), React must compute the minimal set of operations to update the host environment (DOM). Direct DOM reads/writes are expensive, so React computes changes in memory first.

Step-by-step: how React diffs

- 1) A state/props update schedules work.
- 2) React re-runs render to produce a new element tree (work-in-progress).
- 3) Reconciliation compares old vs new trees using heuristics:
- • Different element/component type → replace subtree (don’t deep-diff).
- • Same type → update props and then diff children.
- 4) For children arrays, React uses keys (and type) to match items and determine insert/move/delete.
- 5) React records effects (mutations) and later applies them in the commit phase.

Keys are part of the algorithm (not optional)

Without stable keys, React can’t reliably match list items across reorders/inserts; it may reuse DOM/state for the wrong item. Keys are identity, not just uniqueness.

Misconceptions (follow-up traps)

- “React compares the real DOM” → no, it compares element/fiber trees and produces a mutation plan.
- “React deep-compares props” → no, React compares element type and structure; props updates are applied, but equality is your responsibility for memoization.
- “Virtual DOM is the diff algorithm” → Virtual DOM is the representation; diffing is reconciliation logic.

Senior-level insight: render vs commit

Diffing happens during the render/reconciliation phase (interruptible with Fiber). DOM mutations happen in the commit phase (kept atomic). That separation is key to concurrency features.

Interview-ready framing

“React’s diffing is heuristic-based: it compares element type and structure, replaces subtrees when types change, and uses keys to match list items. It computes a minimal mutation plan during reconciliation and applies it during commit.”

52) What are React fibers and lanes?

What the interviewer is testing

Upamanyu Deka

They're checking whether you understand Fiber as the internal data structure + incremental work model, and lanes as the priority/scheduling mechanism introduced for concurrent features.

Fiber (mental model)

- Fiber node = a JS object representing a component instance in the tree (type, props, state, links to child/sibling/parent).
- Unit of work = React can process one fiber at a time and yield to the browser.
- Double buffering = current tree + work-in-progress tree (alternate) so React can prepare the next UI safely.

Lanes (priority model)

Lanes represent categories/priorities of updates. When updates happen, they're assigned lanes (e.g., urgent input vs non-urgent transitions). React schedules which lanes to work on first.

Step-by-step: how fibers + lanes interact

- 1) An update is enqueued on a fiber (setState, context change, etc.).
- 2) React assigns the update a lane (priority).
- 3) Scheduler chooses which lane(s) to render now vs later.
- 4) Render phase builds a WIP fiber tree and effect list for the chosen lanes.
- 5) Commit applies changes; remaining lanes can be processed later.

Misconceptions

- "Fibers are the Virtual DOM" → fibers store work + state; elements are the declarative description.
- "Lanes mean multi-threading" → lanes are priority buckets; work is still mainly on the main thread in the browser.

Senior insight

Lanes enable React to keep the app responsive: urgent updates can preempt background updates, and transitions can be interrupted if the user types or clicks.

Interview-ready framing

"Fiber is React's incremental rendering architecture (tree of work units). Lanes are how React prioritizes and schedules updates for concurrency—urgent work first, non-urgent work later."

53) What is concurrent rendering?

What the interviewer is testing

They want you to describe concurrency as interruptible rendering + prioritization, not true parallel threads. Also, they'll probe for 'tearing', commit atomicity, and user-perceived performance.

Thinking-level explanation

Concurrent rendering means React can start rendering an update, pause/yield, and later continue or abandon that work if higher-priority updates arrive. This prevents long renders from blocking the main thread.

Step-by-step

- 1) React schedules work with a priority (lane).
- 2) React begins render (reconciliation) of the work-in-progress tree.
- 3) If time runs out or higher priority work arrives, React yields/pause.

- 4) React resumes later or discards partial work if superseded.
- 5) Commit still happens atomically (apply DOM mutations together).

Why commit is kept atomic (senior detail)

If React committed partial UI in the middle of a render, the screen could show inconsistent combinations of old/new state (tearing). React avoids this by committing only when a consistent tree is ready.

Misconceptions

- “Concurrent rendering = multi-threading” → it’s cooperative scheduling/time-slicing on the main thread.
- “Concurrent means effects run multiple times in prod” → StrictMode double-invocation is dev-only; concurrency is about render scheduling.

Interview-ready framing

“Concurrent rendering lets React interrupt and prioritize rendering work to keep the UI responsive. Rendering can be paused/resumed, but commits are atomic to avoid inconsistent UI.”

54) What is hydration and how does it work?

What the interviewer is testing

They’re checking whether you understand SSR output vs client ownership: matching server HTML to React tree, attaching event handlers, and what happens on mismatches.

Thinking-level explanation

Hydration is the process where React 'adopts' existing server-rendered HTML and attaches event listeners and internal state so the page becomes interactive without throwing away and re-creating the DOM.

Step-by-step hydration flow

- 1) Server renders HTML for the initial route and sends it to the browser.
- 2) Browser paints the HTML quickly (fast first content).
- 3) Client loads JS bundle; React builds the component tree for the same route.
- 4) React matches its tree to the existing DOM nodes and attaches listeners/state (hydrate).
- 5) After hydration, future updates work like normal client-side React.

Hydration mismatches (common follow-up)

If server HTML doesn’t match what the client would render (different data, time-dependent rendering, random IDs), React may warn and may need to re-render parts client-side. This can hurt performance and cause flicker.

Senior insights

- Stabilize SSR output: avoid non-determinism (Date.now(), Math.random()) during render; move to effects.
- Use consistent data: ensure the same initial data is used on server and client.
- Streaming + Suspense can hydrate progressively in modern frameworks, improving perceived performance.

Interview-ready framing

“Hydration is React attaching to server-rendered HTML to make it interactive. React matches the existing DOM to its component tree and wires events/state; mismatches force re-rendering and can hurt UX.”

55) SSR vs CSR vs ISR vs SSG — differences?

What the interviewer is testing

They're checking if you can choose the right rendering strategy based on SEO, time-to-first-byte, interactivity, caching, and data freshness.

Thinking-level comparison (decision lens)

- CSR (Client-Side Rendering): HTML shell first; JS fetches data and renders UI. Great for app-like UX, but slower first meaningful paint and SEO depends on crawler support.
- SSR (Server-Side Rendering): server generates HTML per request. Faster first content and SEO-friendly; requires server compute and careful caching.
- SSG (Static Site Generation): HTML generated at build time. Fastest delivery (CDN), but data can be stale unless rebuilt.
- ISR (Incremental Static Regeneration): hybrid—serve static page but re-generate in the background on a schedule or trigger for freshness.

Step-by-step: what changes across strategies

- Where HTML is produced: client (CSR) vs server request-time (SSR) vs build-time (SSG) vs build+background refresh (ISR).
- Data freshness: CSR/SSR can be fresh per request; SSG is fixed at build; ISR refreshes periodically.
- Caching: SSR often needs caching layers; SSG/ISR are CDN-friendly by default.

Misconceptions

- "SSR means no client JS" → you still hydrate for interactivity.
- "SSG can't be dynamic" → dynamic routes can be prebuilt; ISR adds freshness without full rebuild.

Interview-ready framing

"I choose based on content needs: CSR for app-like dashboards, SSR for SEO + fresh per-request pages, SSG for mostly static content at scale, and ISR when I want CDN speed with periodic freshness."

56) What is useReducer hook and when to use it?

Thinking-level explanation

useReducer is a structured way to manage state transitions. Instead of setting state directly, you dispatch actions to a reducer function that computes the next state. This is valuable when state logic becomes complex or when multiple state fields change together.

When useReducer beats useState

- Complex state objects with multiple fields that change together.
- State transitions driven by explicit events (actions).
- You want predictable transitions + easier testing (pure reducer).
- You need to avoid bugs from scattered setState calls.

Example (predictable transitions)

```
function reducer(state, action) {
  switch (action.type) {
    case "start": return { ...state, loading: true, error: null };
    case "success": return { loading: false, data: action.data, error: null };
    case "error": return { loading: false, data: null, error: action.error };
    default: return state;
  }
```

Upamanyu Deka

```
}

function DataWidget() {
  const [state, dispatch] = React.useReducer(reducer, { loading: false, data: null, error: null });

  const load = async () => {
    dispatch({ type: "start" });
    try {
      const res = await fetch("/api/data");
      dispatch({ type: "success", data: await res.json() });
    } catch (e) {
      dispatch({ type: "error", error: e.message });
    }
  };

  return (
    <div>
      <button onClick={load}>Load</button>
      {state.loading && <p>Loading…</p>}
      {state.error && <p>Error: {state.error}</p>}
      {state.data && <pre>{JSON.stringify(state.data, null, 2)}</pre>}
    </div>
  );
}
```

Misconceptions

- “useReducer is only for Redux-like apps” → it’s for local component state too.
- “Reducers always improve performance” → main win is correctness and structure; perf depends on re-render patterns.

Interview-ready framing

“I use useReducer when state transitions are complex or event-driven. It centralizes state logic in a reducer, making transitions predictable and testable.”

57) What is state management in React?

Thinking-level explanation

State management is about where state lives, how it changes, and how it flows through the UI without becoming inconsistent. The 'right' approach depends on scope: local UI state vs shared feature state vs app-wide state.

Step-by-step decision process (senior approach)

- 1) Classify state: local UI, shared within feature, global app state, server cache state.
- 2) Choose the smallest scope that works (keep state close to where it’s used).
- 3) Use lifting state up / context / store based on how many consumers and how often it changes.
- 4) Separate server state from client UI state when possible (data fetching libraries handle caching).

Common categories

- Local UI state: input values, modals, toggles → useState/useReducer.
- Shared UI/feature state: filters, selections across siblings → lift state or context.
- Global state: auth, permissions, theme, cross-feature coordination → context or store.
- Server state: fetched data, caching, invalidation → data libraries (React Query/SWR) or framework fetch caching.

Misconceptions

Upamanyu Deka

- “Redux for everything” → overkill for local state; adds ceremony.
- “Context replaces state libraries” → context is transport; frequent updates can cause broad re-renders unless optimized.

Interview-ready framing

“State management is choosing the right home and update model for state—local when possible, lifted when shared, context/store when truly global, and server-cache state handled via dedicated data libraries.”

58) How does Context API compare to Redux?

What the interviewer is testing

They want trade-offs: when context is enough, when Redux (or another store) is better, and how re-render behavior differs.

Context (what it is good at)

- Passing shared values through the tree without prop drilling (theme, auth, locale).
- Simple global-ish state with limited updates.
- Low ceremony, built-in to React.

Redux/store (what it is good at)

- Complex global state with many actions and cross-feature updates.
- Selective subscriptions (selectors) to avoid broad re-renders.
- Dev tooling: time travel, logging, predictable reducers/middleware ecosystem.

Re-render behavior (follow-up readiness)

Context updates cause all consuming components to re-render when the provider value identity changes. Stores can offer fine-grained subscriptions so only components whose selected slice changed update.

Example: context value identity pitfall

```
function Provider({ children }) {
  const [count, setCount] = React.useState(0);
  // ■ new object every render -> all consumers re-render
  // const value = { count, setCount };

  // ■ memoize value
  const value = React.useMemo(() => ({ count, setCount }), [count]);

  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
}
```

Interview-ready framing

“Context is great for shared configuration-like data and avoiding prop drilling. Redux (or similar stores) shines for complex, frequently updated global state with selectors and tooling; it can avoid broad re-renders via selective subscriptions.”

59) How do you optimize React app performance?

Senior approach: don’t optimize blind

Performance work starts with measurement (Profiler, web vitals). Most issues come from unnecessary renders, expensive renders, large bundles, or too many DOM nodes.

Step-by-step playbook

- 1) Measure: React DevTools Profiler + browser Performance panel + Web Vitals (LCP/INP/CLS).
- 2) Fix unnecessary renders: memoize expensive children, stabilize props (useCallback/useMemo), avoid lifting state too high, avoid context value churn.
- 3) Fix expensive work: virtualize long lists, memoize heavy computations, split components.
- 4) Fix loading: code-split routes, lazy load heavy widgets, prefetch on intent.
- 5) Fix network/data: cache server state, avoid waterfalls, debounce inputs, use transitions for non-urgent updates.

Common misconceptions

- “useMemo everywhere” → only helps when it prevents real recomputation or re-rendering; otherwise overhead.
- “Virtual DOM makes it fast” → unnecessary renders still compute; DOM writes are minimized, not computation.

Interview-ready framing

“I profile first, then reduce unnecessary renders (memo + stable props + correct state scope), reduce expensive UI work (virtualization), and reduce bundle/load time (code splitting + prefetch).”

60) What is memoization in React?

Thinking-level explanation

Memoization is caching the result of work so it isn’t repeated when inputs haven’t changed. In React, memoization is used in two main places: (1) caching render results (React.memo) and (2) caching computed values/functions (useMemo/useCallback).

Where memoization helps (and where it doesn’t)

- Helps when re-renders are frequent and a subtree/computation is expensive.
- Helps when you pass stable props to memoized children to enable bailouts.
- Does NOT help if inputs change every render (inline objects/functions), or if the work is cheap.

Examples

```
const Row = React.memo(function Row({ item }) {
  return <div>{item.name}</div>;
});

function Example({ items }) {
  const total = React.useMemo(() => items.reduce((s, x) => s + x.price, 0), [items]);
  return (
    <div>
      <p>Total: {total}</p>
      {items.map(i => <Row key={i.id} item={i} />)}
    </div>
  );
}
```

Misconceptions (follow-up traps)

- “Memoization stops renders” → it only skips re-render if React can prove inputs didn’t change (shallow compare).
- “useMemo is a performance guarantee” → it’s a hint; React may still re-run renders, and memoization adds memory/CPU overhead.

Upamanyu Deka

Senior insight: stabilize what you pass

Memoization often fails because props are unstable (new object/function per render). The biggest win is designing state and props so referential equality can hold.

Interview-ready framing

“Memoization is caching work based on stable inputs. In React, React.memo caches component rendering by props, and useMemo/useCallback cache computed values/functions. It’s useful when it reduces real repeated work and inputs are stable.”

61) How to prevent unnecessary re-renders?

Senior answer structure: scope → stability → memoization → architecture → measure

Unnecessary re-renders are rarely a 'React problem'—they're usually a state/prop design problem. A senior answer shows you can (1) identify the cause, (2) reduce the blast radius, and (3) verify with profiling.

Step-by-step playbook

- 1) Profile first (React DevTools Profiler): identify which components re-render frequently and which commits are slow.
- 2) Fix state scope: keep state as local as possible; avoid lifting state high unless multiple consumers need it.
- 3) Stabilize props: avoid inline objects/functions passed to memoized children; useMemo/useCallback where it prevents real downstream work.
- 4) Use memoization selectively: React.memo for expensive components; memoize computed props that otherwise change identity each render.
- 5) Control context updates: split contexts and memoize provider values to avoid broadcast re-renders.
- 6) For big lists: virtualize (memo alone won't save DOM/layout cost).
- 7) Re-profile to confirm improvements (avoid placebo optimization).

Misconceptions

- "Stop all re-renders" → re-renders are normal; you avoid expensive and frequent ones.
- "useMemo/useCallback everywhere" → overhead + complexity; only use when it prevents meaningful work.

Example: prop stability enabling React.memo

```
const Row = React.memo(function Row({ item, onPick }) {
  return <button onClick={() => onPick(item.id)}>{item.name}</button>;
});

function List({ items }) {
  const onPick = React.useCallback((id) => console.log("pick", id), []);
  return items.map(i => <Row key={i.id} item={i} onPick={onPick} />);
}
```

Interview-ready framing

"I prevent unnecessary re-renders by scoping state, stabilizing props, using React.memo selectively, avoiding context churn, virtualizing large lists, and validating with the Profiler."

62) What are Suspense boundaries?

Thinking-level explanation

A Suspense boundary is a UI 'containment zone' that defines what portion of the tree may temporarily show a fallback while something inside is not ready (code/data). It’s not just a loading spinner—it’s a deliberate UX boundary that enables progressive rendering.

Step-by-step flow

- 1) A component inside the boundary suspends (e.g., lazy import or data read in Suspense-enabled patterns).
- 2) React stops rendering that subtree and shows the boundary’s fallback instead.
- 3) When the suspended resource becomes ready, React retries rendering the subtree.
- 4) Commit swaps fallback → real UI atomically for that boundary.

Senior insights

- Boundary placement is a UX decision: put them around slow widgets so the rest of the screen stays interactive.
- Prefer skeleton fallbacks to preserve layout and avoid CLS.
- Combine with transitions to avoid jarring fallback flashes during small interactions.

Example: progressive boundaries

```
<React.Suspense fallback={<PageSkeleton />}>
  <Header />
  <React.Suspense fallback={<SidebarSkeleton />}>
    <Sidebar />
  </React.Suspense>
  <React.Suspense fallback={<MainSkeleton />}>
    <Main />
  </React.Suspense>
</React.Suspense>
```

Interview-ready framing

“Suspense boundaries define where React can show a fallback while a subtree isn’t ready. Good placement enables progressive rendering and prevents whole-page loading states.”

63) What is server-side rendering (SSR)?

Thinking-level explanation

SSR is about generating HTML for the initial view on the server so the user sees content earlier (and SEO is easier). But SSR is not the end—React still hydrates on the client to attach interactivity.

Step-by-step

- 1) Server runs React rendering for a route and produces HTML.
- 2) Browser paints that HTML quickly (fast first content).
- 3) Client downloads JS, React hydrates: matches existing DOM and attaches event handlers/state.
- 4) After hydration, updates behave like a normal client-rendered app.

Misconceptions

- “SSR means no client JS” → hydration still needed for interactivity.
- “SSR always faster” → SSR can increase TTFB; you need caching/streaming for best UX.

Interview-ready framing

“SSR renders HTML on the server for fast first content and SEO, then hydration makes it interactive on the client.”

Upamanyu Deka

64) How does React handle asynchronous rendering?

Architecture-level explanation

With Fiber, React can split rendering into units of work and yield to the browser. This is the basis for time slicing and concurrency: React can start rendering, pause, resume, or abandon work based on priority.

Step-by-step flow

- 1) Updates are scheduled with priorities (lanes).
- 2) React begins render (reconciliation) for certain lanes.
- 3) If time runs out or higher priority arrives, React yields/pause.
- 4) React resumes later, or discards work-in-progress if superseded.
- 5) DOM is only updated during commit, which stays atomic (no partial UI).

Senior insight

The key is: asynchrony applies to the render phase (planning), not the commit phase (mutations). This prevents tearing.

Interview-ready framing

“React handles async rendering by time-slicing the render phase with Fiber, yielding based on priority, while keeping commits atomic for consistency.”

65) What is event delegation in React?

Thinking-level explanation

Event delegation means React doesn’t attach a separate DOM listener for every element. Instead, it listens at a higher level and dispatches events through React’s synthetic event system. This reduces listener overhead and enables consistent behavior across browsers.

Step-by-step

- 1) React registers event listeners on a root/container (implementation details vary by version).
- 2) When an event occurs, React receives it and creates a SyntheticEvent wrapper.
- 3) React runs capture/bubble handlers along the React tree path (not just DOM tree).
- 4) React batches updates triggered by event handlers.

Misconceptions

- “React always attaches only one listener” → conceptually yes via delegation; implementation can involve multiple listeners per event type.
- “Synthetic events are slower” → the overhead is usually negligible; the benefit is consistency and batching.

Interview-ready framing

“React uses event delegation with its synthetic event system: listen at the root, normalize events, and dispatch through the React tree with batching.”

66) How to handle long lists efficiently in React?

Thinking-level explanation

Long lists are often slow because DOM size/layout/paint dominate. The biggest win is usually virtualization: render only what’s visible.

Upamanyu Deka

Step-by-step approach

- 1) Use virtualization (react-window/react-virtualized) for large lists to keep DOM small.
- 2) Keep row components cheap: split rows, avoid heavy computations per item.
- 3) Stabilize item identity: use stable keys from data.
- 4) Memoize rows only after ensuring props are stable (otherwise memo does nothing).
- 5) Avoid layout thrash: prefer transforms; batch measurements.

Example: conceptual virtualization

```
// Pseudo (library-specific code varies):
// <FixedSizeList height={600} itemCount={items.length} itemSize={48}>
//   {({ index, style }) => <Row style={style} item={items[index]} />}
// </FixedSizeList>
```

Interview-ready framing

“For long lists, virtualization is the primary tool. Then I optimize row rendering, ensure stable keys, and profile to confirm.”

67) What is React Profiler?

Thinking-level explanation

React Profiler is a measurement tool (DevTools and API) to understand render frequency and commit durations. Senior usage is: measure → identify cause → fix → verify.

What you look for

- Frequent commits during input/scroll: likely state too high or context churn.
- Large commit duration: expensive render or too many DOM nodes.
- Re-renders without visible change: unstable props, derived state, or unnecessary parent renders.

Interview-ready framing

“Profiler tells me which components render and how long commits take. I use it to find hot spots and verify optimizations.”

68) How does useTransition improve performance?

Thinking-level explanation

useTransition improves perceived performance by keeping urgent interactions responsive while scheduling expensive UI updates as non-urgent. It doesn't make the expensive work faster; it makes the app feel smoother.

Step-by-step

- 1) Mark expensive state updates as a transition.
- 2) React schedules them in a lower-priority lane.
- 3) Urgent updates (typing/click feedback) render first.
- 4) Transition rendering can be interrupted and restarted.

Example

```
const [isPending, startTransition] = React.useTransition();
startTransition(() => setResults(expensiveFilter(input)));
```

Interview-ready framing

“useTransition keeps UI responsive by demoting heavy updates to non-urgent work that React can interrupt.”

69) What is startTransition and its benefits?

Thinking-level explanation

startTransition is the API to mark updates as non-urgent without needing a component hook. Benefits: smoother typing/navigation, fewer jarring loading states, and better interruption behavior under load.

Example (keep input urgent, results deferred)

```
function onChange(e) {
  const next = e.target.value;
  setQuery(next); // urgent
  React.startTransition(() => setResults(expensiveFilter(next))); // non-urgent
}
```

Misconceptions

- “It’s like debounce” → debounce is time-based; transition is priority/workload-based.
- “It makes rendering faster” → it improves responsiveness, not compute speed.

Interview-ready framing

“startTransition marks updates as non-urgent so React can prioritize responsiveness and interrupt work under load.”

70) How to handle error states in React hooks?

Thinking-level explanation

Error handling in hooks is about controlling the state machine: idle → loading → success → error, plus retry and cancellation. Seniors also separate UI state from server state and avoid unhandled promise rejections in effects.

Step-by-step pattern

- 1) Track status (loading/error/data) in state or useReducer.
- 2) In effects, handle cancellation/abort to avoid setState after unmount.
- 3) Provide retry mechanisms and user-friendly error UI.
- 4) For app-wide failures, use Error Boundaries (for render-time errors) and error reporting.

Example: reducer-based async state

```
function reducer(s, a) {
  switch (a.type) {
    case "start": return { ...s, loading: true, error: null };
    case "ok": return { loading: false, data: a.data, error: null };
    case "err": return { loading: false, data: null, error: a.error };
    default: return s;
  }
}
```

Interview-ready framing

“I model async hooks as a state machine, handle cancellation, show meaningful error UI with retry, and use Error Boundaries for render-time errors.”

Upamanyu Deka

71) What are the benefits of immutability in React?

Thinking-level explanation

Immutability is the mechanism that makes change detection cheap and reliable. React memoization patterns rely on referential equality; if you mutate objects in place, React (and your own comparisons) can’t confidently detect what changed.

Benefits that matter in real apps

- Correctness: prevents subtle bugs where UI doesn’t update because references didn’t change.
- Performance: enables shallow comparisons (React.memo, PureComponent, selectors).
- Predictability: easier debugging, time-travel patterns, and safe caching.

Example: mutation breaks memo

```
const Row = React.memo(({ user }) => <div>{user.name}</div>);

function App() {
  const [user, setUser] = React.useState({ name: "Ava" });

  const bad = () => { user.name = "Mia"; setUser(user); }; // ■
  const good = () => setUser(u => ({ ...u, name: "Mia" })); // ■

  return (<><Row user={user} /><button onClick={bad}>Bad</button><button onClick={good}>Good</button>
}
```

Interview-ready framing

“Immutability enables reliable change detection via reference changes, which makes memoization and predictable updates possible.”

72) How does React’s synthetic event system work?

Step-by-step flow

- 1) React registers delegated listeners at the root container.
- 2) Browser fires a native event; React intercepts it.
- 3) React creates a SyntheticEvent wrapper (normalization + consistent API).
- 4) React dispatches handlers along the React tree path (capture then bubble).
- 5) React batches state updates triggered by event handlers into fewer renders/commits.

Senior insights

- Synthetic events provide consistent behavior across browsers.
- Event pooling used to exist historically; modern React removed pooling—still good to know for legacy discussions.

Interview-ready framing

Upamanyu Deka

“React uses delegated listeners and a SyntheticEvent wrapper to normalize events, dispatch capture/bubble handlers through the React tree, and batch updates.”

73) What are StrictMode double renders?

Thinking-level explanation

In development, React StrictMode intentionally double-invokes certain functions (like rendering and effects) to surface unsafe side effects and non-idempotent logic. It helps you discover code that would break under concurrency (e.g., effects that aren’t properly cleaned up).

What gets double-invoked (practical)

- Render functions may run twice (development only).
- Effects may mount → cleanup → mount to ensure cleanup correctness.
- This does not happen in production in the same way (dev diagnostic behavior).

Misconceptions

- “My API is being called twice in production” → StrictMode double effects are dev-only; still fix because it indicates unsafe effect logic.

Interview-ready framing

“StrictMode double-invokes in dev to reveal unsafe side effects and missing cleanups—preparing code for concurrent rendering.”

74) Explain the render and commit phases in React.

Architecture-level explanation

Render is planning; commit is applying. Render computes the next UI tree (can be interrupted). Commit mutates the DOM atomically (not interruptible).

Step-by-step

- 1) Render/reconciliation: React runs components to compute a work-in-progress tree and effects list.
- 2) Commit: React applies DOM mutations and updates refs.
- 3) Layout effects run before paint; passive effects after paint.

Interview-ready framing

“Render computes the next tree (interruptible). Commit applies DOM changes atomically, then effects run in their phases.”

75) How does React manage memory during re-renders?

Thinking-level explanation

React uses a work-in-progress tree alongside the current tree (double buffering via fiber alternates). This allows React to build the next UI in memory without mutating the current UI until commit.

Step-by-step

- 1) Current fiber tree represents what’s on screen.
- 2) React builds a work-in-progress tree for updates.
- 3) After commit, the WIP becomes current; old nodes can be reused or garbage collected.
- 4) React reuses fiber objects via alternates to reduce allocations.

Upamanyu Deka

Senior insight

Your code influences memory too: large arrays in state, recreating big objects each render, and retaining closures in refs can increase memory pressure.

Interview-ready framing

“React uses a current + work-in-progress fiber tree (double buffering) to prepare UI updates safely, reusing structures where possible and letting old trees be collected after commit.”

76) What is key collision in React lists?

Thinking-level explanation

Key collision means multiple siblings share the same key. React uses keys for identity; collisions break identity mapping and can cause incorrect reuse, lost state, and unpredictable UI.

Typical causes

- Using non-unique IDs.
- Using array index in a list that can reorder/insert/delete.
- Generating keys from unstable values (Math.random).

Interview-ready framing

“Keys define identity. Collisions cause React to match the wrong nodes, leading to state and DOM mis-association. Use stable unique keys from data.”

77) What are some common React anti-patterns?

Thinking-level list (with why they hurt)

- Storing derived state: duplicates source-of-truth and gets out of sync.
- Mutating state in place: breaks memoization and change detection.
- Overusing context for frequent updates: causes broad re-renders.
- Effects that mirror props to state without guards: loops and stale bugs.
- Inline heavy computation in render: slow renders; should memoize or move work.
- Using index keys for reorderable lists: state bugs.

Interview-ready framing

“Anti-patterns are usually about breaking React’s assumptions: purity of render, immutability, correct identity (keys), and correct separation between derived values and side effects.”

78) How does React handle forms with validation libraries?

Thinking-level explanation

React doesn’t 'handle validation' by itself; it provides controlled/uncontrolled patterns. Validation libraries manage form state, validation rules, and rerender strategies—often optimizing to avoid rerendering the whole form on every keystroke.

What to evaluate (senior)

- Controlled vs uncontrolled approach used by the library.
- How it minimizes re-renders (field-level subscriptions).
- Validation timing: onChange vs onBlur vs onSubmit.
- Async validation and race handling (latest wins).

Upamanyu Deka

Interview-ready framing

“React provides input patterns; validation libraries manage form state + rules. I pick libraries that minimize re-renders with field-level subscriptions and support async validation safely.”

79) What are the drawbacks of context API?

Thinking-level explanation

Context is great for sharing values, but it’s not a selector-based store by default. The main drawback is update granularity: a provider value change can re-render many consumers.

Key drawbacks

- Broadcast updates: all consumers re-render when provider value identity changes.
- Harder performance tuning without splitting contexts/memoizing values.
- Encourages ‘mega context’ anti-pattern (everything in one provider).

Senior mitigation

- Memoize provider values, stabilize callbacks.
- Split contexts by concern.
- Use selector-based stores for frequent global updates.

Interview-ready framing

“Context is transport, not a store with selectors. Its drawback is broad re-renders on value changes; mitigate with memoization, splitting, or use a store.”

80) What is server component architecture?

Thinking-level explanation

Server Components let parts of your component tree run on the server and not ship their code to the client, reducing bundle size and enabling direct access to server resources. Client components remain the interactive islands.

Step-by-step mental model

- 1) Server components execute on the server, can read DB/secrets, and output a serialized representation.
- 2) Client components are boundaries that hydrate and handle events (useState/useEffect).
- 3) The tree is composed with clear server/client boundaries to optimize bundle size and data fetching.

Misconceptions

- “Server components replace SSR” → different: SSR is HTML generation; server components are about where components execute and what ships to the client.

Interview-ready framing

“Server component architecture splits the tree: non-interactive/data-heavy parts run on the server (no client bundle), while interactive parts remain client components.”

81) How does Suspense integrate with data fetching libraries?

Thinking-level explanation

Suspense integration means a data layer can 'pause' rendering when data isn’t ready, letting React show a fallback at the nearest boundary. Instead of manually juggling loading flags everywhere, the data layer coordinates readiness with rendering.

Step-by-step integration flow (conceptual)

- 1) Component reads data via a Suspense-aware API (library abstraction).
- 2) If data is not ready, the read suspends (conceptually throws a promise).
- 3) React catches it at the nearest Suspense boundary and shows fallback.
- 4) When the promise resolves, React retries rendering and commits the ready UI.

Senior insights

- Boundary placement is critical for good UX (avoid whole-page fallback).
- Combine with transitions to keep current UI while next data loads.
- You still need error boundaries (suspense is not error handling).

Interview-ready framing

“Suspense-aware data libraries integrate by suspending rendering until data is ready, letting boundaries control loading UX and enabling progressive rendering.”

82) Explain the useSyncExternalStore hook.

Concurrency-safe external subscriptions

useSyncExternalStore is the official way to subscribe to external mutable stores (Redux-like, custom stores) in a way that’s safe with concurrent rendering and avoids tearing.

Step-by-step contract

- subscribe(cb): register and call cb on store changes; return unsubscribe.
- getSnapshot(): read current store state consistently.
- React reads snapshots during render and can re-check for consistency under concurrency.

Interview-ready framing

“useSyncExternalStore provides concurrency-safe subscriptions to external stores by letting React read consistent snapshots and update without tearing.”

83) What is React’s concurrency model?

Thinking-level explanation

Concurrency in React is about *interruptible rendering* and *prioritized scheduling*—not multi-threading. React can start rendering an update, pause/yield, and resume or abandon it, while keeping commits atomic.

Step-by-step

- 1) Updates are assigned priorities (lanes).
- 2) React renders high-priority work first; low-priority work can be deferred.
- 3) Render can be paused/resumed to keep the main thread responsive.
- 4) Commit stays atomic to avoid inconsistent UI (tearing).

Interview-ready framing

“React’s concurrency model uses Fiber + lanes to prioritize and interrupt rendering while keeping commits atomic for UI consistency.”

84) What is React hydration mismatch and how to fix it?

Thinking-level explanation

Hydration mismatch occurs when the server-rendered HTML doesn’t match what the client would render on first pass. React warns and may re-render parts client-side.

Common causes

- Non-deterministic rendering (Date.now, Math.random, locale formatting) during render.
- Client-only data used during initial render (localStorage, window size).
- Server/client data differences (different initial payloads).

Fix strategies

- Make render deterministic: move non-determinism into effects.
- Align initial data on server and client (serialize/rehydrate).
- Isolate client-only UI behind a mounted flag (useEffect).

Interview-ready framing

“Hydration mismatch is server HTML != client initial render. Fix by removing non-determinism from render, aligning initial data, and isolating client-only logic.”

85) What is double buffering in React rendering?

Architecture-level explanation

Double buffering means React maintains two representations: the current committed tree and a work-in-progress (WIP) tree. React builds the next UI in WIP and swaps it in during commit.

Why it matters

- Allows interruptible rendering: React can prepare work without mutating the live UI.
- Enables consistency: only swap when a full coherent update is ready.

Interview-ready framing

“Double buffering (current + WIP tree) lets React build updates safely and commit atomically, which is essential for concurrency.”

86) How does React maintain UI responsiveness?

Thinking-level explanation

Responsiveness is about keeping the main thread available for user input and painting frames. React contributes by time-slicing render work, prioritizing urgent updates, batching, and enabling transitions.

Step-by-step levers

- 1) Prioritize urgent updates (input) over heavy UI updates (lanes).
- 2) Time-slice render work (yield) so long renders don’t block.
- 3) Batch updates to reduce commit frequency.
- 4) Use transitions/deferred values to keep typing smooth.
- 5) Reduce DOM work: virtualization, code-splitting, memoization where it matters.

Interview-ready framing

“React maintains responsiveness via prioritized scheduling and interruptible rendering, plus batching; the rest is app architecture: virtualization, splitting, and avoiding expensive renders.”

87) Explain the concept of time slicing.

Architecture-level explanation

Time slicing is React’s ability to break rendering into small units of work and yield control back to the browser, rather than blocking for a long synchronous render.

Step-by-step

- 1) Render work is split into fiber units.
- 2) React performs some units, then checks if it should yield (time budget).
- 3) Browser can handle input/paint.
- 4) React resumes later from where it left off.

Misconceptions

- “Time slicing means parallel threads” $\rightarrow$ it’s cooperative yielding on the main thread.

Interview-ready framing

“Time slicing keeps the UI responsive by allowing React to pause rendering work and yield to the browser, then resume later.”

88) What is cooperative scheduling?

Thinking-level explanation

Cooperative scheduling means React voluntarily yields control rather than monopolizing the main thread. The scheduler decides when to continue based on priority and available time.

Senior insight

This is why render must be pure: React might restart renders; side effects must not run during render.

Interview-ready framing

“Cooperative scheduling is React yielding rendering work back to the browser so input and paint remain responsive, then continuing later based on priority.”

89) How to measure React performance using Profiler API?

Thinking-level explanation

Upamanyu Deka

The Profiler API allows you to instrument parts of the tree and capture commit timings and render reasons programmatically, beyond DevTools.

Example usage (conceptual)

```
function onRender(id, phase, actualDuration, baseDuration, startTime, commitTime) {
  console.log({ id, phase, actualDuration, baseDuration, startTime, commitTime });
}

<React.Profiler id="SearchResults" onRender={onRender}>
  <SearchResults />
</React.Profiler>
```

Senior insight

Use it for regression monitoring in development builds, or to compare before/after changes. Don’t over-instrument production unless you have a strategy for sampling.

Interview-ready framing

“Profiler API gives commit timing telemetry for specific subtrees via onRender, helping identify and verify performance hot spots.”

90) What is React Fiber tree reconciliation?

Architecture-level explanation

Reconciliation is the process of turning a new render output into a plan of updates, matching the new element tree to the existing fiber tree (identity + structure), producing a WIP fiber tree and an effect list for commit.

Step-by-step

- 1) A render produces a new element description.
- 2) React compares it to the current fiber tree using heuristics (type + keys).
- 3) It reuses fibers when identity matches; replaces subtrees when types differ.
- 4) It records effects (insert/update/delete) to apply during commit.

Misconceptions

- “Reconciliation is DOM diffing” → it’s tree diffing/planning; DOM mutations happen in commit.

Interview-ready framing

“Fiber reconciliation matches new elements to existing fibers (using type/keys) to build a work-in-progress tree and an effect list that commit applies atomically.”

Upamanyu Deka

91) How does React coordinate updates between components?

What the interviewer is testing

They’re probing your model of data flow and scheduling: ownership of state, parent/child rendering, context propagation, and how React batches and prioritizes updates so multiple components stay consistent.

Thinking-level explanation

React coordinates updates by treating UI as a function of state. When state changes anywhere, React schedules a render of the smallest necessary part of the tree, computes a consistent next tree, then commits changes atomically. Coordination is not ‘components talking to each other’—it’s shared state ownership and deterministic propagation.

Step-by-step coordination flow (architecture-level)

- 1) Update source: setState/useReducer dispatch, context value change, external store subscription, or parent props change.
- 2) Update is enqueued on a fiber with a priority (lane).
- 3) React finds the root and schedules work for the affected lanes.
- 4) Render phase: React re-runs components along the affected paths to compute the next element/fiber tree.
- 5) Reconciliation decides which subtrees need updates and can bail out (memo/PureComponent).
- 6) Commit phase applies all DOM mutations atomically so the UI doesn’t show partial states.
- 7) Effects run after commit (layout before paint; passive after paint), keeping side effects isolated from rendering.

How multiple components stay consistent

- Single owner state: siblings coordinate by lifting state to the closest common owner.
- Context/store: multiple consumers update together based on the same provider/store snapshot.
- Batching: multiple setState calls within the same tick/event are grouped into one render+commit.

Misconceptions (follow-up traps)

- “React updates each component immediately when setState runs” → updates are scheduled and often batched.
- “Parent re-render means DOM changes everywhere” → re-render is compute; DOM changes are minimal and committed selectively.
- “Context is like global state with selectors” → context broadcasts to consumers; selectors require a store pattern.

Senior-level insight

Coordination is easier when you design state boundaries: keep state local by default, lift only for shared coordination, and separate server state from UI state. In concurrent rendering, React can coordinate urgent vs non-urgent updates via lanes while keeping commits atomic to avoid tearing.

Upamanyu Deka

Interview-ready framing

“React coordinates updates by scheduling state changes on fibers with priorities, computing a consistent next tree during render, and committing DOM changes atomically. Components coordinate through state ownership (lifted state), context, and stores—plus batching.”

92) What are transitions in React 18?

What the interviewer is testing

They want: urgent vs non-urgent updates, interruptibility, and why transitions improve responsiveness without making computation faster.

Thinking-level explanation

A transition is a way to mark certain updates as non-urgent. React can prioritize urgent updates (typing, clicking, hover feedback) over expensive UI updates (filtering lists, rendering heavy routes). Transitions are interruptible: if the user triggers more urgent work, React can pause/abandon the in-progress transition render and restart with newer inputs.

Step-by-step flow

- 1) A user action triggers both an urgent update (e.g., input value) and a heavy update (e.g., filter results).
- 2) You wrap the heavy update in startTransition/useTransition.
- 3) React schedules that update in a lower-priority lane.
- 4) Urgent updates render first to keep the UI responsive.
- 5) Transition work renders when time allows; it can be interrupted and restarted.

Example: keep typing responsive while filtering

```
function Search({ items }) {
  const [q, setQ] = React.useState("");
  const [results, setResults] = React.useTransition(items);
  const [isPending, startTransition] = React.useTransition();

  const onChange = (e) => {
    const next = e.target.value;
    setQ(next); // urgent

    startTransition(() => {
      setResults(items.filter(x => x.name.toLowerCase().includes(next.toLowerCase())));
    });
  };

  return (
    <div>
      <input value={q} onChange={onChange} />
      {isPending && <span>Updating…</span>}
      {results.map(x => <div key={x.id}>{x.name}</div>)}
    </div>
  );
}
```

Misconceptions

- “Transitions run on another thread” → no; they’re scheduled at lower priority on the main thread.
- “Transitions make work faster” → they improve responsiveness by prioritizing urgent updates, not raw speed.

Upamanyu Deka

Senior insight: UX strategy

Transitions are best when you want the UI to stay interactive even if the next screen/data is expensive. Combine with Suspense boundaries for smooth loading patterns.

Interview-ready framing

“Transitions mark non-urgent updates so React can keep urgent interactions responsive. They’re interruptible, priority-based scheduling—not multi-threading.”

93) How to handle slow network rendering gracefully?

What the interviewer is testing

They want a UX + architecture answer: loading states, skeletons, progressive rendering, caching, avoiding waterfalls, and how Suspense/transitions help.

Thinking-level principles

- Avoid blank screens: show stable layout skeletons to prevent CLS and reduce anxiety.
- Make progress visible: show partial content as soon as it’s ready (progressive rendering).
- Avoid refetch storms: cache server state and dedupe requests.
- Avoid waterfalls: fetch in parallel when possible; prefetch likely next navigation.

Step-by-step strategy (practical and senior)

- 1) Use a server-state cache (React Query/SWR/framework) to dedupe, cache, retry, and handle stale-while-revalidate.
- 2) Show skeleton fallbacks for expensive areas; keep app shell stable.
- 3) Use optimistic UI for actions (like, add to cart) and reconcile on response.
- 4) Use transitions for navigation/filters that may suspend, so UI remains responsive.
- 5) Prefetch code/data on intent (hover, intersection observer, idle).
- 6) Handle error states: timeouts, retry buttons, offline indicators.

Example: skeleton + retry (conceptual)

```
function Users() {
  const [retryKey, setRetryKey] = React.useState(0);
  // conceptually: useQuery(["users", retryKey], fetchUsers)
  // if (isLoading) return <UsersSkeleton />
  // if (error) return <ErrorState onRetry={() => setRetryKey(k => k + 1)} />
  return <div>{/* users list */}</div>;
}
```

Misconceptions

- “A spinner is enough” → spinners often cause layout shifts and feel slower than skeletons.
- “Just increase timeout” → better to design retries, caching, and progressive UI.

Interview-ready framing

“For slow networks I use stable skeletons, progressive rendering, server-state caching with retries, prefetching to avoid waterfalls, and transitions/Suspense to keep navigation responsive.”

94) Explain React’s commit priorities.

What the interviewer is testing

They want you to distinguish: priorities affect *render scheduling*, but commits are atomic. Also, they'll probe for sync vs concurrent, lanes, and what can be interrupted.

Thinking-level explanation

React assigns priorities (lanes) to updates. This strongly affects the render phase: which work React does first and whether it yields. Commit is intentionally not interruptible: once React commits, it applies a consistent set of mutations. So 'commit priorities' is really about which completed render gets committed first and how urgent updates preempt background work.

Step-by-step: how priorities influence what commits

- 1) Updates are assigned lanes (priority) and scheduled.
- 2) React renders the highest-priority lanes first; lower-priority work may be postponed.
- 3) When a render completes for a set of lanes, React commits that result.
- 4) If higher-priority updates arrive, React may abandon in-progress lower-priority renders before they reach commit.
- 5) Effects run after commit (layout then passive), preserving consistency.

Senior insight: why commit must stay atomic

Atomic commit prevents tearing—showing partially updated UI. React may render multiple versions in memory (work-in-progress), but it only commits a consistent snapshot.

Misconceptions

- "React commits partial UI while rendering" → no; commits are atomic.
- "Priority means commit can be paused" → priority affects scheduling before commit; commit itself is not interruptible.

Interview-ready framing

"React prioritizes *render work* via lanes. Lower-priority renders can be paused/abandoned, but commits are atomic; the highest-priority completed render is committed to keep UI consistent."

95) What is lazy initialization in useState?

Thinking-level explanation

useState(initial) evaluates the initial value on the first render only. If computing that initial value is expensive, you can pass a function to compute it lazily—so the expensive work runs only once, not on every render.

Example: expensive init

```
function expensiveInit() {
  // pretend this is heavy: parsing, big computation
  const raw = localStorage.getItem("settings");
  return raw ? JSON.parse(raw) : { theme: "light", pageSize: 20 };
}

function App() {
  // ■ function form: expensiveInit runs only once
  const [settings, setSettings] = React.useState(() => expensiveInit());
  return <pre>{JSON.stringify(settings, null, 2)}</pre>;
}
```

Misconceptions

- “useState(expensiveInit()) runs once” → it runs on every render because JS evaluates arguments before calling useState.
- “Lazy init improves re-render performance” → it only improves initial render work; re-renders depend on other factors.

Interview-ready framing

“Lazy initialization is passing a function to useState so expensive initial computation runs only on mount, not every render.”

96) How to manage derived state correctly?

What the interviewer is testing

They want you to avoid the classic anti-pattern: storing something that can be computed from props/state. They’ll also probe for memoization, reducers, and when derived state is justified.

Thinking-level rule

If a value can be computed from existing props/state at render time, don’t store it in state. Stored derived state risks getting out of sync. Compute it during render or memoize if expensive.

Step-by-step decision process

- 1) Ask: can I compute this from current props/state? If yes, derive during render.
- 2) If derivation is expensive, use useMemo with correct dependencies.
- 3) Only store derived state if you need to 'snapshot' a value across time (e.g., previous props), or if it’s user-editable separate from source.
- 4) When you must sync state from props, do it carefully with guards to avoid loops and stale values.

Example: derive vs store

```
// ■ storing derived value
const [full, setFull] = React.useState("");
React.useEffect(() => setFull(first + " " + last), [first, last]);

// ■ derive
const fullName = React.useMemo(() => first + " " + last, [first, last]);
```

Legit derived state case: user-editable draft

If server provides an initial profile, but user edits a draft, you keep draft state separate and reset only when appropriate (e.g., userId changes).

```
function ProfileEditor({ user }) {
  const [draft, setDraft] = React.useState(() => user);

  React.useEffect(() => {
    // reset draft when switching users
    setDraft(user);
  }, [user.id]); // guard on identity, not entire user object

  // draft is intentionally separate from user
  return <input value={draft.name} onChange={e => setDraft(d => ({...d, name: e.target.value}))} />;
}
```

Misconceptions

- “Derived state is always bad” → it’s bad when it duplicates source-of-truth without a reason.
- “Sync props to state in useEffect by default” → often unnecessary; derive instead.

Interview-ready framing

“I avoid derived state duplication. I compute derived values during render (useMemo if expensive). I only store derived state for drafts/snapshots and sync carefully with guarded dependencies.”

97) How does React isolate side effects?

Architecture-level idea

React isolates side effects by enforcing that rendering is pure and by running effects in well-defined phases after commit. This prevents side effects from corrupting reconciliation and enables features like time slicing and re-render retries.

Step-by-step: where side effects can happen

- Render phase: should be pure; no subscriptions, no DOM mutations, no data fetching that causes external effects.
- Commit phase: apply DOM mutations (React-controlled side effects).
- Layout effects (useLayoutEffect): run after commit before paint; for layout reads/writes.
- Passive effects (useEffect): run after paint; for subscriptions, network, timers, logging.

Why this matters in concurrent rendering

Because React may start rendering, pause, and even abandon a render. If side effects ran during render, you could run them multiple times or for renders that never commit. By isolating side effects to post-commit phases, React keeps external systems consistent.

Misconceptions

- “useEffect is the lifecycle” → it’s for syncing external systems; timing is deliberate.
- “Effects run once” → effects can run multiple times in dev StrictMode; your cleanup must be correct.

Interview-ready framing

“React keeps render pure and runs side effects only after commit (layout before paint, passive after paint). This isolation is critical for concurrency because renders can be paused/abandoned safely.”

98) Explain batching strategy in concurrent mode.

What the interviewer is testing

They want you to connect batching with scheduling/lanes: grouping updates, prioritizing them, and how React avoids excessive commits while staying responsive.

Thinking-level explanation

Batching groups multiple updates into fewer renders/commits. In concurrent mode, batching is aligned with priorities: React can batch updates within the same priority lane, and it can delay non-urgent work to avoid blocking urgent interactions.

Step-by-step

- 1) Multiple updates occur (event handlers, async callbacks).
- 2) React queues updates and groups them (automatic batching in React 18 across more async boundaries).
- 3) Updates are assigned lanes (priorities).
- 4) React renders higher-priority lanes first; lower-priority updates may wait and get batched together.
- 5) React commits a consistent snapshot for the lanes it completed.

Upamanyu Deka

Example: batching + functional updates

```
function Counter() {
  const [c, setC] = React.useState(0);

  const onClick = () => {
    setC(x => x + 1);
    setC(x => x + 1);
    setC(x => x + 1);
  };

  return <button onClick={onClick}>{c}</button>;
}
```

Senior insight

In concurrent rendering, React may render work-in-progress trees multiple times before committing. Batching reduces commit frequency and keeps the app responsive by allowing React to wait for more low-priority updates and do them together.

Misconceptions

- “Batching means state updates are async” → batching groups; scheduling determines flush timing.
- “Batching removes the need for functional updates” → functional updates still prevent stale state when updates depend on previous state.

Interview-ready framing

“React batches updates to reduce redundant renders/commits. In concurrent mode, batching aligns with priorities (lanes): urgent work flushes sooner, non-urgent work can be delayed and grouped, while commits remain consistent snapshots.”

99) How does React decide which components to re-render?

Thinking-level explanation

React doesn’t re-render based on DOM diffs; it re-runs components to compute a new tree. The decision is driven by update propagation: a component re-renders when its state changes, its props change, or the context/store snapshot it reads changes.

Step-by-step (what triggers re-render)

- 1) Local state update: setState/useReducer dispatch schedules re-render of that component’s fiber.
- 2) Parent re-render: child component function may re-run because parent returns it again.
- 3) Context update: consumers re-render when provider value identity changes.
- 4) External store update: subscriber triggers update (ideally via useSyncExternalStore).
- 5) Force update in legacy scenarios.

How React avoids unnecessary work (bailouts)

- React.memo/PureComponent: shallow compare props to skip re-render.
- Key stability: preserves identity and avoids remounts.
- State colocation: reduces the number of components affected by frequent state changes.

Example: parent causes child re-run unless memoized

```
const Child = React.memo(() => {
  console.log("Child render");
  return <div>Child</div>;
});
```

Upamanyu Deka

```
function Parent() {
  const [n, setN] = React.useState(0);
  return (
    <>
      <Child />
      <button onClick={() => setN(x => x + 1)}>Inc {n}</button>
    </>
  );
}
```

Misconceptions

- “React re-renders only changed components” → parents can cause child re-execution; bailouts decide if it continues.
- “Memoization always helps” → only if props are stable; otherwise it’s overhead.

Interview-ready framing

“React re-renders when state/props/context/store snapshots change. Parent renders can re-run children; React avoids waste via bailouts (memo/PureComponent) and good state scoping.”

100) How does React handle layout thrashing?

First: define the real problem (thinking-level)

Layout thrashing happens when code alternates between reading layout (which may force the browser to flush layout) and writing DOM styles repeatedly, causing expensive reflow/repaint cycles. This is a browser rendering pipeline issue, not purely React-specific.

How React helps (and where it can’t)

- React batches DOM mutations during commit, which can reduce interleaved read/write patterns compared to ad-hoc DOM manipulation.
- But if your code (effects) reads layout and writes styles repeatedly, you can still thrash.
- React recommends separating measurement (layout reads) into useLayoutEffect and minimizing writes.

Step-by-step: avoiding thrash in React

- 1) Avoid repeated layout reads in loops. Cache measurements.
- 2) Separate reads from writes: read all sizes first, then apply writes.
- 3) Use requestAnimationFrame for animation updates; avoid synchronous loops.
- 4) Prefer CSS transforms for animations (less layout) instead of top/left changes.
- 5) Use useLayoutEffect for initial measurement to avoid flicker, but keep it lightweight.

Example: read-then-write pattern

```
React.useLayoutEffect(() => {
  const nodes = Array.from(document.querySelectorAll(".item"));

  // READ phase
  const rects = nodes.map(n => n.getBoundingClientRect());

  // WRITE phase
  rects.forEach((r, i) => {
    nodes[i].style.transform = `translateY(${Math.round(r.top)}px)`;
  });
}, []);
```

Misconceptions

Upamanyu Deka

- “React prevents all layout thrashing automatically” → no; effects can still cause it.
- “useLayoutEffect is always the fix” → it can worsen performance if heavy; it blocks paint.

Interview-ready framing

“Layout thrashing is caused by interleaving layout reads and DOM writes. React helps by batching commits, but you still must avoid read/write alternation in effects—read all measurements first, then write, use transforms, and keep layout effects minimal.”

Upamanyu Deka

101) What are React’s internal linked lists (Fiber nodes)?

Thinking-level explanation: Fiber is a tree encoded as linked lists

React Fiber represents the component tree as a set of nodes where traversal is optimized for incremental work. Instead of a simple recursive tree, Fiber encodes relationships using pointers (linked-list style) so React can pause/resume rendering and traverse without deep recursion.

Key pointers (how the structure works)

- child: first child fiber (down pointer).
- sibling: next sibling fiber (side pointer).
- return: parent fiber (up pointer).
- alternate: the 'other' version of the node (current vs work-in-progress) for double buffering.

Why linked-list pointers matter

- Efficient DFS traversal without recursion (avoid call stack limits).
- Ability to yield mid-traversal and resume later (cooperative scheduling).
- O(1) navigation among parent/child/siblings, which is perfect for incremental reconciliation.

Step-by-step: what React does during render traversal

- 1) Start at the root work-in-progress fiber.
- 2) Perform work on a fiber (beginWork): compute next children.
- 3) Go to child if it exists; else completeWork and move to sibling; else bubble to parent via return.
- 4) Repeat until the unit-of-work budget is exhausted (yield) or work completes.

Senior insight: effect lists & flags

Fibers also carry flags/effect tags indicating what mutations are needed in commit (placement, update, deletion). Historically React built explicit effect linked lists; modern versions use flags and subtree flags to collect work efficiently during traversal.

Misconceptions

- “Fiber is the Virtual DOM” → Fiber is React’s internal work structure; elements are the 'what', fibers are the 'how'.
- “Linked list means slow” → the pointer structure is chosen for incremental traversal and scheduling efficiency.

Interview-ready framing

“Fiber encodes the component tree using child/sibling/return pointers (linked-list style) plus alternates for double buffering. This enables incremental traversal, yielding, and atomic commits.”

102) How does React implement cooperative multitasking?

Thinking-level explanation

Upamanyu Deka

Cooperative multitasking means React voluntarily yields control back to the browser so input and paint can happen, instead of blocking the main thread with a long synchronous render. React does this by breaking rendering into units of work (fibers) and using the scheduler to decide when to continue.

Step-by-step flow

- 1) An update is scheduled with a priority (lane).
- 2) React begins rendering work by processing fibers (units of work).
- 3) After some work, React checks 'shouldYield' (time budget / higher priority pending).
- 4) If it should yield, React pauses and returns control to the browser (input/paint).
- 5) React resumes later from the next fiber unit, preserving progress in the WIP tree.

Senior insight: why render must be pure

Because React can pause or abandon in-progress renders, render must not cause side effects. Side effects are isolated to post-commit phases (layout/passive effects).

Misconceptions

- “Cooperative multitasking is multithreading” → no; it’s single-threaded yielding on the main thread.
- “React yields during commit” → commit is not interruptible; only render can yield.

Interview-ready framing

“React implements cooperative multitasking by time-slicing the render phase into fiber units, yielding based on scheduler time/priority, and resuming later—while keeping commits atomic.”

103) Explain the architecture of React’s scheduler.

Thinking-level view: a priority-based task queue

React’s scheduler is a small priority-based task runner that coordinates when React should perform work. It does not render the UI itself—it orchestrates *when* rendering work runs, so React can remain responsive.

Key concepts (what exists)

- Tasks with priorities (immediate, user-blocking, normal, low, idle).
- Expiration times / timeouts (to prevent starvation).
- A work loop that runs tasks until it should yield.
- A host callback mechanism (e.g., MessageChannel) to continue work in future ticks.

Step-by-step: how scheduling typically works

- 1) React requests the scheduler to run work at a certain priority.
- 2) Scheduler enqueues a task in a priority queue.
- 3) Scheduler starts a host callback to process tasks.
- 4) Work loop executes tasks; periodically checks if it should yield.
- 5) If it yields, it schedules another callback to continue later.

Senior insight: relationship to lanes

Think of lanes as React’s internal prioritization model for updates. Scheduler priorities decide when CPU time is allocated; lanes decide which updates are included in a render. They work together: scheduler gives time; lanes choose work.

Misconceptions

Upamanyu Deka

- “Scheduler is the same as concurrent mode” → scheduler is a mechanism; concurrency is how React uses it with Fiber + lanes.
- “Scheduler guarantees exact frame timing” → it cooperates with the browser; it aims for responsiveness, not deterministic frame budgets.

Interview-ready framing

“React’s scheduler is a priority-based task queue that runs work until it should yield, then reschedules. It allocates CPU time; React lanes decide which updates to process.”

104) What is React’s diff optimization heuristic?

Thinking-level explanation

React’s diff is not a full tree-edit-distance algorithm. That would be too slow. Instead, React uses heuristics that are fast and match common UI patterns. The core heuristic: if element type changes, React replaces the subtree; if type matches, React updates in place and diffs children using keys.

Step-by-step: reconciliation heuristics

- 1) Same component type (and key for siblings) → reuse existing fiber/DOM and update props.
- 2) Different type → throw away old subtree and mount a new one.
- 3) For lists: match children by key (stable identity). Without keys, match by index order.
- 4) Moves are detected in keyed lists by comparing old indices to new positions; insertions/deletions create placement/deletion effects.

Senior insight: why keys are correctness, not just performance

Keys define identity across renders. Wrong keys cause React to match the wrong nodes, which can attach state/DOM to the wrong item (bugs).

Misconceptions

- “React compares DOM nodes” → it compares element/fiber trees and plans mutations; DOM changes happen in commit.
- “React can detect moved elements without keys” → without keys, it mostly assumes positional matching.

Interview-ready framing

“React uses fast heuristics: same type+key means reuse and update; different type means replace subtree. Children are reconciled by keys for stable identity; otherwise by position.”

105) How do React hooks work internally?

Thinking-level explanation: hooks are a per-fiber linked list of 'hook cells'

Internally, hooks are stored on the fiber as a linked list (or list-like structure) of hook objects. Each hook call corresponds to a slot in that list. During render, React walks the hook list in call order to retrieve/update state and queue updates.

Step-by-step render of a function component with hooks

- 1) React sets the currently rendering fiber as context.
- 2) For each hook call, React either mounts a new hook cell (first render) or reads the next existing hook cell (updates).
- 3) useState returns state from that hook cell and a dispatch that queues updates onto that hook’s update queue.

Upamanyu Deka

- 4) After rendering, queued updates are processed to compute the next state for subsequent renders.

Conceptual model (very simplified)

```
// Pseudo mental model
fiber.memoizedState = hook1 -> hook2 -> hook3 ...

useState():
  currentHook = nextHookInList()
  state = currentHook.memoizedState
  dispatch = (action) => enqueueUpdate(currentHook.queue, action)
```

Senior insight: why this design enables composition

Because hooks are order-based slots, you can compose logic by calling custom hooks. The hook cells are still linear slots in the component render, regardless of how deep the custom hook is.

Misconceptions

- “Hooks are magic global variables” → they’re stored per component instance (fiber).
- “Hooks depend on names” → they depend on call order, not names.

Interview-ready framing

“Hooks are stored as an ordered list of hook cells on the fiber. Each render replays hooks in the same order to read/update state and enqueue updates.”

106) How does React identify hook order consistency?

Thinking-level explanation

React relies on hook call order to map each hook invocation to its stored hook cell. If the order changes between renders (due to conditional hooks), React will read the wrong hook cells—state gets misaligned. So React enforces: hooks must be called unconditionally and in the same order.

Step-by-step: what goes wrong if order changes

- 1) Render #1: useState (slot 1), useEffect (slot 2), useState (slot 3).
- 2) Render #2 (conditional): useState (slot 1), useState (slot 2) ... now slot 2 is not the same hook type.
- 3) React reads the wrong hook cell for the second call → incorrect state/effect mapping.

How React catches it

In development, React includes additional checks and warnings (Rules of Hooks). Lint rules (eslint-plugin-react-hooks) catch most cases at build time. At runtime, React detects mismatch when traversing hooks and finding unexpected null/extra hooks or differing hook types in DEV.

Example: conditional hook bug

```
function Example({ enabled }) {
  const [a] = React.useState(0);
  if (enabled) {
    React.useEffect(() => { /* ... */ }, []); // ■ conditional hook
  }
  const [b] = React.useState(1);
  return null;
}
```

Interview-ready framing

“React maps hook calls to stored hook cells by call order. Changing order breaks that mapping, so hooks must be unconditional; React/lint tooling enforces this and DEV runtime can warn on mismatches.”

Upamanyu Deka

107) What are React lanes and priorities in concurrent mode?

Thinking-level explanation: lanes are bitmasks representing priority buckets

Lanes are React's internal way to represent update priorities and to group updates. Each lane corresponds to a priority class (urgent vs transition vs idle). Using a bitmask allows React to represent multiple pending lanes simultaneously and decide what to render next.

Step-by-step: how lanes are used

- 1) An update is assigned to a lane (based on context: event, transition, etc.).
- 2) Root tracks pending lanes as a bitmask.
- 3) Scheduler/React picks the highest priority lanes to render.
- 4) React renders those lanes into a WIP tree; lower lanes can wait.
- 5) After commit, completed lanes are cleared.

Senior insight: why lanes matter

Lanes enable: interruption (urgent lanes preempt others), batching within priority, and starvation avoidance (lanes can expire). They're a more scalable model than a single global priority.

Misconceptions

- "Lanes are threads" → lanes are priority categories, not execution threads.
- "All lanes render together" → React selects a set of lanes based on priority; others can be deferred.

Interview-ready framing

"Lanes are React's internal priority buckets represented as bitmasks. Updates are assigned lanes; React picks lanes to render based on urgency, enabling interruption, batching, and fairness."

108) How does React perform time slicing for rendering?

Thinking-level explanation

Time slicing is React splitting render work into small units (fibers) and yielding between them to keep the main thread available. It's implemented via the Fiber work loop and the scheduler's shouldYield checks.

Step-by-step render loop (conceptual)

- 1) React starts work on the next unit (fiber).
- 2) After processing some units, React checks shouldYield().
- 3) If shouldYield is true, React pauses and schedules continuation.
- 4) Later, React resumes from the next unit using the WIP tree state.
- 5) Once render completes, React commits atomically.

Pseudo-code mental model

```
while (nextUnitOfWork && !shouldYield()) {
  nextUnitOfWork = performUnitOfWork(nextUnitOfWork);
}
if (nextUnitOfWork) scheduleContinuation();
else commitRoot();
```

Misconceptions

- "Time slicing applies to commit" → commit is synchronous/atomic.
- "Yielding guarantees 60fps" → it improves responsiveness but can't overcome extreme CPU/DOM workload alone.

Upamanyu Deka

Interview-ready framing

“React time-slices by processing fibers as units of work and yielding when the scheduler says, then resuming later—committing only when a consistent render completes.”

109) What is SuspenseList and how is it used?

Thinking-level explanation

SuspenseList coordinates the reveal order of multiple Suspense boundaries. Without it, children may appear as they resolve (which can look jittery or out-of-order). SuspenseList lets you control whether content reveals together or in a specified order to improve UX.

How it works (conceptual step-by-step)

- 1) You wrap multiple Suspense boundaries in a SuspenseList.
- 2) React tracks which children are ready vs suspended.
- 3) Based on revealOrder (e.g., forwards, backwards, together), React decides which children can reveal.
- 4) Tail handling controls whether unresolved items show fallback or stay hidden.

Example (conceptual API)

```
// API details depend on support in your setup/version
// <SuspenseList revealOrder="forwards" tail="collapsed">
//   <Suspense fallback={<RowSkeleton />}><Row1 /></Suspense>
//   <Suspense fallback={<RowSkeleton />}><Row2 /></Suspense>
// </SuspenseList>
```

Senior insight

It’s a UX coordination tool: choose it when partial reveals cause confusing layout shifts or when you want progressive but ordered reveal (e.g., feed items).

Misconceptions

- “SuspenseList improves performance” → it mainly improves perceived UX and reveal consistency.

Interview-ready framing

“SuspenseList coordinates multiple Suspense boundaries so content reveals in a controlled order (forwards/backwards/together), improving UX and reducing jittery partial loads.”

110) How do Suspense boundaries affect data fetching?

Thinking-level explanation

Suspense boundaries don’t fetch data by themselves; they define the UI behavior when a subtree is not ready. In Suspense-enabled data fetching, the data layer can 'suspend' rendering until data is ready, and the nearest boundary decides what the user sees (fallback vs existing UI).

Step-by-step: what happens with Suspense-aware data reads

- 1) Component tries to read data during render via a Suspense-aware API.
- 2) If data isn’t available, rendering suspends (promise).
- 3) React shows fallback at the nearest boundary (or keeps prior UI if combined with transitions).
- 4) Data resolves; React retries render; boundary swaps fallback for real UI atomically.

Senior-level insights (follow-up ready)

- Boundary placement controls waterfall vs parallel feel: coarse boundaries can blank too much UI; fine boundaries enable progressive rendering.
- Combine with transitions to avoid jarring fallback flashes on navigation/filtering.
- You still need error boundaries: Suspense handles waiting, not failures.
- Caching is essential: without caching, retries can refetch and thrash; good data libraries cache promises/results.

Misconceptions

- “Suspense replaces loading state everywhere” → only if your data layer supports Suspense and you design boundaries properly.
- “Suspense handles errors” → errors need ErrorBoundary patterns.

Interview-ready framing

“Suspense boundaries control the UX when a Suspense-aware data read isn’t ready. They enable progressive rendering, but require caching, good boundary placement, and error boundaries for failures.”

Upamanyu Deka

111) Explain how React queues state updates internally.

Thinking-level explanation: updates are enqueued on fibers and processed during render

When you call setState/useState dispatch, React doesn't immediately change the state and re-render synchronously (in most cases). Instead, it creates an update object and enqueues it on the component's fiber (and specifically on the hook/state update queue). Later, during the render phase, React processes the queue to compute the next state.

Step-by-step internal flow (conceptual)

- 1) dispatch(setState) creates an Update (action + lane/priority + timestamp).
- 2) Update is appended to the component's update queue (for hooks: a per-hook queue; for classes: fiber update queue).
- 3) The root is marked with pending lanes; React schedules work for that root.
- 4) During render, React clones fibers into a work-in-progress tree and processes each fiber's update queue.
- 5) Updates are reduced in order to compute the new memoized state for that fiber/hook.
- 6) After render completes, commit applies DOM mutations; the computed state becomes the new current state.

Why queues exist (senior insight)

Queues enable batching, priority scheduling, and interruption. React can collect many updates, decide which lanes to process, and compute a consistent result without mutating the live UI mid-render.

Misconceptions

- "setState immediately updates state" → state becomes available in the next render (except some sync flush cases).
- "Updates are always processed in call order" → they're ordered, but lanes/priorities affect which updates are included in a render pass.

Interview-ready framing

"React queues state updates as update objects on fibers/hook queues with lanes. During render it processes the queue to compute next state, then commits atomically."

112) What are the trade-offs of React's virtual DOM model?

Thinking-level explanation

Virtual DOM is a means to an end: it lets React describe UI declaratively and compute changes in memory before touching the real DOM. The trade-offs are about CPU work in JS vs fewer DOM operations, plus the abstraction benefits and costs.

Major benefits (why it works)

- Declarative model: UI = f(state) reduces manual DOM bookkeeping bugs.
- Batching: compute many changes then apply a minimal DOM commit.

Upamanyu Deka

- Portability: React’s renderer abstraction enables React Native and custom renderers.
- Predictability: reconciliation provides consistent update behavior.

Major trade-offs (what you pay)

- Extra JS work: reconciliation and rendering costs CPU; for very simple pages, direct DOM can be faster.
- Memory overhead: fibers/elements represent UI structures in JS.
- Not a free performance win: if you render huge trees often, VDOM still costs; you need architecture (memoization, virtualization).
- Abstraction leaks: understanding render vs commit is necessary for performance and correctness.

Senior insight: when DOM is the bottleneck vs JS

For large lists, the bottleneck is usually DOM/layout/paint. Even if VDOM is efficient, you still need virtualization to reduce DOM nodes. For complex computations, memoization helps more.

Misconceptions

- “VDOM is faster than DOM” → not always; it reduces *unnecessary* DOM writes, but adds JS overhead.

Interview-ready framing

“VDOM enables declarative UI and batched DOM commits, but it adds JS/memory overhead. Performance still depends on render frequency, tree size, and DOM cost—often requiring memoization and virtualization.”

113) How does React handle reconciliation at scale?

Thinking-level explanation

At scale, reconciliation is about doing less work: keeping identity stable (keys), avoiding re-rendering huge subtrees, and leveraging Fiber’s ability to prioritize and interrupt work. React’s diffing is O(n) per sibling list with heuristics, so the key is reducing how often and how much you reconcile.

Step-by-step: what 'scale' means in practice

- 1) Tree size: thousands of nodes → DOM + reconciliation cost rises.
- 2) Update frequency: rapid updates (typing/scroll) can cause repeated renders.
- 3) Data volume: big lists require virtualization; memoization alone doesn’t shrink DOM.

Senior levers React provides (and how you use them)

- Stable keys and types: preserve identity to reuse fibers/DOM correctly.
- Bailouts: React.memo/PureComponent to skip expensive subtrees when props unchanged.
- State colocation: keep frequent state local to avoid propagating re-renders.
- Concurrency primitives: transitions/deferred values to prioritize responsiveness.
- Virtualization: limit mounted DOM for huge lists.

Example: isolate frequent updates

```
function SearchPage() {
  return (
    <div>
      <SearchBox />   {/* frequent keystrokes */}
      <Results />     {/* expensive list */}
    </div>
  );
```

Upamanyu Deka

```
}

// Keep SearchBox state local; only pass stable query when needed.
```

Misconceptions

- “React handles scale automatically” → you must architect state boundaries and list rendering.

Interview-ready framing

“At scale, React relies on stable identity, bailouts, localized state, and concurrency scheduling. For huge lists, virtualization is often the key. Reconciliation is fast, but the goal is to reconcile less.”

114) Explain how React batches updates in concurrent rendering.

Thinking-level explanation

Batching means React groups multiple updates into fewer renders/commits. In concurrent rendering, batching also interacts with lanes: React can collect low-priority updates and process them together, while allowing urgent updates to preempt.

Step-by-step

- 1) Multiple updates are enqueued (possibly across async boundaries in React 18).
- 2) Each update is assigned a lane (priority).
- 3) React schedules work for the root and chooses which lanes to render next.
- 4) Updates in the same lane are reduced together during render.
- 5) React commits once per completed render snapshot, reducing DOM thrash.
- 6) If urgent work arrives, React may pause/abandon lower-priority render and later batch more work into it.

Example: functional updates stay correct under batching

```
setCount(c => c + 1);
setCount(c => c + 1);
setCount(c => c + 1); // results in +3 reliably
```

Misconceptions

- “Batching makes state updates asynchronous” → batching groups updates; scheduling decides when they flush.
- “Batching can reorder updates” → updates are processed in a deterministic order within lanes; priorities affect inclusion timing.

Interview-ready framing

“In concurrent rendering, React batches updates and schedules them by lanes. It can delay and group non-urgent work, while urgent updates preempt, and commits remain atomic.”

115) How does React pause and resume rendering?

Thinking-level explanation

React pauses/resumes by keeping render work in a work-in-progress fiber tree. Because fibers encode traversal state via pointers, React can stop after some units of work and later continue from the next fiber, without losing progress.

Step-by-step

- 1) React begins rendering a set of lanes into a WIP tree.

Upamanyu Deka

- 2) It processes fibers as units of work.
- 3) Scheduler signals shouldYield; React stops the work loop.
- 4) React retains the WIP tree + next unit pointer.
- 5) Later, React resumes the work loop from the saved pointer.
- 6) If a higher-priority update arrives, React may discard the WIP and restart with the new lanes.

Misconceptions

- “React can pause during commit” → commit is synchronous and not interruptible.
- “Paused renders show partial UI” → partial work stays in memory; UI changes only on commit.

Interview-ready framing

“React pauses/resumes in the render phase by time-slicing fiber units, storing progress in the work-in-progress tree, and resuming later; commits remain atomic.”

116) What are React’s scheduler APIs (requestIdleCallback, postTask)?

Thinking-level explanation

React’s scheduler is implemented in userland and uses host scheduling primitives to run and continue work without blocking the browser. Historically, requestIdleCallback was explored but is unreliable for strict responsiveness; React commonly uses mechanisms like MessageChannel (macro-task scheduling) and time slicing. Newer web APIs like scheduler.postTask can provide better priority scheduling when available.

Step-by-step: what React needs from the host

- A way to schedule continuation of work (run soon, but not immediately).
- A way to yield and resume (break long work into chunks).
- A way to represent priorities (urgent vs idle) and avoid starvation.

Practical senior insight

You don't need to memorize exact internal implementation details, but you should know the principle: React cooperates with the browser using scheduling primitives; availability differs by environment.

Misconceptions

- “React depends on requestIdleCallback” → not strictly; React uses multiple primitives and fallbacks.
- “postTask is required” → it’s a newer capability; React still works without it.

Interview-ready framing

“React’s scheduler relies on host primitives to schedule and yield work (e.g., MessageChannel/timeouts, and potentially newer APIs like postTask). The goal is priority-aware cooperative scheduling, not hard real-time guarantees.”

117) How does React handle cross-component dependencies?

Thinking-level explanation

React itself avoids implicit cross-component dependencies. Coordination happens through explicit data flow: props/state ownership, context, and external stores. At architecture level, you manage dependencies by choosing the right shared owner or state mechanism.

Step-by-step patterns

- Sibling coordination: lift state to the closest common ancestor; pass data down and callbacks up.

Upamanyu Deka

- Many consumers: use context for coarse-grained shared values (auth/theme).
- Frequent shared updates: use a store with selectors to avoid broad re-renders (useSyncExternalStore-based).
- Side-effect coordination: centralize in a data layer (query cache) rather than component-to-component effects.

Example: lift state for dependency

```
function Parent() {
  const [selectedId, setSelectedId] = React.useState(null);
  return (
    <>
      <List selectedId={selectedId} onSelect={setSelectedId} />
      <Details id={selectedId} />
    </>
  );
}
```

Misconceptions

- “Components directly trigger each other’s renders” → renders are driven by state/props/context changes.

Interview-ready framing

“React handles cross-component dependencies via explicit state ownership (lifting), context for shared concerns, and external stores with selector patterns for high-frequency global state.”

118) How does React ensure UI consistency across threads?

First, correct the premise

In the browser, React rendering generally happens on the main thread. React concurrency is not multi-threaded by default; it’s interruptible rendering on one thread. However, 'consistency across threads' is really about ensuring the UI does not show partial states while work is in progress.

Thinking-level explanation: atomic commits + consistent snapshots

- Atomic commit: React only mutates the DOM when it has a complete, consistent render result.
- Consistent snapshots: external stores use useSyncExternalStore to avoid tearing (reading inconsistent values mid-render).
- Purity of render: since render can be paused/retried, side effects are isolated after commit.

Step-by-step

- 1) React computes WIP UI in memory; current UI stays on screen.
- 2) If interrupted, WIP is paused or discarded; UI doesn’t partially change.
- 3) When ready, React commits atomically, producing a consistent UI snapshot.
- 4) Effects run after commit; external stores provide consistent snapshots to prevent tearing.

Misconceptions

- “React renders on multiple threads” → not typically; concurrency is cooperative scheduling.
- “Partial renders can show” → only committed snapshots affect the DOM.

Interview-ready framing

“React ensures consistency by keeping render pure and interruptible, computing updates in a WIP tree, and committing DOM changes atomically. For external stores, useSyncExternalStore prevents tearing.”

Upamanyu Deka

119) What happens in React’s commit phase in detail?

Thinking-level explanation: commit is where React touches the outside world

Commit is the non-interruptible phase where React applies the effects of reconciliation to the host environment (DOM). If render is planning, commit is execution.

Step-by-step commit phases (high-level)

- 1) Before-mutation: capture snapshots (e.g., getSnapshotBeforeUpdate in classes).
- 2) Mutation phase: apply DOM insert/update/remove operations; update refs.
- 3) Layout phase: run useLayoutEffect and componentDidMount/Update.
- 4) Schedule passive effects: useEffect runs after paint.

What’s important for interviews

- Commit is atomic and not interruptible.
- DOM mutations are applied based on flags/effects computed during render.
- Refs are updated during commit so they match the committed DOM.

Misconceptions

- “useEffect runs during commit” → passive effects are scheduled after paint.
- “Commit can be paused” → no; only render can yield.

Interview-ready framing

“Commit applies DOM mutations atomically, updates refs, runs layout effects/lifecycles, and schedules passive effects. It’s the non-interruptible phase that makes the planned update real.”

120) Explain fiber reconciliation with examples.

Architecture-level explanation

Fiber reconciliation is React matching the newly rendered element tree to the existing fiber tree to build a work-in-progress fiber tree and a set of mutation flags. It’s optimized with heuristics: type+key identity first; replace subtrees when types change; reconcile siblings by keys.

Step-by-step reconciliation

- 1) Render produces new elements for a component.
- 2) React compares new element type/key to current fiber.
- 3) If same identity: reuse fiber and update props; reconcile children.
- 4) If different identity: create new fiber and mark old for deletion.
- 5) Build effect flags (placement/update/deletion) to apply in commit.

Example 1: type change replaces subtree

```
// Old: <div><A /></div>
// New: <div><B /></div>
// Since A and B types differ, React discards A subtree and mounts B subtree.
```

Example 2: keyed list preserves identity

```
// Old: [ {id: 'a'}, {id: 'b'} ] -> keys a,b
// New: [ {id: 'b'}, {id: 'a'} ] -> keys b,a
// React moves fibers based on keys; state stays with the correct item.
```

Common pitfalls

Upamanyu Deka

- Index keys with insertions cause state to 'move' to different rows.
- Generating random keys forces remounts and loses state.

Interview-ready framing

“Fiber reconciliation matches new elements to existing fibers by type and key, reuses where possible, replaces subtrees on type changes, and records mutation flags for an atomic commit.”

121) How does React schedule priority-based updates?

Thinking-level explanation: lanes + scheduler = priority-based rendering

Priority scheduling in React comes from two layers working together: (1) React lanes decide which updates are urgent vs non-urgent and which can be delayed; (2) the scheduler decides when to give React CPU time to perform work without blocking the browser.

Step-by-step flow

- 1) An update is triggered (setState, context change, store update).
- 2) React assigns the update to a lane based on origin: discrete events (high), transitions (low), idle work (lowest).
- 3) The root tracks pending lanes in a bitmask.
- 4) React asks the scheduler to run work at an appropriate priority.
- 5) React renders the highest-priority lanes first; lower-priority lanes wait.
- 6) If a higher priority update arrives, React can pause/abandon in-progress low-priority rendering and restart.
- 7) When render finishes, commit applies the result atomically.

Senior insight: 'urgent' is about UX, not importance

Urgent means the user is waiting for feedback (typing, clicking). Non-urgent (transitions) means it's okay if the UI updates shortly after, as long as it stays responsive.

Misconceptions

- “Priority scheduling is multithreading” → it’s cooperative yielding on one thread.
- “React always finishes renders in order” → renders can be interrupted and restarted; commits stay consistent.

Interview-ready framing

“React schedules priority updates by assigning lanes and using the scheduler to time-slice rendering. Urgent lanes preempt, transitions defer, and commits remain atomic.”

122) How does React diff keyed lists efficiently?

Thinking-level explanation: O(n) mapping with identity keys

React uses a linear-time heuristic for lists. Keys allow React to map old children to new children by identity rather than position, enabling correct moves and preserving component state.

Step-by-step algorithm (conceptual)

- 1) React first walks old and new lists in order and reuses items while keys match (fast path).
- 2) When it hits a mismatch, it builds a map from remaining old children: key → old fiber.
- 3) It walks the rest of the new children: for each key, reuse/move the matching old fiber if present; otherwise create a new fiber.

Upamanyu Deka

- 4) Any old fibers not reused are marked for deletion.
- 5) Placement/move flags are recorded to apply in commit.

Why this is efficient

The 'map only when needed' approach avoids building maps for every list update and keeps common cases fast.

Example: stable keys preserve state across moves

```
// Old order: a, b, c  (keys: a,b,c)
// New order: b, a, c  (keys: b,a,c)
// React matches by keys -> swaps fibers for a and b without remounting them.
```

Misconceptions

- “Index keys are fine” → only safe if list order never changes and items are never inserted/removed.
- “Keys are only for performance” → keys are correctness for identity and state preservation.

Interview-ready framing

“React diffs keyed lists by matching identity keys in O(n): fast-path sequential matching, then key→fiber mapping for the remainder, recording moves/insertions/deletions for commit.”

123) What are microtask queues and how do they affect React updates?

Thinking-level explanation: microtasks change *when* updates flush

Microtasks (Promise callbacks, queueMicrotask) run after the current JS call stack but before the browser paints and before macrotasks (setTimeout). They matter to React because update batching and flushing boundaries can depend on when your code runs (event, microtask, macrotask). React 18 expanded automatic batching across more async boundaries, but timing still affects UX and paint.

Step-by-step: where microtasks sit

- 1) JS call stack runs (event handler).
- 2) Microtasks run (Promise.then).
- 3) Browser may render/paint a frame.
- 4) Macrotasks run (setTimeout, MessageChannel callbacks).

How this affects React (practical)

- Multiple setState calls within the same tick (including many microtasks) can batch into fewer renders.
- If you need DOM to paint before doing work, microtasks can delay paint because they run before rendering.
- React’s scheduler may schedule continuation using macrotasks to allow paint/input between chunks.

Example: microtask can delay paint (conceptual)

```
button.onclick = () => {
  setLoading(true); // schedule UI update
  Promise.resolve().then(() => {
    // heavy sync work here can delay paint of loading state
    for (let i = 0; i < 1e8; i++) {}
  });
};
```

Senior guidance

If you need to yield to paint, avoid doing heavy work in microtasks; use requestAnimationFrame, setTimeout, or transitions to keep UX responsive.

Upamanyu Deka

Interview-ready framing

“Microtasks run before paint, so they can affect when React updates become visible. React batches more updates in React 18, but heavy microtask work can still block paint and make UI feel laggy.”

124) Explain the architecture of React Server Components.

Thinking-level explanation

Server Components (RSC) split the component graph into two execution environments: server and client. Server components run on the server, can access backend resources, and don’t ship their JS to the client. Client components are interactive islands that hydrate and handle events.

Step-by-step architecture flow (high level)

- 1) Server renders server components and produces a serialized component payload (not just HTML).
- 2) Client receives this payload and reconstructs the component tree for display.
- 3) Client components act as boundaries: their code is shipped, and they hydrate for interactivity.
- 4) Server components can stream in as data becomes available; Suspense boundaries control reveal.

Key constraints (interview follow-ups)

- Server components cannot use client-only hooks like useState/useEffect (no browser).
- Client components can import server components only via boundaries dictated by the framework bundler rules.
- Data fetching can happen directly in server components without client waterfalls.

Senior insight: what RSC optimizes

RSC primarily optimizes bundle size and data-fetching topology (fewer client waterfalls). It changes 'what ships to client' and 'where data is fetched'.

Misconceptions

- “RSC is SSR” $\rightarrow$ SSR is HTML generation; RSC is where components execute and what code ships.
- “RSC removes hydration” $\rightarrow$ client components still hydrate for interactivity.

Interview-ready framing

“RSC architecture splits the tree: server components run on the server and stream a serialized UI payload without shipping JS; client components are interactive boundaries that hydrate.”

125) How does React isolate rendering work from user interaction?

Thinking-level explanation: prioritize input + yield

Isolation means user interactions stay responsive even when the app has heavy rendering to do. React achieves this by prioritizing discrete events and by time-slicing the render phase so it can yield to input/paint.

Step-by-step

- 1) Discrete events (click/type) are scheduled at high priority lanes.
- 2) Non-urgent work (transitions) is scheduled at lower priority.
- 3) Scheduler yields during render to let input handlers and paint run.
- 4) React can interrupt low-priority rendering if new urgent input arrives.
- 5) UI changes commit atomically, avoiding partial updates.

Example: keep typing responsive with transition

```
const [isPending, startTransition] = React.useTransition();

function onChange(e) {
  const next = e.target.value;
  setQuery(next); // urgent
  startTransition(() => setResults(expensiveFilter(next))); // non-urgent
}
```

Misconceptions

- “React isolates work automatically for all code” → your own heavy synchronous code can still block input; avoid heavy work in event handlers/microtasks.

Interview-ready framing

“React isolates rendering from interaction by prioritizing discrete events, time-slicing rendering, and allowing low-priority work to be interrupted—keeping input responsive.”

126) How does React optimize hydration performance?

Thinking-level explanation

Hydration is attaching React to existing server-rendered DOM. Hydration performance matters because the page can look ready but feel unresponsive until hydration attaches handlers. React optimizes hydration by allowing selective/partial hydration and by deferring non-critical hydration work when possible (framework integration).

Step-by-step (conceptual)

- 1) Server sends HTML for fast first paint.
- 2) Client loads JS; React starts hydration, matching fibers to existing DOM.
- 3) React attaches event listeners and sets up state without recreating DOM.
- 4) Suspense boundaries can delay hydration of parts until data/code is ready.
- 5) Frameworks can use streaming/partial hydration to hydrate incrementally.

Senior insights (what you do in apps)

- Use code-splitting to reduce hydration JS cost.
- Keep initial client component tree small; move non-interactive UI to server components where possible.
- Avoid non-determinism that causes hydration mismatch and forces client re-render.
- Use Suspense boundaries to avoid blocking hydration on slow data.

Interview-ready framing

“React optimizes hydration by matching existing DOM, attaching handlers efficiently, and enabling selective hydration via boundaries. Practical wins come from smaller client bundles, fewer client components, and avoiding hydration mismatches.”

127) How does React handle memory leaks in async hooks?

Thinking-level explanation: React doesn’t magically prevent them—you must cleanup

React isolates side effects to effects, but it cannot automatically prevent memory leaks caused by uncleaned subscriptions, pending timers, or async callbacks retaining references. The core pattern is: cleanup on dependency change/unmount, and prevent stale async completions from setting state.

Upamanyu Deka

Step-by-step safe patterns

- Abort/ignore stale fetches using AbortController or request-id guards.
- Cleanup subscriptions (WebSocket, event listeners) in the effect cleanup function.
- Clear timers/intervals in cleanup.
- Avoid retaining large objects in refs/closures unnecessarily.

Example: abort + safe setState

```
React.useEffect(() => {
  const c = new AbortController();
  (async () => {
    const res = await fetch("/api/data", { signal: c.signal });
    const json = await res.json();
    setData(json);
  })().catch(e => {
    if (e.name !== "AbortError") console.error(e);
  });
  return () => c.abort();
}, []);
```

Misconceptions

- “React 18 fixes memory leaks” → it can surface issues in StrictMode, but your code must clean up.

Interview-ready framing

“React isolates side effects in effects, but preventing leaks is on you: clean up subscriptions/timers and cancel or ignore stale async work to avoid setState after unmount.”

128) Explain Suspense’s role in streaming SSR.

Thinking-level explanation: stream HTML progressively with boundaries

In streaming SSR, Suspense boundaries allow the server to send the shell HTML immediately while deferring slower parts. When a suspended part becomes ready, the server streams in the HTML for that boundary and the client 'reveals' it without waiting for the entire page.

Step-by-step

- 1) Server begins rendering the page.
- 2) When it hits a Suspense boundary with unavailable data/code, it emits the fallback HTML instead of blocking.
- 3) The rest of the page continues rendering and streams to the client.
- 4) When the suspended data becomes ready, server streams the real content for that boundary.
- 5) Client swaps fallback → real content for that boundary (hydration ties in later).

Senior insight: why this matters

It reduces TTFB-to-content gaps and prevents slow data from blocking the whole response. Proper boundary placement yields huge UX wins.

Misconceptions

- “Suspense is just a client loading spinner” → in streaming SSR it’s a server rendering control mechanism too.

Interview-ready framing

“Suspense enables streaming SSR by letting the server send fallbacks immediately and stream resolved boundary content later, achieving progressive rendering.”

Upamanyu Deka

129) How does React integrate with frameworks like Next.js?

Thinking-level explanation

Frameworks orchestrate React’s rendering modes and data loading: routing, SSR/SSG, streaming, server components, bundling client/server boundaries, and deployment concerns. React provides the rendering primitives; Next.js provides the app/runtime model.

Step-by-step integration points

- Routing + rendering mode selection (SSR/SSG/ISR/CSR).
- Data fetching model and caching (server functions, fetch caching, revalidation).
- Bundling and splitting for client vs server components (RSC).
- Streaming SSR with Suspense boundaries for progressive HTML.
- Hydration strategies and script loading prioritization.

Senior insight

In modern Next.js, the key is understanding server/client component boundaries: what runs on server, what hydrates, and how to place Suspense boundaries for UX and performance.

Interview-ready framing

“Next.js integrates by providing routing, data fetching/caching, SSR/streaming, and RSC bundling rules. React supplies the component model and Suspense/concurrency primitives; Next orchestrates them.”

130) What is the difference between legacy and concurrent rendering?

Thinking-level explanation

Legacy rendering is primarily synchronous: once React starts rendering an update, it runs to completion and blocks the main thread. Concurrent rendering allows React to interrupt and resume rendering work, prioritizing urgent updates and keeping the UI responsive.

Step-by-step comparison

- Legacy: render work is mostly uninterrupted; long renders block input/paint.
- Concurrent: render can be time-sliced; React can yield and resume; urgent updates can preempt.
- Both: commit is atomic; DOM updates happen in commit.

What changes for developers

- Render must be pure (no side effects) because renders can be restarted/abandoned.
- Effects must be resilient with cleanups (StrictMode helps surface issues).
- Use transitions/deferred values to classify non-urgent work.

Misconceptions

- “Concurrent means parallel rendering on multiple cores” → not necessarily; it’s about interruptibility on the main thread.

Interview-ready framing

“Legacy rendering is synchronous and blocking; concurrent rendering is interruptible and priority-based, keeping UI responsive while still committing updates atomically.”

Upamanyu Deka

131) How does React manage deferred values?

Thinking-level explanation: deferred values are a priority-based lagging view of state

A deferred value (useDeferredValue) lets React keep urgent UI updates responsive while allowing expensive derived UI to render using a slightly older value when the main thread is busy. This is not time-based (like debounce). It’s priority-based: React may delay updating the deferred value under load.

Step-by-step flow

- 1) You update an 'urgent' value (e.g., input text).
- 2) React computes a deferred version that can lag behind under heavy rendering.
- 3) Expensive subtrees render from the deferred value, reducing jank while typing.
- 4) When React has time, deferred value catches up and expensive subtree updates.

Example: type smoothly, render expensive list later

```
function Search({ items }) {
  const [q, setQ] = React.useState("");
  const dq = React.useDeferredValue(q);

  const filtered = React.useMemo(
    () => items.filter(x => x.name.toLowerCase().includes(dq.toLowerCase())),
    [items, dq]
  );

  return (
    <>
      <input value={q} onChange={e => setQ(e.target.value)} />
      {filtered.map(x => <div key={x.id}>{x.name}</div>)}
    </>
  );
}
```

Misconceptions

- “Deferred means delayed by N ms” → it’s workload/priority-driven, not time-based.
- “It makes filtering faster” → it improves responsiveness; total compute still exists.

Senior insight

useDeferredValue is best for derived rendering that can safely be 'eventually consistent' (like search results), not for critical UI like form validation errors.

Interview-ready framing

“Deferred values provide a priority-based lagging view of a value so expensive derived UI can update later, keeping urgent interactions responsive.”

132) How does React handle component unmounts during rendering?

Thinking-level explanation: unmount is a commit-time effect; render can be aborted

Upamanyu Deka

In concurrent rendering, React may start rendering a subtree and later decide it should not commit it (because higher priority work arrived or the state changed). Unmounting (and cleanup) is associated with commits: React only runs unmount cleanup for components that were actually committed and are being removed from the committed tree.

Step-by-step scenarios

- Scenario A – Aborted render: React started rendering an update, then abandons it before commit. No effects were committed, so no unmount cleanup runs for that abandoned WIP tree.
- Scenario B – Committed then removed: a component was on screen (committed), then later the committed tree removes it → React runs cleanup (useEffect cleanup / componentWillUnmount) in commit.

Senior insight: why effect cleanup correctness matters

StrictMode dev behavior (mount → cleanup → mount) helps catch effects that leak because in concurrency a component can mount/unmount more often than you expect due to transitions and retries.

Misconceptions

- “Unmount happens during render” → render is planning; unmount side effects run during commit.
- “Aborted renders run cleanups” → no; if never committed, effects were never established in the committed UI.

Interview-ready framing

“React may abandon renders, but unmount cleanups only run for components removed from the committed tree during commit. Aborted WIP work doesn’t trigger unmount cleanups.”

133) How does React optimize repeated renders via memoization trees?

Thinking-level explanation: reuse work by bailing out subtrees when inputs are stable

React reduces repeated work by reusing previous results when it can prove inputs haven’t changed. This happens at multiple levels: element identity (type/key), fiber reuse, and component-level memoization (React.memo/PureComponent) that triggers bailouts.

Step-by-step: what happens on re-render with memoization

- 1) Parent re-renders; it returns elements for children.
- 2) Reconciliation matches by type+key to reuse child fibers.
- 3) For memoized children, React shallow-compares props; if equal, it bails out of rendering that subtree.
- 4) Bailout means React skips calling the component function and also skips reconciling its children (huge savings).
- 5) Commit applies minimal DOM changes only where something actually changed.

Example: memo works only with stable props

```
const Child = React.memo(function Child({ options }) {
  return <pre>{JSON.stringify(options)}</pre>;
});

function Parent({ theme }) {
  const options = React.useMemo(() => ({ theme }), [theme]); // stabilizes identity
  return <Child options={options} />;
}
```

Misconceptions

Upamanyu Deka

- “React.memo always helps” → if props change identity every render (inline objects/functions), memo won’t bail out.
- “Memoization prevents parent re-render” → it only skips rendering the memoized child subtree.

Senior insight

If you over-memoize, you add comparison overhead and complexity. Optimize by profiling: bail out expensive subtrees, not everything.

Interview-ready framing

“React optimizes repeated renders by reusing fibers and bailing out memoized subtrees when props are stable, skipping component execution and child reconciliation.”

134) What is React’s bailout mechanism?

Thinking-level explanation: a bailout is React deciding 'this subtree cannot have changed'

A bailout is when React skips work for a fiber/subtree because it determines there’s no need to re-render it. Bailouts can happen due to unchanged props/state, memoization boundaries, and unchanged context dependencies.

Step-by-step: common bailout path

- 1) React reaches a fiber during render.
- 2) It checks if there are updates scheduled for that fiber (lanes) or if props/context changed.
- 3) If no relevant updates and inputs are referentially equal, React bails out.
- 4) Bailout reuses previous child fibers and skips reconciling deeper.

Where bailouts come from

- React.memo / PureComponent shallow compare props.
- No scheduled updates for the fiber’s lanes.
- Context optimization: component only re-renders when the contexts it read changed.

Misconceptions

- “Bailout means DOM doesn’t change” → DOM change is decided at commit; bailout means React skipped computation for that subtree.
- “Bailout is always correct” → it’s correct only if your app follows immutability and stable references.

Interview-ready framing

“Bailout is React skipping rendering and reconciliation for a subtree when it can prove inputs haven’t changed—enabled by lanes, memoization, and referential equality.”

135) How does React’s Fiber tree store alternate versions?

Architecture-level explanation: double buffering via fiber.alternate

React maintains two versions of the fiber tree: the current committed tree and a work-in-progress (WIP) tree. Each fiber has an alternate pointer to its counterpart in the other tree. This is double buffering: build the next UI without mutating what’s on screen.

Step-by-step

- 1) Current fiber represents committed UI.
- 2) On update, React creates or reuses an alternate as WIP for that fiber.
- 3) React writes new props/state/effect flags to the WIP fibers during render.

Upamanyu Deka

- 4) After render completes, commit swaps the root's current pointer to the WIP tree.
- 5) The previous current becomes the alternate for future updates.

Senior insight

Alternates reduce allocations by reusing fiber objects across renders. They also enable interruption: WIP can be thrown away if superseded.

Misconceptions

- "Alternate means a diff copy of DOM" → it's React's internal structure, not the DOM.

Interview-ready framing

"Fiber stores alternates via fiber.alternate, forming a current and WIP tree. React builds updates on WIP, then swaps atomically during commit."

136) How does React integrate with non-React systems?

Thinking-level explanation: treat external systems as side-effectful resources

Integration usually means syncing React state with an external imperative system (DOM libraries, maps, charts, analytics, legacy code). The key is to isolate imperative interactions to effects and to define a clear ownership boundary: React owns the container; the external system owns its internal DOM/state.

Step-by-step pattern

- 1) Render a container element (div/ref).
- 2) Initialize the external system in useEffect/useLayoutEffect using the ref.
- 3) Update the external system when relevant props/state change (effect with deps).
- 4) Cleanup on unmount (dispose listeners, destroy instances).

Example: integrating a chart library

```
function Chart({ data }) {
  const ref = React.useRef(null);

  React.useEffect(() => {
    const chart = createChart(ref.current); // imperative init
    return () => chart.destroy();           // cleanup
  }, []);

  React.useEffect(() => {
    updateChart(ref.current, data);         // imperative update
  }, [data]);

  return <div ref={ref} />;
}
```

Misconceptions

- "Just manipulate DOM in render" → render must be pure; use effects for DOM side effects.
- "useEffect vs useLayoutEffect doesn't matter" → layout effects run before paint; use carefully for measurement/flicker avoidance.

Interview-ready framing

"I integrate non-React systems by isolating imperative init/update/cleanup in effects, using refs as boundaries, and keeping React render pure."

137) How do Suspense and transitions interact?

Upamanyu Deka

Thinking-level explanation: transitions prevent jarring fallback switches

Suspense controls what to show when a subtree isn't ready; transitions mark an update as non-urgent so React can keep showing the previous UI while preparing the next. Together, they make loading feel smoother: instead of instantly replacing UI with a fallback, React can keep the current screen until the new one is ready.

Step-by-step pattern

- 1) User triggers navigation/filter likely to suspend.
- 2) Wrap state update in startTransition.
- 3) React renders next UI in background; if it suspends, it can keep current UI visible.
- 4) When ready, React commits the new UI; optionally show pending indicator.

Example (conceptual)

```
const [isPending, startTransition] = React.useTransition();
<button onClick={() => startTransition(() => setRoute("reports"))}>Reports</button>

<Suspense fallback={<PageSkeleton />}>
  {route === "reports" ? <Reports /> : <Home />}
</Suspense>
```

Misconceptions

- "Transitions stop Suspense fallback entirely" → they influence timing/priority; boundaries still matter.

Interview-ready framing

"Suspense decides fallback UI when not ready; transitions mark updates as non-urgent so React can keep prior UI during suspension, reducing jarring loading states."

138) How to handle Suspense boundaries in nested trees?

Thinking-level explanation: boundaries define progressive reveal layers

Nested Suspense boundaries allow you to control loading at different granularities. The goal is to keep as much of the UI stable as possible while only showing fallback for the slow subtree.

Step-by-step boundary placement strategy

- 1) Put a top-level boundary for route-level loading (rarely triggered if you also use nested boundaries).
- 2) Add nested boundaries around slow widgets (sidebar, comments, chart) so they can load independently.
- 3) Prefer skeleton fallbacks that preserve layout to avoid shifts.
- 4) Avoid too many tiny boundaries if it makes UX noisy (too many spinners).

Example: progressive boundaries

```
<Suspense fallback={<ShellSkeleton />}>
  <Header />
  <Suspense fallback={<SidebarSkeleton />}>
    <Sidebar />
  </Suspense>
  <main>
    <Suspense fallback={<FeedSkeleton />}>
      <Feed />
    </Suspense>
    <Suspense fallback={<CommentsSkeleton />}>
      <Comments />
    </Suspense>
```

Upamanyu Deka

```
    </main>
  </Suspense>
```

Senior insight

Boundary placement is a product decision. You balance perceived speed (more boundaries) against visual stability and complexity.

Interview-ready framing

“Use nested Suspense boundaries to progressively reveal independent parts. Place boundaries around slow subtrees, use stable skeleton fallbacks, and avoid making loading UI too noisy.”

139) What are React’s internal debugging tools?

Thinking-level answer: know what you can use in real projects

In practice, React debugging is a combination of official DevTools and profiling/tracing mechanisms. Internally, React has build-time feature flags and debug modes, but interviews typically want what you’d use day-to-day plus what exists conceptually.

Tools you actually use

- React Developer Tools (Components tab): inspect props/state/hooks, highlight updates.
- React Profiler (DevTools): measure commits, render durations, identify hot paths.
- Profiler API (): programmatic timings for subtrees.
- StrictMode: surfaces unsafe side effects (double-invokes in dev).
- Error Boundaries: capture render-time errors and component stack traces.

Senior insights

- Use the Profiler before optimizing; avoid guessing.
- Add lightweight logging around transitions/pending states to understand UX stalls.
- In production: use error reporting (Sentry) with source maps and React component stack traces.

Misconceptions

- “console.log is enough” → useful but misses commit timing and render reasons; profiler is essential.

Interview-ready framing

“My go-to debugging tools are React DevTools + Profiler, StrictMode to catch unsafe effects, Profiler API for targeted telemetry, and error boundaries/reporting for production diagnostics.”

140) How does React track hook dependencies efficiently?

Thinking-level explanation: deps are stored and compared by reference

For hooks like useEffect/useMemo/useCallback, React stores the dependency array from the previous render in the hook cell. On the next render, it compares each dependency using Object.is (shallow reference equality). If all dependencies are equal, React can skip re-running the effect or recomputing the memoized value.

Step-by-step

- 1) First render: hook stores deps array (and computed value/effect).
- 2) Next render: React compares new deps[i] to previous deps[i] via Object.is.
- 3) If any differs, it marks the effect to run (or recomputes memo).
- 4) If all equal, it bails out and reuses previous value/effect.

Upamanyu Deka

Example: why inline objects break deps

```
function Comp({ user }) {
  const opts = { theme: "dark" }; // new object every render
  React.useEffect(() => {
    // runs every render because opts identity changes
  }, [opts]);
}
```

Senior guidance

You fix dependency churn by stabilizing references (useMemo/useCallback), but only when it prevents meaningful work. Over-stabilizing adds overhead.

Misconceptions

- “Deps are deep-compared” → no; it’s reference equality.
- “Missing deps is harmless” → missing deps can cause stale closures and correctness bugs.

Interview-ready framing

“React tracks deps by storing the previous deps array per hook and comparing entries with Object.is. Stable references enable bailouts; unstable inline objects/functions cause repeated effect runs.”

141) Explain React’s coordination between rendering and layout.

Thinking-level explanation: React separates 'compute' from 'measure/mutate'

React’s key coordination trick is phase separation: it computes the next UI in the render phase (pure, interruptible), then mutates the DOM in commit (atomic), then runs layout-sensitive work in layout effects before paint.

Step-by-step flow across the browser pipeline

- 1) Render (reconciliation): React computes what the UI *should be* for a state snapshot. No DOM reads/writes.
- 2) Commit (mutation): React performs DOM insert/update/delete operations and updates refs.
- 3) Layout effects: useLayoutEffect runs after DOM mutations but before the browser paints. Safe for measurements.
- 4) Paint: browser calculates layout/paint.
- 5) Passive effects: useEffect runs after paint (good for IO/subscriptions).

Why this prevents layout thrash

Layout thrash happens when code alternates DOM writes and reads that force synchronous layout. React’s model encourages: write DOM in commit, then measure in layout effects in a controlled window, rather than repeatedly in render or random callbacks.

Example: correct measurement pattern

```
function Tooltip() {
  const ref = React.useRef(null);
  const [w, setW] = React.useState(0);

  React.useLayoutEffect(() => {
    setW(ref.current.getBoundingClientRect().width); // measure after DOM update, before paint
  }, []);

  return <div ref={ref}>Width: {w}</div>;
}
```

Misconceptions

- “You can safely read layout in render” → render can run multiple times and should be pure; layout reads can cause forced reflow.
- “useEffect is fine for measurement” → it runs after paint, can cause visible flicker if you adjust layout afterward.

Interview-ready framing

“React coordinates rendering and layout by separating phases: render plans without touching DOM, commit mutates atomically, layout effects measure before paint, and passive effects run after paint.”

142) How does React manage re-entrant rendering?

Upamanyu Deka

Thinking-level explanation: React guards against nested work and treats renders as restartable

Re-entrant rendering means starting a new render while already rendering (e.g., updates triggered during render or while processing work). React's concurrency model assumes renders can be interrupted/restarted, but it still enforces boundaries: render must be pure, and updates during render are either warned about, queued, or deferred to avoid corrupting invariants.

Step-by-step: what happens when an update happens 'during render'

- 1) React is rendering a component tree for a given lanes set.
- 2) If code triggers setState during render (anti-pattern), React detects it.
- 3) React typically warns (DEV) and schedules another update pass; it avoids mutating the in-progress calculation inline.
- 4) For legitimate cases (e.g., state updates in effects), updates enter queues and are processed in a later render.

Senior insight: purity is the real mechanism

The simplest way React 'manages re-entrancy' is by disallowing side effects in render. If render has no side effects, React can safely restart it, pause it, or interleave it with higher-priority work.

Misconceptions

- "Re-entrant rendering is common and fine" → it's usually a symptom of doing work in render that belongs in effects or event handlers.

Interview-ready framing

"React treats rendering as restartable and enforces render purity. Updates triggered during render are guarded/warned and scheduled for a later pass rather than corrupting the in-progress work."

143) What are React's internal object pools?

Thinking-level explanation: pooling is an optimization to reduce allocations, but it has trade-offs

Historically, React used object pooling in places like SyntheticEvent to reduce garbage creation under high-frequency events. The trade-off is developer surprise: pooled objects can be 'nullified' after use, breaking async access patterns. Modern React removed some pooling optimizations (notably SyntheticEvent pooling) because modern JS engines handle GC better and pooling confused users.

Step-by-step: what pooling means

- 1) Create an object (e.g., event wrapper).
- 2) After handler finishes, reset fields (nullify) and return object to a pool.
- 3) Next event reuses the same object instance with new data.

Senior insights: where 'pool-like' reuse still exists conceptually

Even without explicit pools, React optimizes allocations by reusing Fiber nodes via alternates (double buffering) and by keeping internal structures stable across renders. That reuse serves a similar goal: reduce allocations and improve cache locality.

Misconceptions

- "Pooling always improves performance" → it can hurt due to de-opts and complexity; modern engines often do better with fresh allocations.

Upamanyu Deka

Interview-ready framing

“Object pools are allocation-reduction optimizations (e.g., historical SyntheticEvent pooling). Modern React removed pooling where it caused confusion, while still reusing larger internal structures like fibers via alternates.”

144) How does React schedule IO and CPU-bound tasks?

Thinking-level explanation: React schedules UI work; IO belongs to effects and data layers

React’s scheduler is about CPU time for rendering UI. IO (network, storage, subscriptions) is triggered from effects or from server-side components/framework layers. The senior framing is: separate IO from render; classify updates by urgency; avoid CPU-heavy work on the main thread.

Step-by-step: CPU-bound UI work

- 1) User input creates urgent updates (high priority lanes).
- 2) Expensive derived updates are marked as transitions/deferred values (low priority).
- 3) React time-slices render work and yields to keep input responsive.

Step-by-step: IO-bound tasks

- 1) Trigger IO in useEffect (after paint) or in server components/framework loaders.
- 2) Cache results so retries don’t refetch unnecessarily.
- 3) Use Suspense boundaries to coordinate 'not ready yet' UI states (especially with streaming SSR).

Example: isolate CPU work and classify it

```
const [isPending, startTransition] = React.useTransition();

function onChange(e) {
  const next = e.target.value;
  setQuery(next); // urgent
  startTransition(() => setResults(expensiveFilter(next))); // CPU-heavy, non-urgent
}
```

Misconceptions

- “React schedules fetch requests” → React schedules rendering; fetching is done by your code/framework and then fed into React via state.

Interview-ready framing

“React schedules CPU-bound rendering via lanes + time slicing. IO should happen outside render (effects/server), then you feed results into state/Suspense—classifying resulting UI updates as urgent or transitional.”

145) How does React detect infinite render loops?

Thinking-level explanation: React detects repeated re-renders and warns/throws in DEV

Infinite render loops usually come from updating state during render or from effects that update state in a way that retriggers themselves without stabilization. React detects pathological patterns by counting nested updates / re-renders within a single render cycle and throws an error to prevent locking the browser (commonly: 'Too many re-renders').

Step-by-step: common loop patterns

- Render-time loop: component calls setState unconditionally while rendering → immediate re-render → repeat.
- Effect loop: useEffect depends on a value that changes every render (unstable deps) and sets state → triggers re-render → repeat.

Example: render-time loop

```
function Bad() {
  const [x, setX] = React.useState(0);
  setX(x + 1); // ■ during render
  return null;
}
```

Example: effect loop due to unstable dep

```
function Bad2({ value }) {
  const [s, setS] = React.useState(null);
  const obj = { value }; // new each render
  React.useEffect(() => setS(obj.value), [obj]); // ■ runs every render
  return null;
}
```

Senior mitigation

- Never set state during render; use event handlers or effects.
- Stabilize dependencies (useMemo/useCallback) or adjust deps to the real primitives.
- Use functional updates to avoid capturing stale values and reduce needless deps.

Interview-ready framing

“React prevents infinite render loops by detecting excessive re-renders/nested updates (DEV errors like 'Too many re-renders'). Root causes are usually state updates during render or effects with unstable dependencies.”

146) What are future improvements coming to React Fiber?

Thinking-level answer: focus on official directions, not speculation

A senior answer avoids random predictions and instead points to themes the React team publicly discusses: better scheduling/UX primitives, improved pre-rendering of hidden UI, and smoother transitions and hydration/streaming improvements.

Concrete directions mentioned publicly (high-level)

- More UX primitives around transitions and view transitions (to make state-to-state UI changes feel smoother).
- Better ways to pre-render or keep work off-screen without impacting visible performance (e.g., Activity/hidden rendering patterns).
- Continued investment in Server Components + streaming + selective/partial hydration patterns via frameworks.
- Tooling that reduces manual memoization burden (compiler-assisted optimizations).

Senior insight: how to answer follow-ups safely

If asked to go deeper, describe the *why*: Fiber exists to enable interruptible rendering and prioritization. Improvements generally target (1) better priority classification, (2) less wasted work, (3) better integration with modern browser APIs, and (4) improved developer ergonomics.

Misconceptions

- “Fiber will be replaced” → improvements are typically iterative: better scheduling, better primitives, and better integration—not a complete rewrite.

Interview-ready framing

“Future Fiber improvements are mostly about better scheduling and UX primitives (transitions/view transitions), more efficient offscreen/pre-render work (Activity-like patterns), and tighter integration with server-first/streaming and compiler tooling.”

147) How does React integrate with WebAssembly and Web Workers?

Thinking-level explanation: keep React on the main thread; offload heavy compute elsewhere

React’s rendering and DOM mutations happen on the main thread in the browser. Web Workers and Wasm are used to move CPU-heavy computations off the main thread, then you stream results back into React state. The integration is an architectural boundary: worker/Wasm owns computation; React owns UI.

Step-by-step integration pattern

- 1) Start a Worker (or load Wasm module) once; keep it stable (ref/singleton).
- 2) Post tasks to worker/Wasm for heavy compute (parsing, analytics, ML).
- 3) Receive results via message/event and update React state (often in a transition).
- 4) Cancel or ignore stale results when newer requests supersede old ones.

Example: worker results fed via transition

```
const workerRef = React.useRef(null);
const [isPending, startTransition] = React.useTransition();
React.useEffect(() => {
  workerRef.current = new Worker(new URL("./worker.js", import.meta.url));
  return () => workerRef.current.terminate();
}, []);

function run(input) {
  workerRef.current.postMessage(input);
}

React.useEffect(() => {
  const w = workerRef.current;
  if (!w) return;
  const onMsg = (e) => startTransition(() => setResult(e.data));
  w.addEventListener("message", onMsg);
  return () => w.removeEventListener("message", onMsg);
}, []);
```

Misconceptions

- “Workers render React off-thread” → React DOM rendering is main-thread; workers are for compute/IO.

Interview-ready framing

“Integrate by offloading heavy compute to workers/Wasm and feeding results back into React state—often as transitions—while keeping UI rendering on the main thread.”

148) How does React achieve atomic UI updates?

Thinking-level explanation: atomicity comes from commit

Upamanyu Deka

React achieves atomic UI updates by ensuring the DOM is only mutated in the commit phase once a complete consistent render result is ready. Render can be paused/restarted, but users never see partial intermediate states because nothing is committed until the snapshot is coherent.

Step-by-step

- 1) Build next UI in memory (work-in-progress fiber tree).
- 2) Compute a list of mutations (flags/effects).
- 3) Apply all DOM mutations in one commit (synchronous, not interruptible).
- 4) Update refs and run layout effects to ensure post-commit consistency.

Senior insight: avoiding tearing

Atomic commit prevents 'tearing' where some components reflect new state while others show old state. External stores need consistent snapshot APIs (useSyncExternalStore) for the same reason.

Misconceptions

- “Atomic means no intermediate visual changes ever” → within a single commit yes, but across commits you can still see progressive changes if you design multiple boundaries/commits.

Interview-ready framing

“React’s atomicity comes from the commit phase: it computes the next tree in memory and applies DOM mutations synchronously as a single consistent snapshot, preventing partial UI states.”

149) What is a React WorkLoop and how does it operate?

Thinking-level explanation: the work loop is the engine that processes units of work

The WorkLoop is React’s internal render engine: it repeatedly processes the next fiber unit of work, yielding when necessary, until the render is complete or paused. It’s the mechanism that turns 'pending lanes' into a finished work-in-progress tree ready for commit.

Step-by-step (conceptual)

- 1) Pick next root and lanes to work on (based on priority).
- 2) Set nextUnitOfWork to the root WIP fiber.
- 3) Loop: performUnitOfWork(nextUnitOfWork) → returns next fiber to process (child/sibling/parent traversal).
- 4) Periodically check shouldYield; if true, pause and schedule continuation.
- 5) When no work remains, finish render and commit if appropriate.

Pseudo-code mental model

```
while (nextUnitOfWork && !shouldYield()) {
  nextUnitOfWork = performUnitOfWork(nextUnitOfWork);
}
if (!nextUnitOfWork) commitRoot();
else scheduleContinuation();
```

Misconceptions

- “WorkLoop equals commit” → it’s primarily render/reconciliation; commit is separate and non-interruptible.

Interview-ready framing

“WorkLoop is React’s internal render engine that processes fibers as units of work, yielding when needed, producing a complete WIP tree that commit applies atomically.”

Upamanyu Deka

150) How does React manage partial hydration

Thinking-level explanation: hydrate only what needs interactivity, when it’s needed

Partial (or selective) hydration is the idea that not all server-rendered HTML needs to become interactive immediately. React + frameworks can prioritize hydration for parts the user interacts with first and defer the rest, improving Time-to-Interactive and responsiveness.

Step-by-step (conceptual)

- 1) Server renders HTML (possibly streaming) so content appears fast.
- 2) Client loads JS for interactive parts; hydration starts.
- 3) Hydration is prioritized: visible/interactive boundaries hydrate first; others can wait.
- 4) Suspense boundaries can gate hydration until code/data is ready (avoid blocking).
- 5) User interaction can trigger urgent hydration of a region (so clicks work).

Example: use boundaries to control hydration work

```
<Suspense fallback={<ShellSkeleton />}>
  <Header />

  <Suspense fallback={<SidebarSkeleton />}>
    <Sidebar />   {/* can hydrate later */}
  </Suspense>

  <Suspense fallback={<MainSkeleton />}>
    <Main />      {/* prioritize if primary interaction */}
  </Suspense>
</Suspense>
```

Misconceptions

- “Partial hydration is just SSR” → SSR is HTML generation; partial hydration is client interactivity strategy.
- “React alone does partial hydration everywhere” → frameworks typically orchestrate it using React primitives.

Interview-ready framing

“Partial hydration hydrates interactive parts first and defers the rest. React’s boundaries (Suspense) and framework orchestration enable prioritized, interaction-driven hydration while preserving atomic commits.”

Upamanyu Deka

A few extra questions and some repeated questions with better answers which were re-written after review.

Please ignore the numbering

Upamanyu Deka

61) Explain useContext hook and common pitfalls.

What the interviewer is really testing

They want to know: what problem context solves (prop drilling), how context updates propagate (render behavior), and how to avoid turning context into a re-render machine.

Thinking-level explanation

useContext reads the current value from the nearest matching Provider above in the React tree. It’s essentially a subscription: when the Provider value changes (identity), consumers re-render.

Step-by-step flow

- 1) You create a context with React.createContext(defaultValue).
- 2) A Provider supplies a value for a subtree.
- 3) useContext(Context) reads that value in a consumer.
- 4) When Provider value changes, React schedules updates for all consumers in that subtree.

Example (theme)

```
const ThemeCtx = React.createContext({ theme: "light" });

function App() {
  const [theme, setTheme] = React.useState("light");

  const value = React.useMemo(() => ({ theme, setTheme }), [theme]);

  return (
    <ThemeCtx.Provider value={value}>
      <Toolbar />
    </ThemeCtx.Provider>
  );
}

function Toolbar() {
  const { theme, setTheme } = React.useContext(ThemeCtx);
  return (
    <button onClick={() => setTheme(t => (t === "light" ? "dark" : "light"))}>
      Theme: {theme}
    </button>
  );
}
```

Common pitfalls (follow-up readiness)

- Provider value identity churn: passing a new object/function each render triggers consumer re-renders.
- Using context for high-frequency state (e.g., mouse position) can re-render large parts of the app.
- Overusing context as a 'global store' without selectors causes broad updates.

Senior solutions

- Memoize provider value (useMemo) and callbacks (useCallback).

Upamanyu Deka

- Split contexts by concern (AuthContext vs ThemeContext) to reduce blast radius.
- For complex global state, use a store with selectors to subscribe to slices.

Interview-ready framing

“useContext reads a Provider value and re-renders consumers when that value identity changes. I prevent unnecessary re-renders by memoizing provider values, splitting contexts, and using selector-based stores for high-frequency state.”

62) What is React.forwardRef and when would you use it?

What the interviewer is testing

They’re checking your understanding of imperative escape hatches and component boundaries: how to expose a DOM node (or instance) through a component that otherwise hides it.

Thinking-level explanation

By default, refs don’t pass through components. forwardRef lets a parent attach a ref to something inside the child (usually a DOM element). This is crucial for reusable UI components (Input, Button) that still need focus/scroll integration.

Example: building a reusable Input

```
const Input = React.forwardRef(function Input(props, ref) {
  return <input ref={ref} {...props} />;
});

function Example() {
  const ref = React.useRef(null);
  return (
    <div>
      <Input ref={ref} placeholder="Type..." />
      <button onClick={() => ref.current?.focus()}>Focus</button>
    </div>
  );
}
```

Misconceptions

- “forwardRef is for performance” → no, it’s for ref plumbing/imperative access.
- “Use refs instead of state” → refs are for imperative/non-UI concerns; state is for UI data.

Interview-ready framing

“I use forwardRef when building reusable components that need to expose an inner DOM node (focus/measure/scroll) to consumers without breaking component encapsulation.”

63) What is useImperativeHandle and how does it relate to forwardRef?

What the interviewer is testing

They want to know how to expose a controlled imperative API instead of leaking raw DOM nodes, and how to keep that API stable and minimal.

Thinking-level explanation

useImperativeHandle customizes what the parent receives via a ref. Instead of exposing the entire DOM node, you expose a tiny API (focus, reset, open). This maintains encapsulation and limits misuse.

Step-by-step flow

Upamanyu Deka

- 1) Child uses forwardRef to accept a ref from parent.
- 2) Child uses useImperativeHandle(ref, createHandle, deps) to define what ref.current becomes.
- 3) Parent calls methods on ref.current as an imperative escape hatch.

Example: expose a controlled API

```
const SearchBox = React.forwardRef(function SearchBox(props, ref) {
  const inputRef = React.useRef(null);

  React.useImperativeHandle(ref, () => ({
    focus() { inputRef.current?.focus(); },
    clear() { if (inputRef.current) inputRef.current.value = ""; }
  }), []);

  return <input ref={inputRef} placeholder="Search..." />;
});

function Page() {
  const ref = React.useRef(null);
  return (
    <div>
      <SearchBox ref={ref} />
      <button onClick={() => ref.current?.focus()}>Focus</button>
      <button onClick={() => ref.current?.clear()}>Clear</button>
    </div>
  );
}
```

Misconceptions / senior guidance

- Avoid exposing too much imperative surface area; it creates hidden coupling.
- Prefer declarative control via props/state; use imperative handle only when necessary (focus, scroll, integration).

Interview-ready framing

“useImperativeHandle + forwardRef lets a child expose a small imperative API to the parent, preserving encapsulation compared to exposing raw DOM nodes.”

64) When can useCallback hurt you, and how do you decide to use it?

What the interviewer is testing

They want you to understand memoization overhead and the root problem: unstable references causing unnecessary re-renders.

Thinking-level explanation

useCallback memoizes a function reference. It helps when that function is passed as a prop to memoized children or stored in dependency arrays where stability prevents rework. It can hurt when used everywhere because it adds memory/CPU overhead and makes code harder to read.

Step-by-step decision rule

- 1) Is the callback passed to a memoized child (React.memo) and causing re-renders?
- 2) Is the callback used as a dependency in effects/memos where identity changes cause re-runs?
- 3) Is the cost of recreating the function actually meaningful? (often it isn't)

Example: meaningful useCallback

```
const Row = React.memo(function Row({ onSelect }) {
  // Will re-render if onSelect identity changes
```

Upamanyu Deka

```
  return <button onClick={() => onSelect(1)}>Pick</button>;
});

function List() {
  const onSelect = React.useCallback((id) => console.log(id), []);
  return <Row onSelect={onSelect} />;
}
```

Misconceptions

- “useCallback makes things faster” → only if it prevents downstream work.
- “Functions recreated each render are expensive” → usually cheap; the expensive part is child renders/DOM work.

Interview-ready framing

“I use useCallback when a stable function reference prevents real re-renders or effect re-runs. Otherwise I avoid it to keep code simpler.”

65) How do you profile performance in React (React DevTools Profiler)?

What the interviewer is testing

They want a disciplined approach: measure → identify → fix → re-measure, and knowing the difference between commit time and render frequency.

Step-by-step playbook

- 1) Record interactions in React DevTools Profiler.
- 2) Identify components with high render time or frequent re-renders.
- 3) Determine why they re-render (props changes, context changes, parent renders).
- 4) Apply targeted fixes (memoization, state scoping, virtualization, code splitting).
- 5) Re-profile to confirm improvement.

What to look for (signals)

- Frequent commits during typing/scrolling → look for state lifted too high or context churn.
- Single commit with large duration → expensive rendering or large DOM tree.
- Components re-rendering without visible UI change → unstable props or unnecessary renders.

Interview-ready framing

“I use Profiler to find hot components and why they re-render, then fix the cause (state scope, memoization, stable props, virtualization) and re-measure to ensure it’s real.”

66) Explain startTransition and useTransition (React 18).

What the interviewer is really testing

They want to know: urgent vs non-urgent updates, keeping inputs responsive, and that transitions are interruptible.

Thinking-level explanation

Transitions mark updates as non-urgent. React can keep urgent updates (typing/click feedback) responsive while rendering slower UI updates in the background. If the user triggers new urgent work, transition work can be interrupted and restarted.

Step-by-step flow

- 1) User triggers an action that causes heavy UI work (filtering a big list).
- 2) Wrap the non-urgent state update in startTransition (or useTransition).
- 3) React schedules it in a lower-priority lane.
- 4) Urgent updates (input typing) remain high priority and stay responsive.
- 5) If new updates arrive, React can abandon the in-progress transition render and restart with latest state.

Example

```
function SearchPage({ items }) {
  const [query, setQuery] = React.useState("");
  const [filtered, setFiltered] = React.useState(items);
  const [isPending, startTransition] = React.useTransition();

  const onChange = (e) => {
    const q = e.target.value;
    setQuery(q); // urgent (keeps input responsive)

    startTransition(() => {
      // non-urgent (may be expensive)
      setFiltered(items.filter(x => x.name.includes(q)));
    });
  };

  return (
    <div>
      <input value={query} onChange={onChange} />
      {isPending && <p>Updating results…</p>}
      {filtered.map(x => <div key={x.id}>{x.name}</div>)}
    </div>
  );
}
```

Misconceptions

- “Transitions make code run on another thread” → no; it’s still main-thread, but scheduled with lower priority.
- “Transitions guarantee faster work” → they improve responsiveness, not raw compute speed.

Interview-ready framing

“Transitions mark non-urgent updates so React can prioritize responsiveness. Rendering can be interrupted/restarted to avoid input lag when heavy UI updates happen.”

67) Where do you place Suspense boundaries, and why?

Thinking-level explanation

Suspense boundaries are UX decisions: they define what portion of the UI can 'fallback' while waiting. The goal is to avoid blanking the whole screen and to enable progressive rendering.

Step-by-step strategy (senior)

- 1) Put a coarse boundary at route/page level to avoid a fully blank page on initial load.
- 2) Add finer boundaries around slow widgets (charts, editors) to allow the rest of the page to show.
- 3) Choose fallbacks that match layout to avoid jank/CLS (skeletons instead of spinners).

Example: progressive boundaries

```
<Suspense fallback={<PageSkeleton />}>
  <Header />
```

Upamanyu Deka

```
    <Suspense fallback={<SidebarSkeleton />}>
      <Sidebar />
    </Suspense>
    <Suspense fallback={<MainSkeleton />}>
      <MainContent />
    </Suspense>
  </Suspense>
```

Misconceptions

- “One Suspense for everything” → can cause large blank areas and poor perceived performance.
- “Fallback always replaces the whole page” → only replaces the suspended subtree.

Interview-ready framing

“I place Suspense boundaries to control UX: route-level for safety, component-level for progressive rendering, and I use skeleton fallbacks to preserve layout and avoid jank.”

68) How do you avoid Context API causing excessive re-renders?

What the interviewer is testing

They want to see if you understand that context updates are broadcast to consumers, and how to reduce the blast radius.

Common cause

Provider value identity changes (new object/function each render) causes consumers to re-render even if the 'data' looks the same.

Step-by-step fixes

- 1) Memoize provider value with useMemo.
- 2) Stabilize callbacks with useCallback.
- 3) Split contexts by concern (don’t put everything into one mega context).
- 4) Move high-frequency state out of context; use local state or a store with selectors.

Example

```
const Ctx = React.createContext(null);

function Provider({ children }) {
  const [count, setCount] = React.useState(0);
  const inc = React.useCallback(() => setCount(c => c + 1), []);
  const value = React.useMemo(() => ({ count, inc }), [count, inc]);
  return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
}
```

Senior insight

Context is best for shared configuration-like data. If you need frequent global updates + selective subscriptions, a store is often a better fit.

Interview-ready framing

“To avoid context re-render storms, I memoize provider values, split contexts, and avoid putting high-frequency state into context; for complex cases I use selector-based stores.”

69) What causes hydration mismatch and how do you debug it?

What the interviewer is testing

They want to know that hydration requires deterministic server+client output, and how to handle non-determinism and data consistency.

Common causes (real world)

- Non-deterministic render output: Date.now(), Math.random(), locale-dependent formatting.
- Different data on server vs client at first render (e.g., reading localStorage only on client).
- Conditional rendering that depends on 'window' during render.
- Different HTML structure due to feature flags or A/B tests not aligned between server and client.

Step-by-step debugging approach

- 1) Locate the component subtree that mismatches (console warnings often include hints).
- 2) Check for non-deterministic values during render; move them to useEffect or useMemo with stable inputs.
- 3) Ensure server and client use the same initial data (serialize/rehydrate).
- 4) Guard client-only logic: render placeholders on server, then switch on client in effect.

Example: window/localStorage safe pattern

```
function ClientOnly() {
  const [ready, setReady] = React.useState(false);
  React.useEffect(() => setReady(true), []);
  if (!ready) return null; // or a placeholder consistent with SSR
  return <div>Client-only content</div>;
}
```

Interview-ready framing

“Hydration mismatch happens when server HTML doesn’t match the client’s initial render. I remove non-determinism from render, align initial data, and isolate client-only logic behind effects/placeholders.”

70) Server Components vs Client Components — what’s the difference (and why it matters)?

What the interviewer is testing

They want to see if you understand modern React ecosystems: what runs where, how it affects bundle size, data fetching, and interactivity boundaries.

Thinking-level explanation

Client components run in the browser, can use state/effects, and handle interactivity. Server components render on the server, can access server resources (DB, secrets), and don’t ship their code to the client—reducing bundle size. The key is drawing the right boundary: keep heavy data/formatting on the server, keep interactive bits on the client.

Step-by-step mental model

- 1) Server components render on the server and produce a serialized output that the client can compose.
- 2) Client components are the interactive islands: they hydrate and handle events.
- 3) Data fetching can move earlier (server) to reduce client waterfalls.
- 4) The boundary affects what code ships to the browser and what can use hooks like useState/useEffect.

Misconceptions

- “Server components replace SSR” → they are different concepts; SSR is about HTML generation, server components are about component execution and bundle splitting.
- “Server components can use any hook” → they cannot use client-only hooks like useState/useEffect.

Interview-ready framing

“Client components handle interactivity; server components keep non-interactive rendering and data access on the server to reduce bundle size and client waterfalls. The main skill is choosing boundaries that optimize UX and maintainability.”

71) useEffect vs useLayoutEffect vs useInsertionEffect — when and why?

What the interviewer is testing

They're testing whether you understand the render→commit→paint timing model, and how to avoid flicker, layout thrash, and CSS-in-JS ordering problems.

Thinking-level timing model

- Render phase: compute next UI (no DOM mutations).
- Commit phase: apply DOM mutations.
- Before paint: layout effects run (useLayoutEffect).
- After paint: passive effects run (useEffect).
- Before layout effects (special): useInsertionEffect (primarily for CSS-in-JS to inject styles before layout).

When to use each

- useEffect: fetching, subscriptions, timers, logging — things that don't need to block paint.
- useLayoutEffect: DOM measurement + synchronous DOM writes that must happen before the user sees the frame (avoid flicker).
- useInsertionEffect: library-level hook for injecting styles before layout to prevent incorrect measurements or flashes; rarely used in app code.

Example: measuring layout without flicker

```
function Tooltip({ anchorRef }) {
  const [pos, setPos] = React.useState({ top: 0, left: 0 });

  React.useLayoutEffect(() => {
    const rect = anchorRef.current?.getBoundingClientRect();
    if (rect) setPos({ top: rect.bottom + 8, left: rect.left });
  }, [anchorRef]);

  return <div style={{ position: "fixed", top: pos.top, left: pos.left }}>Tip</div>;
}
```

Misconceptions

- "useLayoutEffect is always better" → it blocks paint; overuse hurts responsiveness.
- "useInsertionEffect is for animations" → it's mainly for style injection libraries.

Interview-ready framing

"I use useEffect for non-visual side effects, useLayoutEffect for layout-critical reads/writes to avoid flicker, and I treat useInsertionEffect as a library hook mainly for CSS-in-JS style injection ordering."

72) What is a stale closure in hooks, and how do you fix it?

Thinking-level explanation

Upamanyu Deka

A stale closure happens when a function (effect/callback) captures old values from a previous render. Because function components re-run per render, closures may refer to outdated state/props if dependencies aren't handled correctly.

Classic example (timer reads old state)

```
function Counter() {
  const [count, setCount] = React.useState(0);

  React.useEffect(() => {
    const id = setInterval(() => {
      // ■ count is 'stale' if not in deps
      console.log(count);
    }, 1000);
    return () => clearInterval(id);
  }, []);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Fix strategies (choose the right one)

- Add correct dependencies so the effect re-subscribes when values change (but watch for loops).
- Use functional state updates (setX(prev => ...)) to avoid relying on captured state.
- Use refs to store the latest value for event-like callbacks without re-subscribing.

Example: ref-based latest value

```
function Counter() {
  const [count, setCount] = React.useState(0);
  const countRef = React.useRef(count);

  React.useEffect(() => { countRef.current = count; }, [count]);

  React.useEffect(() => {
    const id = setInterval(() => console.log(countRef.current), 1000);
    return () => clearInterval(id);
  }, []);

  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}
```

Misconceptions

- "Just disable eslint exhaustive-deps" → that hides bugs; you should fix the cause.
- "Adding deps always causes loops" → loops come from effects that update their own deps; restructure logic.

Interview-ready framing

"Stale closures occur when effects/callbacks capture outdated render values. I fix them by correct dependency arrays, functional updates, or refs for latest values when re-subscribing is undesirable."

73) How do you avoid infinite loops in useEffect?

Root cause (thinking-level)

Infinite loops usually happen when an effect updates state that is listed in its dependencies, so state changes trigger the effect again. The fix is to separate 'derive' vs 'sync', and avoid storing derived state.

Step-by-step debug approach

Upamanyu Deka

- 1) Identify which dependency changes are re-triggering the effect (console/log).
- 2) Determine whether the state update inside the effect is necessary or derived.
- 3) If derived, compute it during render (or useMemo) instead of storing it.
- 4) If the effect must update state, guard it with conditions or move to an event handler.

Example: derived state mistake

```
// ■ anti-pattern: storing derived state
const [fullName, setFullName] = useState("");
useEffect(() => {
  setFullName(first + " " + last);
}, [first, last]);
```

Better approach

```
// ■ derive during render
const fullName = React.useMemo(() => first + " " + last, [first, last]);
```

Misconceptions

- “Empty deps fixes loops” → it hides correct synchronization; might create stale bugs.
- “useEffect is for derived values” → effects are for syncing external systems, not deriving internal values.

Interview-ready framing

“I avoid effect loops by not storing derived state, restructuring effects to sync external systems, and guarding state updates so dependencies don’t self-trigger unnecessarily.”

74) Explain React’s rendering phases and scheduling (render, commit, effects).

What the interviewer is testing

They want a solid mental model for when code runs, what can be interrupted, and how effects fit into commit+paint. This is core for Fiber/concurrency discussions.

Step-by-step pipeline

- 1) Schedule: an update is enqueued with a priority (lane).
- 2) Render (reconciliation): React computes the next UI tree (can be paused/resumed in concurrent rendering). No DOM mutations.
- 3) Commit: React applies DOM mutations atomically (not interruptible).
- 4) Layout effects: useLayoutEffect runs before paint; can read/write layout.
- 5) Paint: browser renders the frame.
- 6) Passive effects: useEffect runs after paint.

Senior insight: why separate render and commit?

Separation allows React to do expensive computation without touching the DOM, then apply a minimal set of DOM writes in one atomic commit, enabling time slicing and responsiveness without tearing.

Misconceptions

- “Rendering means updating the DOM” → render computes; commit mutates DOM.
- “Effects run during render” → effects run after commit (layout) or after paint (passive).

Interview-ready framing

“React schedules work, renders to compute the next tree (interruptible), commits DOM changes atomically, then runs layout effects before paint and passive effects after paint.”

Upamanyu Deka

75) Why does immutability matter in React state updates?

Thinking-level explanation

React relies heavily on referential equality to detect changes cheaply (especially for memoization and PureComponent patterns). If you mutate objects/arrays in place, references don't change and shallow comparisons can't detect updates correctly.

Concrete problems caused by mutation

- Memoized children don't re-render because props reference didn't change.
- State updates can be ignored or lead to confusing bugs when React batches updates.
- Time-travel/debugging becomes impossible because past state was mutated.

Example: mutation bug

```
const [items, setItems] = React.useState([1,2,3]);

function addBad() {
  items.push(4);      // ■ mutates in place
  setItems(items);    // same reference -> memoization can break
}

function addGood() {
  setItems(prev => [...prev, 4]); // ■ new reference
}
```

Senior insight: immutability enables optimization

Immutability isn't only a style preference—it enables cheap change detection, predictable updates, and optimization techniques (memoization, selectors).

Interview-ready framing

"Immutability makes change detection reliable. React and memoization rely on reference changes; mutating state breaks shallow comparisons and can cause skipped renders or incorrect UI."

76) What is useId and what problem does it solve?

Thinking-level explanation

useId generates stable unique IDs that are consistent across server and client renders. Its main purpose is to avoid hydration mismatches when generating IDs for accessibility attributes (label-for, aria-describedby).

Example: accessible input

```
function Field({ label }) {
  const id = React.useId();
  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} aria-describedby={id + "-help"} />
      <small id={id + "-help"}>Helper text</small>
    </div>
  );
}
```

Misconceptions

- "useId is for list keys" → don't use it for keys; keys should come from data identity.

- “I can replace Math.random with useId everywhere” → it’s for stable SSR-safe IDs, mainly a11y wiring.

Interview-ready framing

“useId provides SSR-safe stable IDs to wire accessibility attributes without hydration mismatches. It’s not meant for list keys.”

77) What is useSyncExternalStore and why does it exist?

What the interviewer is testing

They want to see if you understand external stores (Redux/Zustand/custom stores) and how concurrency requires consistent snapshots to avoid tearing.

Thinking-level explanation

useSyncExternalStore is the official hook for subscribing to external mutable stores in a way that is compatible with concurrent rendering. It ensures React reads a consistent snapshot and can re-render safely without showing inconsistent state (tearing).

Step-by-step model

- 1) Provide a subscribe function (how to listen for store changes).
- 2) Provide a getSnapshot function (how to read current store state).
- 3) React uses these to get a consistent snapshot during render.
- 4) When store changes, React schedules updates; in concurrent mode it can re-check snapshots to ensure consistency.

Minimal example

```
function createStore() {
  let value = 0;
  const listeners = new Set();
  return {
    getSnapshot: () => value,
    subscribe: (cb) => { listeners.add(cb); return () => listeners.delete(cb); },
    inc: () => { value += 1; listeners.forEach(l => l()); }
  };
}
const store = createStore();

function Counter() {
  const value = React.useSyncExternalStore(store.subscribe, store.getSnapshot);
  return <button onClick={store.inc}>Value: {value}</button>;
}
```

Misconceptions

- “It’s just a nicer useEffect subscription” → no, it’s designed for concurrent correctness.
- “Only Redux needs it” → any external mutable store subscription can benefit.

Interview-ready framing

“useSyncExternalStore is the concurrency-safe way to subscribe to external stores. It prevents tearing by letting React read consistent snapshots during concurrent rendering.”

78) Server state vs UI state — how do you manage each in React apps?

Thinking-level explanation

Upamanyu Deka

UI state is local and owned by the client (modals, filters, form inputs). Server state is remote and has a lifecycle: fetching, caching, invalidation, retries, background refresh. Treating server state like local state leads to duplication and bugs.

Decision model (senior)

- UI state: useState/useReducer/context/store depending on scope.
- Server state: use a data fetching cache (React Query/SWR/framework caching) for dedupe, caching, and invalidation.
- Keep them separate: derive UI from server state but don't duplicate server data into global UI stores unless necessary.

Example: server state + UI state together

```
function Users() {
  const [query, setQuery] = React.useState("");
  // Pseudo: const { data, isLoading } = useQuery(["users", query], () => fetchUsers(query));
  // Here we show the pattern conceptually:
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      {/* render data + loading states */}
    </div>
  );
}
```

Misconceptions

- "Redux is required for server data" → modern apps often use query caches for server state.
- "Cache means stale data always" → caches can refetch/invalidate; you control freshness.

Interview-ready framing

"I separate UI state from server state. UI state stays local; server state lives in a cache layer with invalidation/retries/deduping. This reduces duplication and makes data flows predictable."

79) Memoization vs virtualization — when do you use each?

Thinking-level explanation

Memoization reduces re-computation/re-rendering when inputs are stable. Virtualization reduces the amount of UI you render at all by only mounting visible items. They solve different bottlenecks.

When memoization is the right tool

- You have expensive computations or expensive subtrees that re-render due to unrelated parent updates.
- Props are stable enough for shallow comparison to bail out.

When virtualization is the right tool

- You have large lists/tables (hundreds/thousands of rows).
- The bottleneck is DOM size/layout/paint, not just JS computation.

Senior insight: combine carefully

Often the biggest win for huge lists is virtualization. Memoization may help inside row components, but virtualization usually dominates by shrinking DOM work.

Interview-ready framing

“Memoization skips repeated work when inputs are stable; virtualization avoids rendering most items entirely. For large lists, virtualization is often the primary win.”

80) How do you prevent unnecessary re-renders in React (holistic answer)?

What the interviewer is testing

They want a system-level approach: state placement, stable references, memoization, context strategy, and measuring with the Profiler.

Step-by-step playbook

- 1) Keep state as local as possible; avoid lifting state too high.
- 2) Split components so changes don’t re-render unrelated UI.
- 3) Stabilize props: avoid inline object/function creation when it matters (useMemo/useCallback).
- 4) Use React.memo for expensive children with stable props.
- 5) Avoid context churn: memoize provider values and split contexts.
- 6) For large lists, virtualize.
- 7) Profile and verify improvements.

Example: unstable vs stable props

```
// ■ unstable: new object each render
<Row style={{ padding: 8 }} onSelect={() => select(id)} />

// ■ stable: memoize
const style = React.useMemo(() => ({ padding: 8 }), []);
const onSelect = React.useCallback(() => select(id), [id]);
<Row style={style} onSelect={onSelect} />
```

Misconceptions

- “Prevent all re-renders” → re-rendering is normal; you prevent wasteful re-renders that cost measurable time.
- “memo fixes everything” → memo only helps when props are stable; otherwise it’s noise.

Interview-ready framing

“I prevent unnecessary re-renders by scoping state correctly, stabilizing props, memoizing expensive subtrees, controlling context updates, virtualizing large lists, and validating with the Profiler.”

81) Controlled vs uncontrolled components — deeper trade-offs and real-world pitfalls.

What the interviewer is actually probing

They want more than definitions: how you choose patterns under constraints (validation, performance, UX), and what bugs appear when you mix approaches.

Decision lens (source of truth + UX requirements)

- If UI depends on the input value in real-time (validation, enabling/disabling buttons, live formatting) → controlled.
- If you only need the value at submit and want fewer re-renders in huge forms → uncontrolled/ref-based forms.
- If you must preserve input state across conditional rendering → avoid unmounting; consider keeping mounted or storing value.

Pitfall: mixing controlled and uncontrolled

React warns when an input switches between controlled and uncontrolled (value undefined → defined). This often happens when you render before data loads.

```
function Email({ initial }) {
  // ■ initial may be undefined initially -> uncontrolled, then controlled
  const [v, setV] = React.useState(initial);
  return <input value={v} onChange={e => setV(e.target.value)} />;
}

// ■ ensure a string
const [v, setV] = React.useState(initial ?? "");
```

Performance insight (senior)

Controlled inputs can re-render on every keystroke; usually fine. But in giant forms, re-render blast radius matters. Split fields into memoized components or use libraries that minimize re-renders.

Interview-ready framing

“Controlled = React is the source of truth; great for validation and dynamic UI. Uncontrolled = DOM is the source of truth; great for simpler or very large forms. I avoid switching modes and manage performance by scoping state and memoizing field components.”

82) Why do keys matter, and what happens when keys are wrong?

Thinking-level explanation: identity + state preservation

Keys are how React preserves identity across renders. Identity determines whether React updates a component in place (preserving state) or destroys and recreates it (resetting state).

Step-by-step: what React does in a list

- 1) Old children are indexed by key (and type).

Upamanyu Deka

- 2) New children are matched by key to reuse existing fibers/DOM.
- 3) If keys change or collide, React may reuse the wrong component instance or remount items.

Real-world bug: inputs jump rows with index keys

```
function List({ items }) {
  return items.map((item, idx) => (
    <input key={idx} defaultValue={item.name} />
  ));
}
// Insert at top -> values appear to 'move' because identity was index-based.
```

Senior insight

Keys are not just for performance; they are correctness. Wrong keys cause state mis-association. Use stable IDs from data.

Interview-ready framing

“Keys define identity in reconciliation. Stable keys preserve state; wrong keys (like indices with reorder) can attach state/DOM to the wrong item.”

83) React.memo vs PureComponent vs shouldComponentUpdate — how do they differ?

Thinking-level: all are bailouts, with different ergonomics

All three are ways to skip re-render work when inputs haven’t meaningfully changed. The key is understanding what comparison is used and what you’re optimizing (component render execution vs DOM writes).

React.memo (function components)

- Wraps a function component and skips re-render when props are shallow-equal.
- You can provide a custom comparator, but be careful with deep compares (costly).

PureComponent (class components)

- Implements a shallow prop/state comparison in shouldComponentUpdate.
- Legacy but common in older codebases.

shouldComponentUpdate (class components)

- Manual control to decide updates; can be precise but easy to get wrong.
- If you return false incorrectly, UI can get stale.

Example (React.memo + stable props)

```
const Item = React.memo(function Item({ user }) {
  return <div>{user.name}</div>;
});

// Works best when user reference is stable via immutability.
```

Misconceptions

- “Memo prevents all re-renders” → it prevents re-render of that component when props are equal; parent still re-renders.
- “Custom comparator is always better” → deep comparisons can cost more than rendering.

Interview-ready framing

“They’re all bailout mechanisms. React.memo is for function components, PureComponent is the class equivalent, and shouldComponentUpdate gives manual control. They rely on immutability and stable references to work well.”

84) When should you use useMemo, and when is it a footgun?

Thinking-level explanation

useMemo memoizes a computed value between renders. It helps when computation is expensive and inputs are stable. It becomes a footgun when used everywhere—adding overhead and complexity without reducing real work.

Decision checklist

- Is the computation expensive (measured) and runs often?
- Does memoization reduce downstream renders by keeping object identity stable?
- Are dependencies stable and correct? (wrong deps = stale bugs)

Example: stabilize object prop to memoized child

```
const Chart = React.memo(({ options }) => { /* ... */ return null; });

function Page({ theme }) {
  const options = React.useMemo(() => ({ theme, smooth: true }), [theme]);
  return <Chart options={options} />;
}
```

Footgun example: memoizing cheap values

Memoizing a simple string concatenation is often worse than computing it.

Misconceptions

- “useMemo guarantees performance” → it’s a hint with overhead; only helps if it prevents meaningful work.
- “useMemo prevents rerenders” → only indirectly, by stabilizing references for memoized children.

Interview-ready framing

“I use useMemo when it avoids expensive recomputation or stabilizes references for memoized children. I avoid it for cheap work because it adds overhead and can introduce stale-deps bugs.”

85) What is useDeferredValue and when would you use it?

What the interviewer is testing

They want to see if you understand 'defer rendering of derived UI' as a concurrency feature, and how it differs from debouncing.

Thinking-level explanation

useDeferredValue lets you render with a deferred version of a value so the UI stays responsive. Instead of blocking the UI while expensive derived rendering happens, React can show the latest input immediately while rendering derived UI slightly behind.

Step-by-step flow

- 1) User types quickly; query state updates urgently.
- 2) deferredQuery lags behind query under load.
- 3) Expensive list renders using deferredQuery, so typing remains smooth.

Upamanyu Deka

- 4) When React has time, deferredQuery catches up.

Example

```
function Search({ items }) {
  const [q, setQ] = React.useState("");
  const dq = React.useDeferredValue(q);

  const filtered = React.useMemo(
    () => items.filter(x => x.name.includes(dq)),
    [items, dq]
  );

  return (
    <div>
      <input value={q} onChange={e => setQ(e.target.value)} />
      {filtered.map(x => <div key={x.id}>{x.name}</div>)}
    </div>
  );
}
```

Misconceptions

- “It’s the same as debounce” → debounce delays updates by time; deferred value delays by rendering priority/workload.
- “It makes filtering faster” → it makes typing responsive; compute cost is still there.

Interview-ready framing

“useDeferredValue keeps the UI responsive by letting derived, expensive renders lag behind urgent input updates. It’s priority-based, not time-based like debounce.”

86) How do Suspense and transitions work together (avoid jarring fallbacks)?

Thinking-level explanation

Transitions mark updates as non-urgent; Suspense can show fallbacks when data/code isn’t ready. Together, they prevent the UI from suddenly switching to a loading state for minor interactions.

Step-by-step pattern

- 1) User triggers navigation/filter that may suspend.
- 2) Wrap the state update in startTransition.
- 3) React keeps showing the previous UI while rendering the next UI in the background.
- 4) If it suspends, fallback may be delayed or shown in a controlled boundary (depends on setup).

Example (conceptual)

```
function Tabs() {
  const [tab, setTab] = React.useState("home");
  const [isPending, startTransition] = React.useTransition();

  return (
    <>
      <button onClick={() => startTransition(() => setTab("home"))}>Home</button>
      <button onClick={() => startTransition(() => setTab("reports"))}>Reports</button>

      {isPending && <span>Updating…</span>}
      <React.Suspense fallback={<div>Loading tab…</div>}>
        {tab === "home" ? <Home /> : <Reports />}
      </React.Suspense>
    </>
```

Upamanyu Deka

```
  );
}
```

Senior insight

Boundary placement is a UX decision. Use multiple Suspense boundaries so only the slow piece shows fallback, not the whole page.

Interview-ready framing

“Transitions keep interactions responsive by marking work as non-urgent; Suspense handles 'not ready' subtrees. Together they avoid jarring loading states by letting React keep the current UI while preparing the next.”

87) How do you avoid race conditions in useEffect data fetching?

What the interviewer is testing

They want you to handle 'latest request wins' and avoid setting state from stale responses when dependencies change quickly.

Step-by-step safe pattern

- 1) Start a request when deps change (e.g., query).
- 2) Track a request id or use AbortController.
- 3) On response, only apply if it’s still the latest (or not aborted).
- 4) Cleanup aborts previous requests on dep change/unmount.

AbortController example

```
function SearchUsers({ q }) {
  const [data, setData] = React.useState(null);

  React.useEffect(() => {
    const c = new AbortController();
    (async () => {
      const res = await fetch(`/api/users?q=${encodeURIComponent(q)}`, { signal: c.signal });
      const json = await res.json();
      setData(json);
    })().catch(e => {
      if (e.name !== "AbortError") console.error(e);
    });

    return () => c.abort();
  }, [q]);

  return <pre>{JSON.stringify(data, null, 2)}</pre>;
}
```

Misconceptions

- “Just put q in deps and it’s safe” → deps correctness doesn’t prevent stale responses.
- “Canceling isn’t needed” → it prevents setting state after unmount and handles out-of-order responses.

Interview-ready framing

“I avoid races by aborting previous requests or using a request-id guard so only the latest response updates state, and I always clean up on dep change/unmount.”

88) Why does React prefer composition over inheritance?

Upamanyu Deka

Thinking-level explanation

React’s component model is built around composing UI pieces and sharing behavior via props/children and hooks. Inheritance creates tight coupling, hidden behavior, and fragile hierarchies; composition keeps dependencies explicit.

How composition solves reuse

- UI reuse: pass children or render props for layout wrappers.
- Logic reuse: custom hooks encapsulate stateful behavior without UI coupling.
- Behavior extension: higher-order components/render props exist but hooks are often cleaner.

Example: composition with children

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}
<Card><Profile /></Card>
```

Interview-ready framing

“Composition keeps reuse explicit and flexible—UI via children/props and logic via hooks—while inheritance tends to create rigid hierarchies and hidden coupling.”

89) State colocation vs lifting state — how do you decide?

Senior decision rule: smallest common owner

Keep state as close as possible to where it’s used. Lift only when multiple components need to read/write it. The goal is to minimize re-render blast radius and keep data flow predictable.

Step-by-step decision process

- 1) Who needs to read this state?
- 2) Who needs to update it?
- 3) Find the smallest common ancestor that satisfies both.
- 4) Lift to that ancestor; pass data down and actions up.
- 5) If many levels deep or many consumers: consider context/store.

Example

```
function Parent() {
  const [query, setQuery] = React.useState("");
  return (
    <>
      <SearchBox value={query} onChange={setQuery} />
      <Results query={query} />
    </>
  );
}
```

Misconceptions

- “Lift everything to global state for simplicity” → increases coupling and rerenders.
- “Keep everything local” → breaks coordination across siblings/features.

Interview-ready framing

“I colocate state by default and lift it to the smallest common owner only when multiple components need to coordinate on it.”

Upamanyu Deka

90) How do you avoid prop drilling (and when is it okay)?

Thinking-level explanation

Prop drilling is a trade-off: explicit data flow vs ergonomics. It becomes a problem when intermediate layers don't care about the props and changes become noisy or fragile.

Options (with trade-offs)

- Context: good for app-wide/shared values (theme/auth). Beware provider churn and large consumer re-renders.
- Component composition: pass children/components rather than threading props.
- Store with selectors: for complex global state with fine-grained subscriptions.
- Custom hooks: encapsulate access to context/store so call sites stay clean.

Example: composition to avoid drilling

```
function Layout({ header, children }) {
  return (
    <div>
      <div className="header">{header}</div>
      <main>{children}</main>
    </div>
  );
}

// Pass the component instead of drilling props through Layout
<Layout header={<UserMenu />}>
  <Dashboard />
</Layout>
```

Senior guidance

A little prop drilling is okay—especially within a feature—because it keeps dependencies explicit. I reach for context/stores when drilling becomes pervasive or brittle.

Interview-ready framing

“I tolerate small prop chains for clarity. When it becomes noisy, I use context for shared concerns, composition to pass UI, and stores/selectors for complex global state—always watching for re-render impact.”

Upamanyu Deka